The Art and Science of Digital Compositing

Free software included on CD-ROM

Ron Brinkmann

The Art and Science of Digital Compositing

Limited Warranty and Disclaimer of Liability

Academic Press ("AP") and anyone else who has been involved in the creation or production of the accompanying code ("the product") cannot and do not warrant the performance or results that may be obtained by using the product. The product is sold "as is" without warranty of any kind (except as hereafter described), either expressed or implied, including, but not limited to, any warranty of performance or any implied warranty of merchantability or fitness for any particular purpose. AP warrants only that the magnetic diskette(s) on which the code is recorded is free from defects in material and faulty workmanship under the normal use and service for a period of ninety (90) days from the date the product is delivered. The purchaser's sole and exclusive remedy in the event of a defect is expressly limited to either replacement of the diskette(s) or refund of the purchase price, at AP's sole discretion.

In no event, whether as a result of breach of contract, warranty or tort (including negligence), will AP or anyone who has been involved in the creation or production of the product be liable to purchaser for any damages, including any lost profits, lost savings or other incidental or consequential damages arising out of the use or inability to use the product or any modifications thereof, or due to the contents of the code, even if AP has been advised of the possibility of such damages, or for any claim by any other party.

Any request for replacement of a defective diskette must be postage prepaid and must be accompanied by the original defective diskette, your mailing address and telephone number, and proof of date of purchase and purchase price. Send such requests, stating the nature of the problem, to Academic Press Customer Service, 6277 Sea Harbor Drive, Orlando, FL 32887, 1-800-321-5068. AP shall have no obligation to refund the purchase price or to replace a diskette based on claims of defects in the nature or operation of the product.

Some states do not allow limitation on how long an implied warranty lasts, nor exclusions or limitations of incidental or consequential damage, so the above limitations and exclusions may not apply to you. This warranty gives you specific legal rights, and you may also have other rights which vary from jurisdiction to jurisdiction.

The re-export of United States origin software is subject to the United States laws under the Export Administration Act of 1969 as amended. Any further sale of the product shall be in compliance with the United States Department of Commerce Administration Regulations. Compliance with such regulations is your responsibility and not the responsibility of AP.

The Art and Science of Digital Compositing

Ron Brinkmann

Morgan Kaufmann

Cover image: "The Two Ways of Life" by Oscar Gustav Rejlander. Image courtesy of The Royal Photographic Society Collection, Bath, England. Web site http://www.rps.org.

This book is printed on acid-free paper. ♾

Copyright © 1999 by Elsevier.

All rights reserved.
No part of this publication may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopy, recording, or any information storage and retrieval system, without permission in writing from the publisher.

All brand names and product names are trademarks or registered trademarks of their respective companies.

ACADEMIC PRESS
An imprint of Elsevier
525 B Street, Suite 1900, San Diego, CA 92101-4495 USA
http://www.academicpress.com

ACADEMIC PRESS
24-28 Oval Road, London NW1 7DX United Kingdom
http://www.hbuk.co.uk/ap/

MORGAN KAUFMANN
340 Pine Street, 6th Floor, San Francisco, CA 94104-3205 USA
http://www.mkp.com

Library of Congress Catalog Number: 99-60086
International Standard Book Number: 0-12-133960-2

Printed in the United States of America
03 04 IP 9 8 7 6 5 4

For Mom and Dad,
of course.

Contents

Acknowledgments *xv*
Preface *xvii*

Chapter 1 Introduction to Digital Compositing *1*
Definition ... 2
Historical Perspective ... 4
Terminology ... 7
Organization of the Book ... 8

Chapter 2 The Digital Representation of Visual Information *13*
Image Generation ... 13
Pixels, Components, and Channels ... 15
Spatial Resolution ... 17
Bit Depth ... 18
Normalized Values ... 20
Additional Channels ... 20
The HSV Color Representation ... 21
Image Input Devices ... 22
Digital Image File Formats ... 23
File Format Features ... 23
Vendor Implementations of File Formats ... 24
Compression ... 24
Choosing a File Format ... 31
Nonlinear Color Spaces ... 32

Chapter 3 Basic Image Manipulation *33*
Terminology ... 34
Color Manipulations ... 36
RGB Multiply ... 36
Add ... 36

Gamma Correction 38
Invert 39
Contrast 40
Channel Swapping 41
HSV Manipulations 42
Look-up Table Manipulations 43
Expression Language 44
Spatial Filters 46
Convolves 46
Blurring 47
Sharpen 50
Median Filter 51
Geometric Transformations 52
Panning 53
Rotation 55
Scale 56
3D Transforms 56
Warping 58
Expression Language 58
Filtering Algorithms 60

Chapter 4 Basic Image Compositing 65
The Matte Image 66
The Integrated Matte Channel 67
Multisource Operators 68
Over 69
Mix 71
Subtract 71
In 72
Out 72
Atop 73
Masks 74
Compositing With Premultiplied Images 74
Color-Correcting and Combining Premultiplied Images 75
Luminosity 77

Chapter 5 Matte Creation and Manipulation 79
Procedural Matte Extraction 80
Keying Based on Luminance 81
Keying Based on Chrominance 81

Difference Matting 82
The Color Difference Method 84
Specialized Keying Software 85
Matting Techniques 86
Garbage Mattes 86
Edge Mattes 88
Combining Mattes 89
Manipulating Mattes 89

Chapter 6 Time and Temporal Manipulations 93
Apparent Motion 93
Temporal Resolution 94
Temporal Artifacts 95
Changing the Length or Timing of a Sequence 96
Keyframing 99

Chapter 7 Image Tracking and Stabilization 103
Tracking an Element Into a Plate 104
Choosing the Feature to Track 107
Limiting the Search Area 109
Human Intervention 109
Using Tracking Curves Manually 110
Stabilizing a Plate 111
Tracking Multiple Points 113

Chapter 8 Interface Interactions 115
Workflow 116
Online Versus Batch 118
Methods of Representing the Compositing Process 121
Compressed Trees 123
Timelines 124
Curve Editors 126
Working With Proxy Images 130

Chapter 9 Image Viewing and Analysis Tools 133
Image Viewers 134
Flipbooks 134
Image Statistics 135
Pixel or Regional Information Tools 135
Histograms 136

Chapter 10 Formats: Media, Resolution, and Aspect Ratios 141
Aspect Ratio ... 143
Nonsquare Pixels ... 144
Deciding on a Resolution When You Have an Aspect Ratio ... 146
Format Conversion Pipeline ... 147
A Format Conversion Example ... 149
Film Formats ... 152
35mm Formats ... 153
16mm Formats ... 155
Specialized Film Formats ... 156
Video Formats ... 158
Lines of Resolution ... 158
Fields ... 159
Color Resolution ... 161
Gamma ... 162
NTSC ... 162
PAL ... 163
SECAM ... 163
HDTV ... 163
Other Formats ... 164
Working With Nonsquare Pixels ... 165
Combining Formats ... 170
Converting Between Film and Video ... 170

Chapter 11 Quality and Efficiency 173
Quality ... 173
Efficiency ... 174
Methodology ... 174
Minimizing Data Loss ... 175
Internal Software Accuracy ... 178
Consolidating Operations ... 180
Region of Interest ... 184
Working in a Networked Environment ... 185
Disk Usage ... 185
Precompositing ... 186

Chapter 12 Learning to See 189
Judging Color, Brightness, and Contrast ... 190
The Camera ... 192
Distance and Perspective ... 193
Perspective and the Camera ... 193

Depth Cues ... *195*
Combining Depth Cues ... *199*
Lens Flares ... 200
Focus ... 201
Motion Blur ... 201

Chapter 13 Creating Elements ... *203*
Lighting ... 204
Interactive Lighting ... *205*
Matched Cameras ... 207
The Reference Stand-in ... 209
Clean Plates ... 211
Film Stock ... 211
Filters ... 213
Choosing a Format ... 214
Lighting and Shooting with Bluescreens ... 216
Bluescreen versus Greenscreen ... *219*
Shooting Order ... 221

Chapter 14 Integration Techniques ... *223*
Scene Continuity ... 224
Lighting ... 225
Shadows ... 226
Lens Flares ... 228
Atmosphere ... 229
Camera Mismatches ... 230
Camera Movements ... 232
Focus ... 233
Motion Blur ... 233
Film Grain ... 234
Bluescreen Integration ... 236
Digital Integration ... 238

Chapter 15 Advanced Topics ... *241*
Beyond Black and White ... 241
White Point and Black Point ... *243*
Nonlinear Color Spaces ... 245
Working with Nonlinear Color Spaces ... *250*
Working with 3D Elements ... 253
Z-Depth Compositing ... *255*
Related 2D Disciplines ... 261

Morphing *261*
Digital Painting *264*
Editing *266*

Chapter 16 Case Studies *269*
James and the Giant Peach 269
Speed 271
Star Trek: Insurrection 273
Con Air 274
Budweiser Lizards Commercial 276
Independence Day 278
X-Files: Fight the Future 280
T-Rex: Back to the Cretaceous 281
The Prince of Egypt 285
Titanic 288

Appendix A Digital Compositing Software: Tools and Features *291*
Color Correction 292
Spatial Filtering 293
Geometric Transformation 294
Miscellaneous 295
Image Combination 295
Control 296
Field Controls 297
Matte Generation 297
Timing/Duration 297
Image Generation 298
Tracking 298
Other 299

Appendix B Digital Compositing Software: Manufacturers *301*

Appendix C Digital Image File Formats *303*
Common File Formats 303
The Cineon File Format 306

Appendix D Common Film and Video Formats *313*
Film 313
Scanning Resolutions and File Sizes *314*
Video 317
HDTV *319*

Bibliography 321

Glossary 325

Index 355

Acknowledgments

This book would never have been completed without the help of a number of different people. Many of these people are listed here, but many, many others who gave me ideas, information, and inspiration are not, primarily due to the fallibility of my memory. To them I apologize, and hope that the fact that their wisdom is being propagated and utilized is at least some small consolation.

Thanks, first of all, to Tom Stone, whose enthusiasm for this book convinced me that it was worth doing, and who was the only publisher I talked to who understood why I was asking for so many of those expensive color plates! Thanks also to Tom's colleagues, Thomas Park and Julie Champagne, who hand-held me throughout the process.

Several different companies contributed the images that are discussed in Chapter 16. Specifically, Centropolis Effects in Santa Monica, CA (*Independence Day*), Digital Domain in Venice, CA (*Titanic* and the Budweiser "Lizards" commercial), Dreamquest Images in Simi Valley, CA (*Con Air*), Dreamworks SKG in Burbank, CA (*The Prince of Egypt*), Sony Pictures Imageworks in Culver City, CA (*James and the Giant Peach* and *Speed*), and VIFX in Marina Del Rey, CA (*T-Rex: Back to the Cretaceous*, *X-Files: Fight the Future*, and *Star Trek: Insurrection*). The following people were interviewed about the creation of the specific shots that are discussed: Jean-Luc Azzis for the Budweiser commercial, Jonathan Egstad for *Titanic*, Conny Fauser for *Independence Day*, Richard Hollander for *T-Rex*, David Morehead for *The Prince of Egypt*, Marlo Pabon for *Con Air*, Mark Rodahl for *Star Trek*, and John Wash for *X-Files*.

A number of other people helped with the book during its development, from the original SIGGRAPH course notes to the final proofreading. Thanks to Jerome Chen, Gary Jackemuk, Ashley Beck, Jeanette Volturno, Amy Wixson, Charlie Clavedetcher, Buckley Collum, John Carey, and Michelle Steinau, for their input, feedback, and support. Thanks to Mike Wassel at Illusion Arts for his help in photographing many of the example images that are used throughout the book.

Thanks to Alex, Aundrea, Alyssa, Rachel, Anna, and Josh, because I would be a poor uncle indeed if I passed up the opportunity for them to see their names in print, and thanks to Mandy and Ace (Plate 28).

A special note of thanks to my associates at Nothing Real, not only for the use of their fine compositing software to create many of the images in this book, but also for the assistance, encouragement, and knowledge of everyone there: Dan Candela, Louis Cetorelli, Allen Edwards, Arnaud and Fleur Hervas, Sid Joyner, Emmanuel Mogenet, and Peter Warner.

Finally, and above all, I have to thank all the digital compositing artists that I have had the pleasure of working with over the years, as well as the rest of the incredibly creative people in this industry who produce the fantastic images that feed and fill our imaginations. Keep up the excellent work.

Preface

This book is about creating images. It will discuss a number of different tools that can be used to create these images, but it is not a book about tools. In the years that I have been involved with the field of digital compositing, there have been dramatic increases in the power and flexibility of the tools that are available. But the basic concepts have remained the same, and this will most likely continue to be the case for the foreseeable future. Consequently, this book is not intended to be about how to use any specific piece of software—there are too many different tools, and they are changing too quickly. Rather, it is intended to give the reader the information that he or she needs in order to sit down in front of any piece of digital compositing software and be productive. Having been through the process of hiring quite a few compositing artists for the films on which I've worked, I almost always base my choice on the candidate's general compositing experience and not on whether they know how to use a specific package. Knowing where the buttons are located isn't nearly as important as knowing why to press those buttons.

The scope of techniques that fall under the category of digital compositing is actually quite large, from simple wire removals to assembling complex scenes with hundreds of disparate elements. The tools can range in cost from hundreds of thousands of dollars at the high end down to virtually nothing (shareware) at the low end. A person may spend only a few hours on a simple shot, or teams of people may spend months on a complex shot. Every shot is new; every shot is different. That's what makes the field so challenging, and it's also why no book can ever hope to be the final word on the subject.

My primary guideline for myself was to write the book that I wish someone had handed me when I first got started in this business. Although there is certainly no substitute for working alongside experienced professionals, I hope this book will be a good starting place for those new to the field. At the same time, I wanted it to be something that people who have been in the business for a while will also find useful. Straddling the fence can be a somewhat uncomfortable position at times, but I hope the compromise proves worthwhile.

Please feel free to send me feedback on the book, including any specific suggestions for improvements and corrections. I can be reached through my publisher, or e-mail me directly at rbrinkmann@nothingreal.com. I can't promise that I'll be able to reply to everyone, but I will read whatever you send.

The Art and Science of Digital Compositing

CHAPTER ONE

Introduction to Digital Compositing

> A massive spacecraft hovers over New York, throwing the entire city into shadow. A pair of lizards, sitting in the middle of a swamp, discuss their favorite beer. Dinosaurs, long extinct, live and breathe again, and the Titanic, submerged for decades, sails once more.

Usually the credit for these fantastic visuals is given to "CGI" (computer-generated imagery) or "computer graphics," an attribution that not only broadly simplifies the technology used, but also ignores the sizeable crew of talented artists who actually created the work. Computer graphics techniques, in conjunction with a myriad of other disciplines, *are* commonly used for the creation of visual effects in feature films. But the term "computer graphics" is broad and covers a wide variety of methods that rely on a computer to help produce images. Many of these methods are merely traditional methods that have been updated to take advantage of modern tools. In fact, even the typesetting of a book like this is now almost completely done using a computer, and as such this page could loosely be considered a piece of "computer graphics."

When dealing with computer graphics as used for the creation and manipulation of images, we will usually break the subject down into two primary subcategories: **3D graphics**[1] and **2D graphics.** The names indicate whether the work is

[1] Do not confuse 3D imagery with stereoscopic imagery, a topic that we will discuss in Chapter 12.

considered primarily three-dimensional in nature, or two-dimensional. The first category, the "3D" work, involves creating a complete model of an object within the computer. This model can be viewed from any angle, can be positioned relative to an imaginary camera, and can generally be manipulated as if it were a real object, yet it exists only within the computer. Even though the way we are interacting with the object is still based on a two-dimensional display device (the computer's monitor), the model itself is a mathematical simulation of a true three-dimensional object. This model can be lit, textured, and given the ability to move and change. Once a particular camera view is chosen and the color, lighting, and animation are acceptable, special software will **render** the scene to produce a sequence of images.

While the 3D aspect of visual effects seems to get a great deal of recognition, it is only one piece of the puzzle that makes up a finished shot. The other half of the visual effects process involves working with preexisting images, manipulating and combining them to produce new images. These images can be from just about any source, including rendered images produced by the 3D process. This process of manipulating existing images is identified as being "2D" work because of the flat, two-dimensional images with which it deals and because there is essentially no attempt to introduce any three-dimensional data into the process. Not every film that makes use of visual effects will include 3D work, but any time there is a visual effect in a film, you can assume that 2D work was done. It is the backbone of visual effects work, and the final, most important step in the creation of the desired imagery.

Even with a fully 3D movie such as Pixar's *Toy Story*, 2D effects and tools are used to enhance and integrate the rendered 3D images. 2D manipulations have historically been accomplished via a number of different methods, as we'll discuss in a moment. But these days, most 2D work is done with the aid of computers, and the bulk of this 2D work is classified as digital compositing.

DEFINITION

Digital compositing, as we are going to be discussing it, deals with the process of integrating images from multiple sources into a single, seamless whole. While many of these techniques apply to still images, we will be looking at tools and methods that are useful and reasonable for large **sequences** of images as well. Before we go any further, let's come up with a specific definition for what this book is all about.

> **Digital Compositing:** The digitally manipulated combination of at least two source images to produce an integrated result.

By far the most difficult part of this digital compositing process is producing the integrated result—an image that doesn't betray that its creation was owed to multiple source elements. In particular, we are usually attempting to produce (sequences of) images that could have been believably photographed without the use of any postprocessing. Colloquially, they should look "real." Even if the elements in the scene are obviously *not* real (huge talking insects standing atop a giant peach, for example), one must be able to believe that everything in the scene was photographed at the same time, by the same camera.

Although so far we've mentioned only a few big-budget examples of digital compositing being put to use, in reality you'll find digital compositing at work just about anywhere you look in today's world of media and advertising. Pull a magazine off the shelf and in all likelihood you will find that most of the art and graphics have been put together using some sort of computerized paint program. Television commercials are more likely than not to have been composited together from multiple sources. Yet whether the digital compositing process is used for a business presentation or for feature-film special effects, the techniques and tools employed all follow the same basic principles.

This book is intended to be an overview that will be useful to *anyone* who uses digital compositing. However, you will probably notice that many topics, descriptions, and examples seem to approach the subject from the perspective of someone working to produce visual effects for feature films. This emphasis is not only due to the author's experience in this field, but also because feature-film work tends to push the limits of the process in terms of techniques, technology, and budgets. Consequently, it is an ideal framework to use in providing an overview of the subject. Additionally, it allows for the use of examples and sample images that are already familiar to most readers.

The title of this book, *The Art and Science of Digital Compositing*, was coined to stress the fact that true mastery of digital compositing includes both technical and artistic skills. As with any art form, the artist must certainly have a good amount of technical proficiency with the tools that will be used. These tools could potentially include any number of different hardware and software compositing systems. But one should also become knowledgeable about the science of the *entire* compositing process, not just specific tools. This would include everything from an understanding of the way that visual data is represented in a digital format to knowledge of how a camera reacts to light and color. Please remember, though, that all these technical considerations are simply factors to weigh when confronted with the question of "Does it *look* right?" The answer will ultimately be a subjective judgment, and a good compositor who is able to consistently make the decisions that produce quality images will always be in high demand.

The combination of multiple sources to produce a new image is certainly nothing new, and was being done long before computers entered the picture (pardon the pun). Although this book is about *digital* compositing, let's spend a moment looking at the foundations upon which digital compositing is built.

HISTORICAL PERSPECTIVE

In the summer of 1857, the Swedish-born photographer Oscar G. Rejlander set out to create what would prove to be the most technically complicated photograph that had ever been produced. Working at his studio in England, Rejlander selectively combined the imagery from 32 different glass negatives to produce a single, massive print. A reproduction of this print, which was titled *The Two Ways of Life*, is shown in Figure 1.1. It is one of the earliest examples of what came to be known as a "combination print."

Had the artist wished to capture this image on a single negative, he would have required a huge studio and many models. Even then, it is doubtful whether he could have lit the scene with as much precision or have positioned all the people in exactly the fashion he desired. Certainly it could have proven to be an expensive, time-consuming process. Instead, he painstakingly shot small groups of people and sets, adjusting each for the position and size that he would need them to be. In some cases, the only way to make them small enough in frame was to photograph them reflected in a mirror. Once the various negatives were created, the combination process involved selectively uncovering only a portion of the printing paper and exposing the desired negative to that area.

The scene that resulted from this complex undertaking was designed to depict the two paths that one may choose in life. The right side of the image represents the righteous path, with individual figures who illustrate Religion, Knowledge, Mercy, Married Life, and so on. The left side of the image depicts somewhat less lofty goals, with figures representing everything from Idleness to Gambling to Licentiousness to Murder. Photography was only just becoming accepted as a true art form, but Rejlander's work was immediately recognized as an attempt at something more than the typical documentary or narrative photographs of the time. This is important to understand, since it points out that Rejlander used this combination technique in pursuit of a specific vision, *not* as a gimmick. There was a great deal of science involved, but more important, a great deal of art.

While *The Two Ways of Life* received quite a bit of recognition, it was also the subject of some controversy. Although part of this had to do with its subject matter (a Scottish exhibition of the work actually hung a drape over the nudity-rich left half of the image), the issue of whether or not such "trick" photography was ethical or artistically valid was continually raised. Eventually Rejlander himself denounced the practice, stating

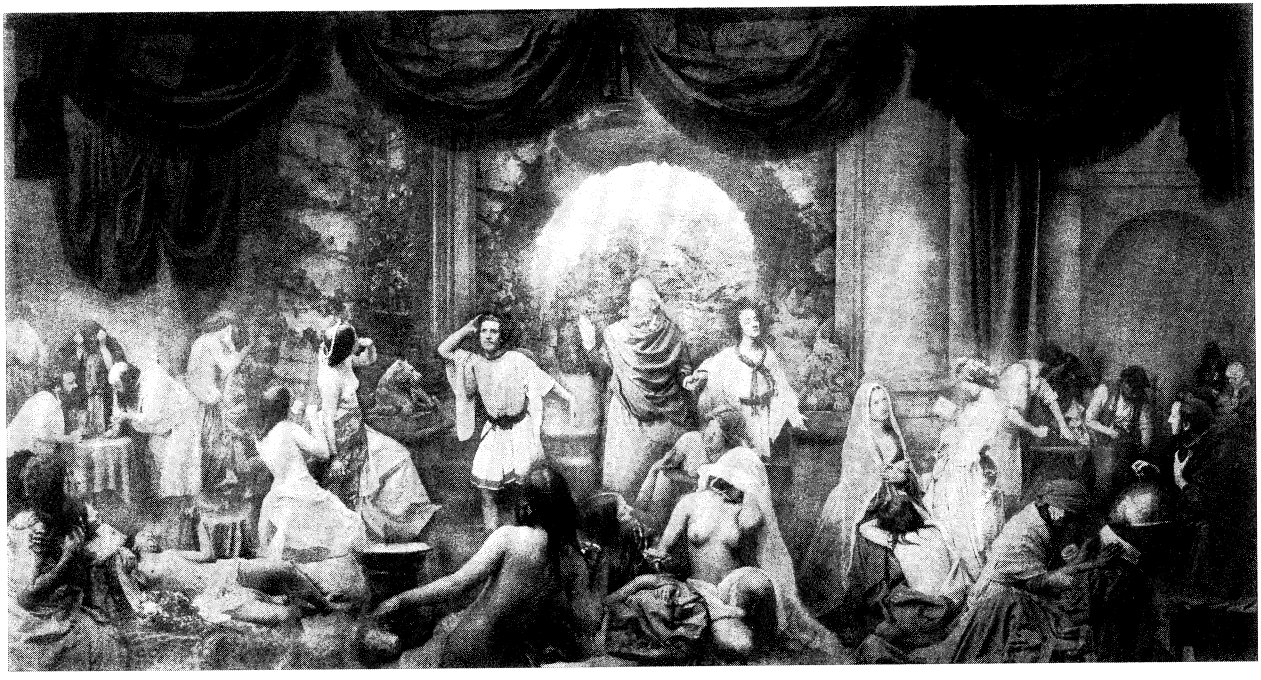


Figure 1.1 *An early composite photograph, Oscar Gustav Rejlander's* The Two Ways of Life. *(Photo courtesy of The Royal Photographic Society Collection, Bath, England. Web site http://www.rps.org)*

> I am tired of photography for the public—particularly composite photographs, for there *can be no gain*, and there is no honour, but cavil and misinterpretation.

Fortunately, the techniques continued to be used and refined even without Rejlander's support, and few people today consider compositing (at least conceptually) to be particularly dishonorable.

Motion picture photography came about in the late 1800s, and the desire to be able to continue this sort of image combination drove the development of specialized hardware to expedite the process. **Optical printers** were built that could selectively combine multiple pieces of film, and **optical compositing** was born. Optical compositing is still a valid and often-used process. Many of the techniques and skills developed by optical compositors are directly applicable to the digital realm, and in many cases, certain digital tools can trace not only their conceptual origin but also their basic algorithms directly to optical methodologies. Consequently, the digital compositing artist would be well served by researching the optical compositing process in addition to seeking out information on digital methods.

A number of early examples of optical compositing can be found in the 1933 film *King Kong*. The image shown in Figure 1.2 is actually a composite image that was created in the following fashion: The giant ape was photographed first—a 16-inch tall miniature that was animated using **stop-motion** techniques. This process involves photographing the model one frame at a time, changing the pose or position of the character between each frame. The result, when played back at normal speed, is a continuously animating object. After this footage was developed and processed, it was projected onto a large **rear projection** screen that was positioned on a full-sized stage. The foreground action (the actress in the tree) was then photographed while the background footage was being projected behind it, producing a composite image. This particular type of compositing is known as an **in-camera effect**, since there was no additional post-production work needed to complete the shot. Other scenes in the film were accomplished using an early form of **bluescreen** photography (which we will discuss further in Chapters 5 and 13), where the foreground and background were photographed separately and then later combined in an optical printing step.

Nowadays, optical compositing equipment has been largely (but not completely) replaced with general-purpose computer systems and some highly specialized software, but the concepts have not really changed. Before we start our discussion of these software tools, let's take a look at an example of one of these digital composites. We won't go into a great deal of detail about this particular example just yet, but will initially use it to start presenting some of the common terminology used throughout the industry, as well as throughout this book.

Figure 1.2 *An early motion-picture composite, from the film* King Kong. *(KING KONG © 1933 Turner Entertainment Co.)*

TERMINOLOGY

The example that we'll be dealing with is a scene from the feature film *James and the Giant Peach*, and is shown in Plate 1. This particular composite was created from a multitude of different original images. We usually refer to the individual pieces from which we create our final composite as **elements**. Elements in this composite include the following:

- The giant peach, shown as a separate element in Plate 2a. The peach is a miniature element, about a foot in diameter, and was photographed on a stage in front of a blue background, or **bluescreen**.
- The giant mechanical shark, shown in Plate 2b. This element is a computer-generated image, built and rendered as a three-dimensional model completely within the computer.
- The water, shown in Plate 2c. The water element is also computer-generated 3D imagery.
- The sky, shown in Plate 2d. This element is a hand-painted backdrop (painted on canvas) that was photographed as a single frame.

Many other original elements make up this composite as well, most of them 3D elements. These include the reflections of the peach and the shark in the water, the smoke coming from the shark, shadows for the various elements, and spray and foam on the top of the water. Throughout the compositing process, additional elements may be generated that are the result of some manipulation performed on an original element. Plate 2e shows such an element: a specialized image known as a **matte** that is derived from the bluescreen peach element and that will be used to selectively add the peach into the scene.

As you can probably tell, most of the elements that we've shown have had some sort of additional processing performed on them as they were added into the final scene. Such processing, which might be used to modify the color of an image or the size of an element in the frame, is done throughout the compositing process in order to better integrate elements into their new environment.

You will commonly hear elements referred to as "layers," since the various elements are layered together to produce the eventual image. Original footage that is shot with a camera and transferred into a computer constitutes a subset of elements usually referred to as **plates**. Thus, the peach element in this example would be referred to as a plate. Typically, a synthetic image such as the water would *not* be termed a plate, nor would any intermediate elements that were generated during the process of creating a composite, such as the peach matte.

As you can see, there is little conceptual difference between the shot that was done for *King Kong* and recent composites such as that produced for *James and the Giant Peach*. The tools used may have evolved, but the intentions and basic methodology remain unchanged.

ORGANIZATION OF THE BOOK

This book will attempt to cover a range of topics related to digital compositing, from some basic concepts dealing with digital image acquisition and storage to specific aesthetic considerations necessary for producing a good composite. Initial chapters will provide enough background so that readers with only a basic knowledge of computers will find the book useful; in addition, there are a number of sections in this book that we hope even the most experienced professional will find useful.

The structure of a book such as this requires that the information be broken into well-defined categories. These categories (represented by chapters or sections within a chapter), while useful for organizational purposes, are ultimately somewhat arbitrary. The topics are all interrelated, and many discussions could easily have been placed into a number of different categories. To help identify related information, you will often find cross-references to relevant material that is located

elsewhere in the book. The specific breakdown for how the chapters are organized is as follows:

- Chapter 2 presents an overview of how images are represented digitally, including some discussion about the process of converting images from other sources into a digital format.
- Chapter 3 covers some of the basic manipulations that are possible with digital images. These include using image processing tools that are designed to modify the color, size, and placement of elements within a scene.
- Chapter 4 takes this topic to the next step, with a look at the process of combining images or image sequences. This chapter is where the concept of a **matte**—an image that is used to selectively control certain combinatorial operations—is first introduced.
- Chapter 5 deals with these matte images in much greater detail—particularly with the methods that are commonly used to create and modify them.
- Chapter 6 is dedicated to concepts and techniques relating to imagery that changes over time, and to image manipulations that do the same.
- Chapter 7 looks at **tracking**, the process of analyzing and duplicating the motion of existing objects in a scene.
- Chapters 8 and 9 discuss a variety of methods that can be used to interact with the data, images, and software that you will be using.
- Chapter 10 is a broad overview of the various formats that might need to make use of digital compositing, from film to video to multimedia.
- Chapter 11 covers some important concepts that must be understood in order to work efficiently while still producing quality imagery—an area of great importance for any sizeable compositing projects.
- Chapter 12 describes characteristics of cameras and vision that must be understood in order to produce realistic, believable composites.
- Chapter 13 deals with some of the things that should happen *before* a composite is started, when the elements that will be used are being created.
- Chapter 14 rounds out a topic that we touch on throughout the book, namely, image and scene integration techniques, and gives some more specific suggestions.
- Finally, Chapter 15 goes into a bit more detail about certain topics that are of a more advanced nature and also touches on some disciplines that are related to compositing, such as digital painting and editing.
- The last chapter of the book, Chapter 16, uses all this preceding information to take an in-depth look at a number of well-known scenes that were created via the use of digital compositing. It should give a sense of how the topics that we've covered can be applied in real-world situations.

The book also includes a few appendices that deal with specific topics in greater detail.

- Appendix A provides a summary of the tools that should be part of a reasonably complete compositing package, extending the list of operators that are covered in the body of the book. This section should also prove useful if you need to evaluate the relative merits of competing pieces of software.
- Appendix B lists companies that provide digital compositing software, as well as software that is often used in conjunction with the compositing process.
- Appendix C attempts to list some of the popular file formats that are used to store digital images. This appendix also includes a more in-depth look at the Cineon file format, which is specifically designed for the representation of images that are shot on film.
- Appendix D documents some specific details about common film and video formats.

Although this book is primarily targeted at those who are actually creating digital composites, much of the information will also be important to individuals who need a more simplified overview of the subject. If this is the case, the following course of action is suggested.

1. In general, the introductory information at the beginning of each chapter gives a basic idea about what is contained in that chapter. If nothing else, read these introductions.
2. Chapter 2 defines a great deal of terminology. A basic understanding of the information contained in the section entitled "Image Generation" should be enough to make the rest of the book a bit more comprehensible.
3. Chapters 3 and 4 concentrate on specific tools, and as such may not be as relevant to someone who will not be involved with actual image creation. The section in Chapter 4 that discusses the concept of a matte is probably the only "must-read" section.
4. Chapters 5, 6, and 7 should be skimmed to get a basic idea of the concepts presented, but probably not much more than that is necessary. Chapters 8 and 9 are even less interesting to someone who will not be using any software directly.
5. Chapter 10 should probably only be looked at if there is a desire to understand a particular format in more detail. This may come up on a case-by-case basis.

6. Chapter 12 is important for anybody who wants to understand the visual nature of the compositing process. It is not terribly technical and should probably be read in its entirety. Chapter 13, on the other hand, will most likely be useful only if you are involved in the actual photography of the elements that will be used in a composite.
7. Chapter 15 is mostly composed of advanced topics, but the end of the chapter does touch on a few disciplines related to compositing that may be useful.
8. The final piece of the book, which may prove to be the most useful, is the glossary. Although it is not something that should be read from beginning to end in one sitting, it should prove to be a valuable resource as one encounters unfamiliar terms.

As you can probably see from some of these descriptions, the first part of this book tends to focus more on the *science* of digital compositing. We go over the basic technical concepts that one should become familiar with, and we describe the tools that are typically available. Once this technical foundation is laid, we then expand the focus to discuss more of the techniques, the *art*, behind the digital compositing process.

Whether or not you actually have any formalized artistic training should not deter you from learning the specific artistic skills that a good **compositor** (one who creates composites) uses on a daily basis. Part of this process involves learning the tools that are necessary. Just as a painter needs to understand the workings of pigments and canvas, a digital artist needs to have a technical understanding of the software involved. It is certainly possible to be an effective compositor without fully understanding all the technical issues that will be presented in this book. As with any other artistic discipline, experience and instinct can go a long way toward producing the final result. However, a basic understanding of the technology behind the tools can greatly improve the efficiency and problem-solving ability of even the best artists.

The rest of the art will come only with experience, although you will probably find that you are already well on your way. This is because you, and every other sighted person, have already spent a lifetime learning what reality looks like. The information may not always be consciously accessible, but the expert subconscious is surprisingly skilled at noticing when an image appears artificial or incorrect. This is both a blessing and a curse, since everyone else who will view your work will be similarly able to detect these problems. However, the ability to determine that there *is* a problem is the largest part of the process of realizing why the problem exists and learning how to fix it. Throughout this book we'll certainly try to point out the scenarios in which visual problems can arise. These problems will often involve well-defined **artifacts**—undesirable items in an image that are

the result of the process used to create the image. But compositing problems can go far beyond specific identifiable glitches. Greater difficulties arise when trying to accurately duplicate the myriad of visual cues that would be present in a real image. These are the things that the subconscious is so much better at perceiving, and this is why someone who creates composite imagery must become an artist. The same knowledge that a painter uses—from the interaction of light and shadow to distance and perspective cues—will be necessary to create a believable digital composite.

There is one final piece of business to be dealt with before we can continue to the rest of the chapters in this book. A disclaimer:

> Different people, countries, and software packages do not always use the same names to refer to particular tools, operators, or subjects. In addition, due to the need to simplify certain things, just about every statement we make could probably be countered with some exception to the rule.

This statement isn't given in order to protect the author from any mistakes he's made within the pages of this book, but rather as a warning to anyone just starting to learn about this field. In many cases, you will find similar or identical concepts referred to by completely different names, depending on what book you are reading or to whom you are talking. Digital compositing is still a very young, volatile discipline. As such, there are many areas within it that have not become terribly well standardized.

A glossary is provided at the end of the book that attempts to cover a wide range of terms as they relate to digital compositing. Throughout the body of the text, terms being introduced are printed in bold to indicate that you will find a definition in the glossary. There are certainly places in the book where we have made some arbitrary decisions about a particular term and its usage. It is hoped this will help, rather than hinder, the process of terminology standardization within the industry.

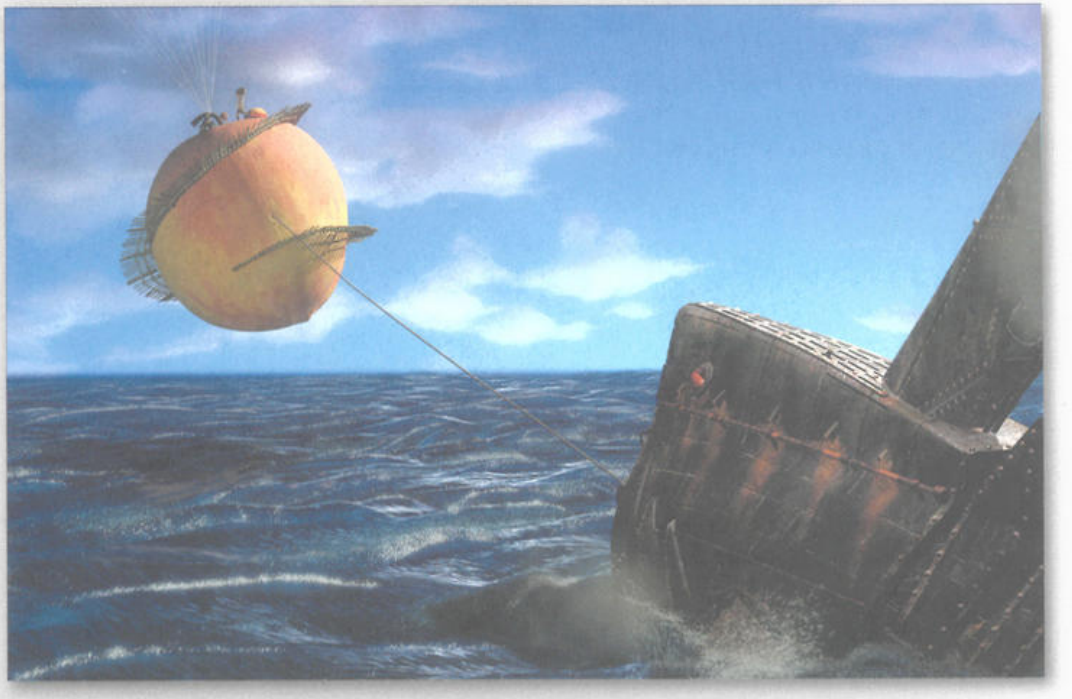


Plate 1 *A composite scene from* James and the Giant Peach. *Plates 2 (a)–(e) show the elements making up Plate 1.*

Plate 2 *(b) The giant mechanical shark.*

Plate 2 *(c) The water.*

Plates 1, 2b, and 2c from *James and the Giant Peach* © Disney Enterprises, Inc.

Plate 2 *(a) The peach.*

Plate 2 *(e) A matte of the peach.*

Plate 2 *(d) The sky.*

Plates 2a, 2e, and 2d from *James and the Giant Peach* © Disney Enterprises, Inc.

Plate 3 *The primary test image. Plates 4 (a)–(c) show the channels composing the primary test image.*

Plate 4 *(a) Red channel.*

Plate 4 *(b) Green channel.*

Plate 4 *(c) Blue channel.*

Plate 5 *The test image after being reduced to a color resolution of four bits per channel.*

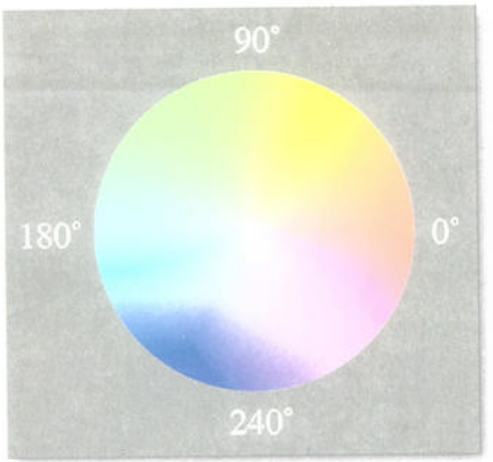


Plate 6 *HSV color representation. (a) Circular color palette representing hue and saturation.*

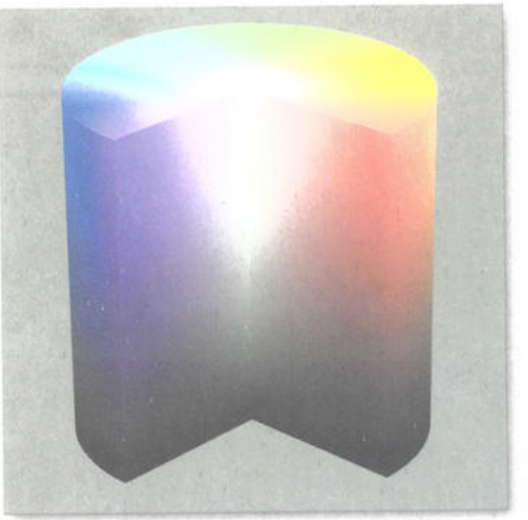


Plate 6 *(b) Three-dimensional color cylinder representing value along the third axis.*

Plate 7 *The test image with a brightness of 2.0 applied.*

Plate 8 *Result of RGB multiplication of (0.1, 1.25, 1.0) on test image.*

Plate 9 *Result of adding 0.2 to every pixel in the test image.*

Plate 10 *Result of gamma correction of 1.7 on the test image.*

Plate 11 *Test image showing result of (a) simple contrast operation.*

Plate 11 *(b) smoother contrast operation.*

Plate 12 *Test image with saturation reduced by approximately 50%.*

Plate 13 *Test image with hues rotated 180°.*

Plate 14 *Look-up table curves from Figure 3.9 applied to test image.*

Plate 15 *Saturation modified based on certain hues.*

Plate 16 *Result of applying a conditional arithmetic expression to the test image.*

Plate 17 *The effects of increased sharpening. (a) Original image.*

Plate 17 *(b) Image after some sharpening has been applied.*

Plate 17 *(c) Image with excessive sharpening. Note the ringing along transition areas.*

Plate 18 *Source images for illustrating multisource operators. (a) Foreground image.*

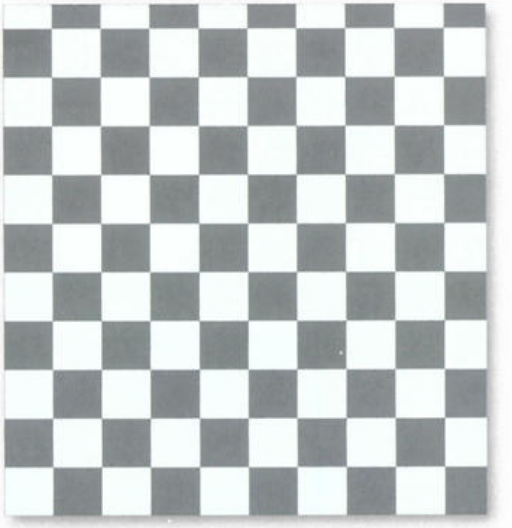


Plate 18 *(b) Background image.*

Plate 19 *Image resulting from adding Plates 18a and 18b.*

Plate 20 *Matte image.*

Plate 21 *Image resulting from placing Plate 18a over 18b using the matte image in Plate 20.*

Plate 22 *Premultiplied image, produced by multiplying Plate 18a by Plate 20.*

Plate 23 *Steps in the Over process. (a) An inverted matte image.*

Plate 23 *(b) The image produced by multiplying the image from Plate 18b by Plate 23a.*

CHAPTER TWO

The Digital Representation of Visual Information

Digital compositing is, ultimately, about working with images. Before we can effectively discuss any of the basic tools of compositing, therefore, we need to build up a bit of background knowledge and vocabulary concerning how images are dealt with in the medium we are using. We initially cover, very briefly, some basic terms and concepts.

For the topic of **digital** compositing, an obvious issue is how the digital image data is stored and represented. Although this topic could conceivably be extended to include everything from binary data coding to disk formatting, we try to present information that will directly affect the decisions a typical compositing artist will need to make on a regular basis. There are dozens (or maybe hundreds) of different, reasonably well-defined ways to store an image digitally. But, in day to day usage, most of these methods have several things in common. Many of the concepts that are covered in the beginning of this chapter will be trivial and obvious to someone who has spent any time working with digital images, but it is suggested that you at least skim the sections because certain terms are introduced and defined that will then be used throughout the book.

IMAGE GENERATION

Before we can discuss how image data is stored, we need to cover where the images come from in the first place. Although the digital compositor will eventually be dealing with images that have all been converted to some specific digital format, the procedures that were used to initially create or capture those images may

vary quite a bit. Elements that will be used for digital compositing generally come from one of the following three places:

1. Hand-painted or human-generated elements. These can range in complexity from a simple black-and-white matte to a photorealistic **matte painting** of a real or imaginary setting. Although in the past much of this type of artwork would have been generated outside the computer using more traditional painting methods, the advent of extremely powerful paint programs has made it far more common to hand-paint elements directly in the computer. An example of this is shown in Plate 60, a matte painting that was created for the film *Speed*, from 20th Century Fox.
2. Computer-generated images. Although no images are truly "computer generated," since their origin can eventually be traced back to a human being who defines a set of algorithms for the computer to use, this term is usually used to refer to elements that have been created or rendered using a specialized 2D or 3D computer animation package. In the *James and the Giant Peach* example from Chapter 1, the mechanical shark and the stylized water were both computer-generated images.
3. Images that have been **scanned** into the computer from some other source (typically film or video).

Breaking this topic up into three categories is extremely simplified—computer-generated elements may contain scanned or painted elements (used as texture maps, for instance), matte paintings often borrow heavily from other scanned sources, and live-action elements may be augmented with hand-painted **wire removals** or **effects animations**.

The first source of imagery, hand-painted elements, will be touched on throughout the body of this book as needed. We will not, however, spend a great deal of time detailing specific software or techniques. There are a number of popular software packages for hand-painting elements in the computer, some of whose manufacturers are listed in Appendix B.

The subject of how computer-generated (CG) images (our second potential source of imagery) are defined and rendered will not be covered in great detail, although there are a few sections of this book (notably in Chapter 15) that give some suggestions about how to ease the integration of 3D elements into composited scenes. Manufacturers of several popular software packages for producing CG images are also mentioned in Appendix B, and there are numerous others.

Some excellent reference works already exist on both of these topics, a few of which are mentioned in the bibliography at the end of this book. Not only are there a variety of books that cover digital painting, modeling, animation, and

rendering from a generalized perspective, but there are also specific "how-to" guides for many of the more common software packages.

The third source of imagery—scanned/digitized "live-action" footage—is still probably the most common source with which we deal in digital compositing. There are a myriad of different formats that this source imagery can come from, some of them discussed in greater detail in Chapter 10 and Appendix D.

Understanding the methods that were used to generate a particular image sequence can be extremely important to a compositor. There are a number of distinguishing factors between CG elements and scanned live-action images in terms of how they are used in the compositing process. For instance, rendered CG elements can usually be generated with additional information that may be used to aid their placement into an existing scene. Information such as an additional matte channel (covered more in Chapter 4) or *Z*-depth information (Chapter 15) will allow the compositor to bypass or simplify several integration steps that are otherwise necessary when dealing with simple scanned images. More important, there needs to be a recognition that digital images that are created from live-action sources are actually just limited **samples** of the real-world scene. There is always a limit to the amount of detail and the range of color and brightness any given image capture technique can provide, a fact that should always be kept in mind as one is working with these images. This point is discussed a bit more in Chapter 11, and Chapter 15 has several sections that consider the relationship between captured images and the real world.

Pixels, Components, and Channels

Digital images are stored as an array of individual dots, or **pixels**. Each pixel will have a certain amount of information associated with it, and although it is still common to present a pixel as having a specific color, in reality there may be a great deal of additional information that is associated with each pixel in an image. But rather than taking the typical computer-graphics route of discussing all the possible characteristics of a pixel first and then looking at how these combine to make an image, we want to think of a color image as a layered collection of simpler images, or **channels**.

Digital images *are*, of course, a rectangular array of pixels. And each pixel does, of course, have a characteristic color associated with it. But the color of a pixel is actually a function of three specific **components** of that pixel: the red, green, and blue (usually simplified to R,G, and B) components.[1] By using a combination of

[1] There are any number of different ways to represent color, but because of its prevalence in the digital world we will primarily deal with the RGB model. A bit later we will talk about another model that represents color via hue, saturation, and value parameters.

these three primary colors at different intensities, we can represent the full range of the visible spectrum for each pixel.

If we look at a single component (red, let's say) of every pixel in an image and view *that* as a whole image, we have what is known as a specific **channel** of the complete color image. Thus, instead of referring to an image as a collection of colored pixels, we can think of it as a three-layer combination of primary-colored channels.

Consider an image such as that shown in Plate 3. The next three images, Plates 4a, 4b, and 4c, show the red channel, the green channel, and the blue channel of this sample image. In this case, we've tinted the individual channels to reflect the color they are associated with, and it is convenient to think of the channels as being transparent slides that can be layered (or projected) together to result in a full-color image. But these channels can also be thought of as monochrome images in their own right. Consider Figure 2.1, which is the same image as Plate 4a, the red channel, but without any colored tint applied. This monochrome representation is really more accurate, in the sense that a channel has no inherent color, and could conceivably be used for any channel in an image.

The reason for dealing with images in this fashion is twofold. First, single pixels are simply too small a unit to deal with individually; in general, compositing artists spend about as much time worrying about individual pixels as a painter might spend worrying about an individual bristle on her paintbrush. But more

Figure 2.1 *Monochrome equivalent of the test image's red channel (see Plate 4a).*

important, dealing with complete channels gives a great deal of additional control and allows for the use of techniques that were pioneered in the days when optical compositing was being developed. Color film actually consists of three different emulsion layers, each sensitive to either red, green, or blue light, and it became useful to photographically separate these three layers, or **records**, in order to manipulate them individually or in combination. A digital image (also known as a **bit-mapped image**) of this type will generally consist of these three color channels integrated into a single image, but it should be clear that these three channels can be thought of as separate entities that can be manipulated separately as well.

Looking at the individual channels for the image in question, we can see which areas contain large values of certain colors, and it is easy to find the correspondence in an individual channel. For instance, the background of the image—the area behind the parrot—is a fairly pure blue. If you look at the red channel of this image (Plate 4a), it should be obvious that there is essentially no red content in this area. On the other hand, the front of the head and the beak are areas with a good deal of red in them (yellow having heavy red *and* green values), which is also obvious when looking at the individual channels. Later we will see how manipulating and combining the individual channels of an image can be used for everything from color correction to matte extraction.

Spatial Resolution

A number of measurements are used to gauge the amount of information that is present in a digital image. The most obvious measurement is the number of pixels used to create the image. The larger the number of pixels that are used, the greater the **resolution** of the image. When we use the term "resolution" in this fashion, we are actually referring to the **spatial resolution** of the image. There are other types of resolutions, such as color or temporal (which will be discussed later), but the general convention is that the pixel count for an image is the primary measurement of the image's resolution.

A square image that is, for example, 500 pixels wide and 500 pixels tall has 250,000 pixels of resolution. Most images are not square, and thus it's generally more useful to speak of an image's resolution by giving the width and height in pixels. The above example would be referred to as an image that is 500 $\times$ 500, or "500 by 500." Greater spatial resolution will allow one to reproduce finer detail in the image, but it will also increase the resources needed to deal with the image.

Different media tend to work in widely different resolutions. At the low end, video games or Web pages may use images that are only 200 or 300 pixels wide. Broadcast video, depending on the standards of the country in question, is usually in the range of 700 $\times$ 500. Feature film work is generally done with an image

that is approximately 2000 pixels wide, though this can vary a great deal. The process of working with various resolutions and formats will be discussed further in Chapter 10.

Some of the highest resolution requirements come from the print industry. Although the terminology is somewhat different and images are generally specified relative to their eventual output medium (i.e., using terms such as "dots per inch," or DPI), the data requirements for large images can be substantial. For instance, a full (two-page) spread in a magazine such as *The National Geographic* will require an image that was stored with a resolution of at least 4000 × 3000 pixels.

Bit Depth

Each component of a pixel can be represented by an arbitrary number of bits. A **bit** is the most basic representation of data within a digital system, and is capable of recording only two states. If we only have a single bit to represent a color, it would probably be used to specify either black or white. More bits will allow us a greater number of color variations, and the use of multiple bits for each component of an image is what allows us to store captured imagery with realistic color information.

The number of bits per component is known as the component's (and the channel's) **bit depth**.[2] The bit depth is actually a way of measuring the **color resolution** of a given image.[3] Just as a larger number of pixels in an image allows for finer image detail, so a larger number of bits allows for finer variations in color. As you will see, many of the issues we mentioned when discussing spatial resolution have counterpart issues when discussing color resolution. Another term for color resolution is **dynamic resolution**, referring to the greater dynamic range available when using a greater number of bits per component, and you will occasionally hear the term **chromatic resolution** as well.

Probably the most common bit depth for image storage is 8 bits per channel, which is also referred to as a "24-bit image" (8 bits each for the red, green and

---

[2] Although you will often see bit depth defined in terms of the number of bits per channel, this definition is technically inaccurate, since the channel of an image refers to the full array of pixels that makes up the image. Theoretically, the number of bits per channel for a given image would be the number of bits per component multiplied by the spatial resolution of the image. However, since "bits per channel" is the more common term, we will consider it to be synonymous with "bits per component" and will use the two interchangeably.

[3] For the purposes of our discussion, we'll consider "color" to include shades of gray as well. Thus, the number of bits dedicated to reproducing the gray tones in a monochrome image can still be considered a measure of its color resolution.

blue channels). Eight bits per channel means that each component of a pixel can have 256 (2^8) different possible intensities and the three components together can represent about 16 million (16,777,216 for the geeks) colors. Although this sounds like a lot of colors (most images will not even have that many pixels), it is often still not enough to reproduce all of the subtle tonal variations that are captured with some analog imaging formats. Feature film work, for instance, often represents digital images with as many as 16 bits per channel, which gives us a palette of 281 trillion different potential colors. On the other hand, lower-end formats in which color gradations are less important may be able to work with a much smaller color resolution. Many video games, for instance, work with only 4 bits per channel, or less!

Incidentally, there is unfortunate confusion about some of this terminology in the personal computer marketplace. The video devices on early PCs were limited in their ability to display a great number of different colors. Many of them could only devote 8 bits *total* to the color specifier. To work around this limitation, they would use these 256 values to look up colors in an index of many more colors. Two different images might not need the same group of 256 colors, and consequently the palette was often dynamically allocated as necessary.[4] Unfortunately, the correct terminology of "8-bit indexed color" was abbreviated in popular use to simply "8-bit color." When systems became available that could display a full 8 bits per *channel*, this then became referred to as "24-bit color." Throughout this book, we will always refer to an image's color resolution in terms of the number of bits per channel.

Plate 5 shows our original sample image (which was initially captured with eight bits per channel) after being decimated to only four bits per channel. Notice the **quantizing** that arises—the noticeable delineations between various bands of colors, particularly in the blue background. This phenomenon, also referred to as "banding," "contouring," or "posterization," arises when we do not have the ability to specify enough unique color values for smooth transitions between different shadings.

Strictly speaking, this noticeable banding should really be called a **quantization artifact**. Quantizing itself is merely the process of assigning discrete values to samples taken from a continuous (i.e., analog) signal. This step is necessary in order to store an image digitally, and if it is done with enough precision, you shouldn't be able to distinguish the digital copy from the original. However, if you do not allocate enough data to store the sampled information, artifacts will

[4] The use of such **indexed color**, while no longer common in display hardware, is still implemented with certain image file formats, such as GIF, and is also commonly used in most dedicated videogame hardware.

result. There is a huge body of information about the proper way to choose the amount of data necessary to accurately represent an original image, but for the purposes of the artist trying to produce a quality image, the main issue is to use enough data so that visual artifacts aren't created. As we'll see in later chapters, one often finds that not having enough color resolution in an original image may only cause problems much later in the process, after many operations have been performed on the image.

Normalized Values

Since different images may be stored at different bit depths, and consequently would be represented by different numerical ranges, it is convenient to refer to the numerical value of a pixel as if the values had all been normalized to floating-point (noninteger) numbers in the range of 0 to 1. Since a pixel with 8 bits per channel can have values in the range of 0 to 255 for each channel, we would divide each value by 255 to come up with these normalized numbers. For instance, a reddish yellow pixel in an 8-bit image might have RGB values of (255, 100, 0). This pixel's normalized values would be specified as (1.0, 0.39, 0). By the same method, a 10-bit image (which can have a range of 2^{10}, or 1024, values per channel) would have every value divided by 1023 to determine its normalized representation.

There are a number of reasons why this is a useful convention. First of all, it frees the user from having to worry about the bit depth of the source images that are being worked with. A good compositing system should be able to deal with images of a variety of differing bit depths and combine them easily and transparently. Additionally, math operations between images are made much easier. For instance, it is a normal mathematical assumption that multiplying any number by 1 will not change the value of the number. If we multiply an image by another image, it is convenient that any pixel that is white (1, 1, 1) does not change the value of whatever it is multiplied by, and multiplying every pixel in an image by a pure white image (i.e., a constant value of 1) will leave the image unaffected. Throughout this book we will assume that an RGB triplet of (1, 1, 1) refers to a 100% white pixel, a triplet of (0, 0, 0) is a 100% black pixel, and (0.5, 0.5, 0.5) is 50% gray.

Additional Channels

In addition to the three color channels, there is often a fourth channel, the **alpha channel**, that can be stored with an image. It is used to determine the transparency of various pixels in the image. This channel is also known as the **matte channel**, and we will use the two terms interchangeably throughout this book. As you'll come to see, nearly all compositing is based on the concept of the matte. Chapter

4 will cover the basic ideas behind mattes and the matte channel, and Chapter 5 will go into greater detail about some specialized processes used to create and manipulate these matte images.

The HSV Color Representation

Up until now, we have specified the color of an image as being based on red, green, and blue components. This model, or **color space**, is certainly the most common way to represent color when using a computer, but it is hardly the only way. A particularly useful alternate method for representing (and manipulating) the colors of an image is known as the **HSV color space**. "HSV" refers to the hue, saturation, and value of a pixel.[5] In many ways, HSV space is a much more intuitive method of dealing with color, since it uses terms that match more closely with the way a layperson talks about color. When speaking of color conversationally, instead of characterizing a color as having 85% red, 0% green, and 90% blue, we would tend to say that the color is a "saturated magenta." The HSV model follows this thinking process, while still giving the user precise definition and control.

The **hue** of a pixel refers to its basic color—red or yellow or violet or magenta, for instance. It is usually represented in the range of 0 to 360, referring to the color's location (in degrees) around a circular color palette. Plate 6a shows an example of such a circular palette. In this example, the color located at 90° corresponds to a yellow green, and pure blue is located at exactly 240°.

Saturation is the brilliance or purity of the specific hue that is present in the pixel. If we look again at our color wheel in Plate 6a, colors on the perimeter are fully saturated, and the saturation decreases as you move to the center of the wheel.

Value, for the most part, can just be thought of as the *brightness* of the color, although strictly speaking it is defined to be the maximum of red, green, or blue values. Trying to represent this third component means that we need to move beyond a two-dimensional graph, and so you should now look at Plate 6b. The value is graphed along the third axis, with the lowest value, black, being located at the bottom of the cylinder. White, the highest brightness value, is consequently located at the opposite end.

Even though we've talked about the HSV color space as an alternate method of representing data, it is generally *not* used to store images. Rather, it is much

---

[5] Variations on the HSV model include HSL and HSB, in which the third component is either lightness or brightness, respectively. HSV seems to be slightly more common with digital compositors, but just about everything we talk about applies equally well to these other models.

more commonly employed as a useful paradigm for manipulating colors. Chapter 3 covers this topic in greater detail.

IMAGE INPUT DEVICES

As we discussed earlier, images that are intended to be used in a digital composite can come from a variety of sources. Those images that were originally created by nondigital methods will need to be scanned into the computer, or **digitized**, before we can use them. "Still" images, images that are not part of a moving sequence, may be simply brought into the system using a wide variety of single-image scanning devices. Flatbed scanners for print material are common, and slide scanners can be used to digitize images that were originally captured on negative or transparency film. Most of the imagery that we will work with when creating digital composites will be sequences of images, not just a single still frame. Sequences of scanned images will probably come from one of two places: either video or film.

Video images, captured with a video camera, can simply be passed through an encoder to create digital data. High-end video storage formats, such as D-1, are already considered to be digitally encoded, and you merely need to have the equipment to transfer the files to your compositing system. Of course, there are a huge number of different video formats, each with its own set of protocols and specifications. This topic will be discussed further in Chapter 10.

Digitizing images that originated on film necessitates a somewhat different approach, and the use of a film scanner is required. Until very recently, film scanners were typically custom-made, proprietary systems that were built by companies for their own internal use. Within the last few years, several companies (such as Kodak and Imagica) have begun to provide off-the-shelf scanners to whomever wishes to buy them. The process of scanning a piece of film is conceptually straightforward. A light source illuminates the film from one side, and an array of sensors mounted on the other side captures the transmitted color. Depending on the device, the array is usually only a single pixel tall but wide enough to capture a full scan line, and the film moves slowly across this array each time a new scan line is captured. The rate at which a film frame can be scanned at high resolution will vary depending on the resolution you are trying to capture as well as the scanner itself—typical speeds can range from only a second or two per frame up to several minutes per frame.

If the intention is to transfer film directly to video, the scanning can be accomplished with a higher speed piece of equipment known as a **telecine** device. Telecine hardware is usually capable of digitizing images in real time, running the film at a rate of 24 frames per second, but the quality is not as high.

Typically, the original negative of the film is scanned, and then digital values are inverted to produce an image that looks correct. By scanning the negative, one is ensured of obtaining the highest possible image quality without suffering any generational loss from creating an intermediate print. The only disadvantage of this technique is that the original negative is essentially irreplaceable, and if it is somehow damaged during the scanning process, the consequences can be dire.

The science of how analog information (such as a piece of film or a video signal) can best be converted to a digital representation is a potentially immense topic. But, as mentioned, the important thing to remember is that the digitization of any image produces something that has a finite, limited amount of information associated with it. The only way to increase the amount of useful information in an image is to re-sample it at a higher resolution, and any additional processing that is applied to an image that has already been sampled can only decrease the accuracy of the data representation. This concept will be repeated and continually referenced throughout this book, as it is critical to always keep in mind the limitations of the images which with one is working.

DIGITAL IMAGE FILE FORMATS

Now that we have a digital representation of an image, presumably residing in the memory of the system used for the digitization, we need to store it in a file on the system's disk. There are a number of different **file formats** that one may choose to store an image. Each format has its own advantages and drawbacks, and as such there is no universally accepted standard format that everyone agrees is the best. File formats vary in their ability to handle a variety of important features, many of which are listed here.

File Format Features

Variable Bit Depths

Certain file formats may be limited to storing images with only a limited or fixed number of bits of data per channel. Other formats are flexible enough that they can store image data of arbitrary bit depth. A good compositing system should be able to read files with different bit depths and allow the user to combine these images without worrying about the original bit depths.

Different Spatial Resolutions

Although most popular formats can be used to store images of any spatial resolution, certain formats (particularly those that are designed to represent only video images) may be limited to specific, fixed resolutions.

Compression

In order to limit the amount of disk space that is used by an image file, certain file formats will automatically compress the image using a predetermined compression scheme. Other file formats may allow you to explicitly specify the type of compression that you wish to use, and of course there are some formats that use no compression whatsoever. The next section gives some information about the different types of compression that are commonly used with image data and also discusses which types of images work well with which compression schemes.

Comment Information in a Header

Some image formats may allow for just about any sort of commentary to be stored with an image, whereas other formats may have a set of predefined niches for comments. Typical information would be the date/time an image was created, the name of the user or piece of software that created the image, or information about any sort of color-correction or encoding schemes that may have been applied to the image before its storage.

Additional Image Channels

Beyond the traditional three color channels, images will often have additional channels that can be carried along with them, such as a matte channel for transparency information or a *Z*-depth channel for determining the relative distance that any given pixel is from a certain fixed location. Some formats will allow the user to store any number of arbitrary additional channels with an image. This can be useful as a method of preserving the identity of the multiple layers that are mixed together to produce a final image.

Vendor Implementations of File Formats

It should be noted that it is up to the vendor of a particular piece of software to follow the proper specifications for any file formats that they claim to support. There are dozens of examples where two different packages may both claim to support the TIFF file format, for example, yet neither package can read the TIFF files created by the other. Beware of these potential conflicts: It is always a good idea to double-check the entire pipeline that an image might travel before you embark on the creation of large amounts of data in a particular format.

We've included a list of the more popular file formats, along with some of their different characteristics, in Appendix C.

Compression

Because high-resolution images can take up a huge amount of disk space, it is often desirable to compress them. There are a number of techniques for this, some

of which will decrease the quality of the image, others which will not. If the compression scheme used to store an image can be completely reversed, so that the decompressed image is digitally identical to the original, then the compression scheme is said to be **lossless**. If, however, it is impossible to completely recreate the original from the compressed version, then we refer to this as a **lossy** compression scheme.

As you will discover, there are almost always opportunities to trade quality for space efficiency. In the case of image storage, we may wish to sacrifice some visual quality to decrease the amount of disk space it takes to store a sequence of images. Fortunately, there are a number of excellent lossless methods as well. To get an idea about one of the methods used to compress images without loss of information, we will look at a technique known as **run-length encoding**. This is just one of many techniques, but it is a fairly common one and examining it will raise most of the typical issues one confronts when discussing image compression.

Run-Length Encoding

Run-length encoding (or RLE) is a fairly simple technique, yet in the best-case scenario, the resulting compression can be significant. Consider the image in Figure 2.2, a white cross on a black background. This figure is an extremely low-

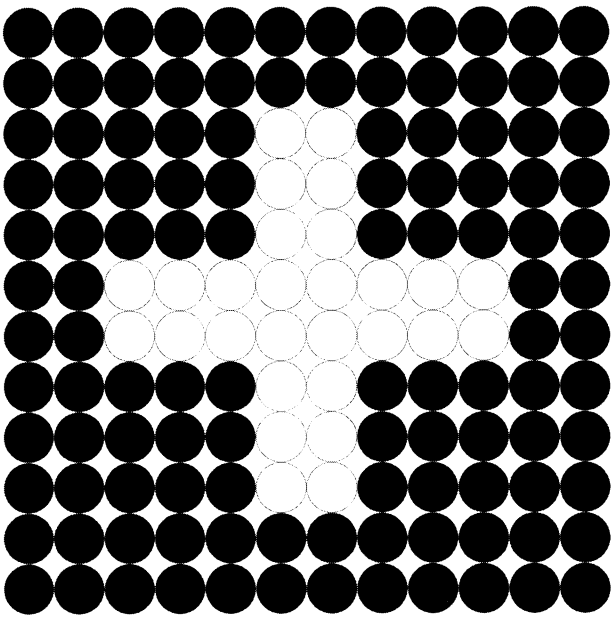


Figure 2.2 *12 × 12 image enlarged to show individual pixels.*

resolution image (12 × 12), enlarged to show the individual pixels. Also, to keep things simple, consider it to be a black-and-white image, so that we only need a single bit of information to determine a pixel's color. In this case, 0 will be a black pixel, 1 will be white. Thus, a numerical representation of this image would be as follows:

0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 1 1 0 0 0 0 0
0 0 0 0 0 1 1 0 0 0 0 0
0 0 0 0 0 1 1 0 0 0 0 0
0 0 1 1 1 1 1 1 1 1 0 0
0 0 1 1 1 1 1 1 1 1 0 0
0 0 0 0 0 1 1 0 0 0 0 0
0 0 0 0 0 1 1 0 0 0 0 0
0 0 0 0 0 1 1 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0

As you can see, we are using 144 characters in a simple matrix to accurately define this image.

Run-length encoding works on the principle of analyzing the image and replacing redundant information with a more efficient representation. For instance, the first line of our image above is a string of twelve 0s. This could be replaced with a simple notation such as 12:0. We've gone from using 12 characters to describe the first line of the image to a 4-character description. Using this convention, we can encode the entire image as follows:

12:0
12:0
5:0, 2:1, 5:0
5:0, 2:1, 5:0
5:0, 2:1, 5:0
2:0, 8:1, 2:0
2:0, 8:1, 2:0
5:0, 2:1, 5:0
5:0, 2:1, 5:0
5:0, 2:1, 5:0
12:0
12:0

The total number of characters we have used to represent the image using this new notation is 104. We have reduced the amount of information used to store

the image by nearly 30%, yet the original image can be reconstructed with absolutely no loss of detail.

It should be immediately obvious, however, that this method may not always produce the same amount of compression if a different image is used. If we had an image that was solid white, the compression ratio could be enormous, but if we had an image that looks like Figure 2.3, then our same compression scheme would give the following results:

1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1
1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0
1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1
1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0
1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1
1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0
1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1
1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0
1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1
1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0
1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1
1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0, 1:1, 1:0

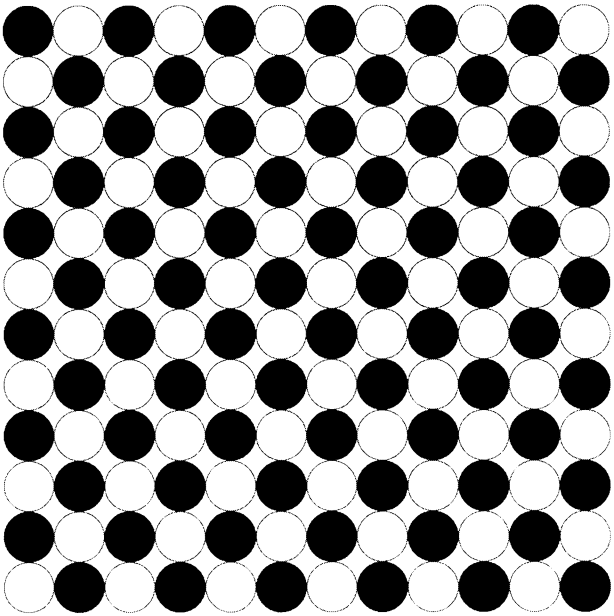


Figure 2.3 *12 × 12 image with alternating and white pixels.*

This change would be a bad thing, since we're now using 562 symbols to represent the original 144.[6]

The point of this example is not to prove anything specific about the run-length encoding method, but rather to underscore the fact that not all encoding methods work well for all types of images. There are certainly more efficient methods for encoding images, and in fact there are far more efficient variations of the run-length encoding scheme than the one shown here. As it turns out, run-length encoding tends to be a fairly good encoding method for computer-generated imagery that is produced by a 3D rendering package because this sort of image tends to have larger expanses of identically colored pixels. On the other hand, run-length encoding is generally a less effective method for storing images that have been digitized from an analog source, such as film or video, since grain, noise, and other irregularities inherent in a natural scene will cause nearly every pixel to be different from its adjacent neighbors.

Lossy Compression

As mentioned previously, a process like run-length encoding is lossless—no information is discarded when an image is stored in a format that uses this scheme. However, there are a number of formats that can compress the storage space necessary for an image by dramatic amounts as long as the user is willing to accept some loss of quality. Probably the most popular of these lossy formats is the one that uses a compression scheme defined by the Joint Photographic Experts Group. Generally, these images are said to be in **JPEG** (pronounced "jay-peg") format. The JPEG format has a number of advantages. First of all, it works particularly well on images that come from film or video, in part because the artifacts it introduces are designed to be less noticeable in areas with some noise. In particular, JPEG compression attempts to maintain brightness and contrast information (which the human eye is very sensitive to) at the expense of some color definition. The format also has the advantage that the user can specify the amount of compression that is used when storing the file. Consequently, intelligent decisions can be made about the trade-off between storage space and quality.

Consider Figure 2.4, in which the original image is shown in part (a) and the two compressed images in parts (b) and (c). Figure 2.4a is the original image, Figure 2.4b has been compressed with a quality level that is considered to be 90% as good as the original, and Figure 2.4c compressed with a quality level of only

---

[6] All but the most basic run-length encoding schemes are intelligent enough to deal with situations in which the encoding would increase the file's size. A final "sanity check" is performed after the image is encoded; if the encoded version is larger than the unencoded original, then the file will be written without any additional attempts at compression.

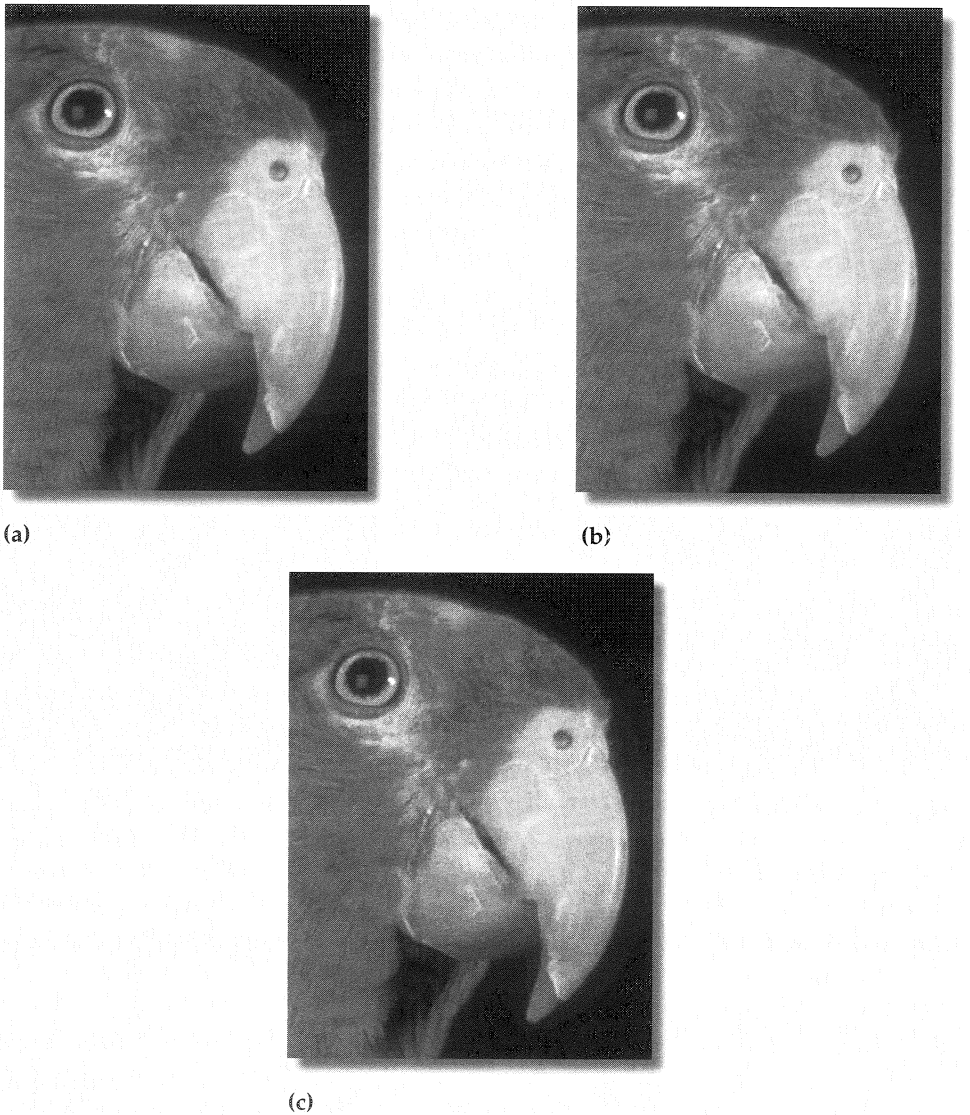


Figure 2.4 *Compression versus image quality. (a) Uncompressed original image. (b) Image compressed with 90% JPEG quality. (c) Image compressed with 10% JPEG quality.*

10%. When we examine the file sizes of the images that result from these compressions, we find that Figure 2.4b takes up less than 1/16th the space of the original, yet has suffered only minor visual degradation. Figure 2.4c is noticeably degraded, yet can be stored using less than 1/200th of the disk space needed to store the original!

JPEG compression is by no means an ideal scheme for compressing every type of image. As mentioned earlier, synthetic or computer-generated scenes will tend to compress quite well using lossless schemes, and in fact the artifacts introduced by JPEG compression are *most* noticeable on synthetic images. Look at Figure 2.5a as compared with 2.5b. Figure 2.5b is a close-up of a section of an image that was compressed for storage using JPEG compression. When decompressed, we can

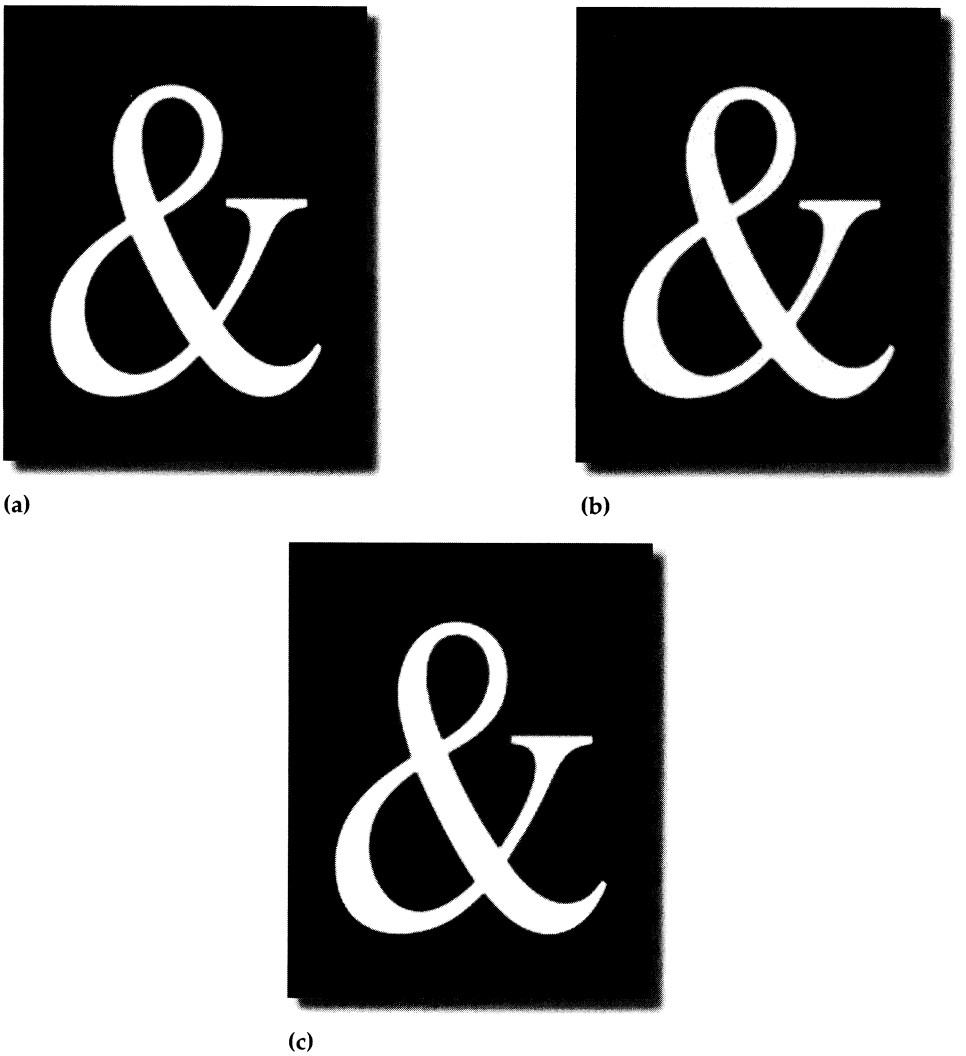


Figure 2.5 *Compression and types of images. (a) Original sample image. (b) Image after JPEG compression, showing artifacts. (c) Image after GIF compression, with little or no noticeable artifacts.*

see that noticeable artifacts around the edges of the text have obviously been introduced. The compression reduced the file size by about two-thirds, but the quality loss may be unacceptable. This is particularly true when you consider Figure 2.5c. This image was saved in the GIF file format, which uses a limited color palette coupled with a compression scheme that is well suited to this type of image. Figure 2.5c actually takes up slightly *less* disk space than it took to store 2.5b, yet appears to be far more accurate.[7]

The only foolproof method for determining the best possible compression scheme for a particular image is to do an exhaustive test. This goal is encouraged, certainly, but it should also be stressed that the blind pursuit of compression is *not* the best way to approach the issue of deciding on a file format. Ultimately, the amount of compression that can be obtained should only be one of many factors that one should take into account. The next section will encourage you to consider a multitude of issues before you make your decision.

Choosing a File Format

By now you've probably realized that there are no definitive guidelines that can help us determine exactly which file formats should be used for a given type of image. Certainly the content of the image is one of the things that we may consider when choosing a format, but the primary determining factor will be the actual scenario for which the images are being used. The only definable guideline is to choose the best compression possible that still produces the desired image quality. Although the amount of compression is an easily measurable quantity, the "desired image quality" is a bit more subjective.

If you are working in a situation in which quality is the top priority, then generally you will not want to use lossy compression schemes at all. You will want every image that you create to be as high quality as possible. But even if you know that your final output images are going to be something like JPEG, it is probably a good idea to use a lossless format for all of the intermediate compositing work that you do. That way you will not be subject to quality loss each time you write out one of these images, accumulating artifacts with each step. Once you have produced your final imagery, *then* convert them to your space-saving file format, choosing the quality level that you find acceptable.

Always be aware of whether the file format you are using is going to lose data, and don't be led into a false sense of security just because you think you are

---

[7] If you look around the Internet, you can find countless examples of people using GIF files to store digitized images and JPEG files to store images of clean text and illustrations. Not only does this practice waste huge amounts of disk space on the server where the files are located, it also causes Web page updates to be dramatically slower than necessary.

using a lossless format. Even if the format doesn't do any explicit compression, there may be some other factor that reduces the image quality. It may not support the bit depth that you need, for instance, which can translate into a huge quality degradation if you are not careful. This information is not always obvious, but a little research into the file format in question (and your software's support of this format) can prevent some catastrophic consequences.

A number of other issues relating to this topic are dealt with in Chapter 11, "Quality and Efficiency."

Nonlinear Color Spaces

In addition to the standard data-compression algorithms mentioned earlier, there is a class of techniques that use specialized color encoding before an image is stored in order to make better use of the bit depth available. Images that are encoded in this fashion are said to be stored in a **nonlinear color space**. Although the topic is fairly complex, the basic process of converting an image into a nonlinear color space is essentially quite simple. First, a specialized color or brightness correction is applied to the image. Then, the image is saved into a lower bit depth than the one in which it was originally stored. The lower bit depth obviously reduces the amount of disk space that is needed to store the image, while the specialized color correction helps to bias the data loss so that more important information is retained.

There are a number of different scenarios that make use of nonlinear color space encoding. In the video world, it is usually referred to as the "gamma" with which an image is stored, while those compositors who work with film imagery will more commonly encounter files stored in the "logarithmic color space" that is used with certain film-specific file formats. To be sure, there are many scenarios in which you may never need to deal with nonlinear color spaces, but both film and video make use of these techniques. Understanding them can therefore be extremely useful in certain situations.

A more precise description of the reasoning behind the use of nonlinear color spaces, as well as the proper way to work with (and even view) such images, is not a trivial subject. This fact, coupled with the consideration that many compositors will not need to deal with nonlinear data at all, has prompted us to reserve any further discussion of the topic for Chapter 15.

CHAPTER THREE

Basic Image Manipulation

Now that we have a basis of understanding for the digital representation of images, we can start to look at methods for modifying these images. Most of the tools we will be discussing initially fall into the category of traditional **image processing** operations.

Throughout this chapter, and really throughout the rest of the book, we will be discussing tools and procedures that modify images. It is extremely important to understand that these modifications do not come without a price. Of course it will take a certain amount of time to perform any given operation, which must be planned for. But more important is the fact that just about *any* image manipulation will ultimately degrade the image. In some cases this degradation will be slight, even imperceptible, but in others the problem can be extreme. A digital image starts with a finite amount of data, and each step in the compositing process can cause some of that data to be lost. The issue of how to recognize and minimize any image degradation is covered in much greater detail in Chapter 11, but you will also find related discussions throughout the book. Because the problem can occur at every single step of the process, it is not possible to present an overall solution. Rather, a good compositing artist needs to learn which operations (or combination of operations) are more likely to introduce artifacts or cause data loss.

We will certainly not be able to provide a detailed description of every possible image processing operator, but will instead discuss several of the most common as well as those that best illustrate a specific concept. Most other tools are merely variations or combinations of the ones that we will be discussing. Appendix A

gives a short description of several more of these tools, and the bibliography lists some relevant books that are more dedicated to the topic.

We will first discuss *single-source* operators, that is, operators that take a single image or sequence as their input and produce a new, modified output. This process is in contrast to what will be discussed in the next chapter, where we see how two (or more) different sources can be combined using multisource operators to produce a result.

TERMINOLOGY

Throughout our discussion of various tools (in particular those which modify the color of an image), we will occasionally show graphs of how the pixels in the image are affected. Graphing the behavior of a function is a useful way of getting a more intuitive feel for how the image will be modified. Our graphs will all follow the format shown in Figure 3.1. Here, the *X*-axis represents the value of pixels in the original, source image. The *Y*-axis is the value to which the pixel has been converted after our color-correction operator has been applied. Thus, an uncorrected image would look like the graph in Figure 3.1, in which a pixel that starts with a value of 0.5 would still have a value of 0.5 after the operation.

Let's assume that we take an image and multiply every pixel by a certain constant value. This is known as a "brightness operation." We'll start with the image shown in Plate 4. The application of a brightness of 2.0 produces Plate 7;

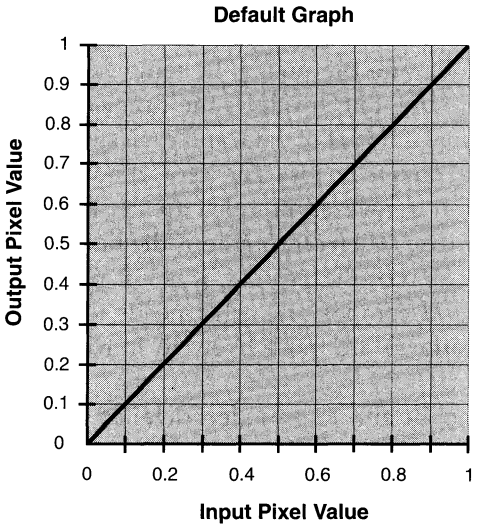


Figure 3.1 *Graph of an unmodified image.*

the graph of the "brightness 2.0" operation is shown in Figure 3.2. Examining the graph, we can see that a pixel value of 0.5 (midtone gray) maps to a pixel value of 1.0 (white) in the new image. In other words, every pixel is now twice as bright as it was before the operation. As usual, normal pixel values are assumed to range from 0 to 1, and any values that are pushed outside this range may or may not be **clipped**, depending on the compositing tool you are using. In our current example, any pixels whose values were pushed out of range were clipped, which is why any input pixels that were originally above 0.5 are now all equal to 1.0. Chapter 11 discusses the issue of data clipping in greater detail.

For some of the more simple tools, we also provide a brief equation to describe the operator. For these equations, we usually treat the entire image as if it were a single variable. In these cases, we use the following conventions:

I = Input image

O = Output image

Thus, $O = I \times 2.0$ would refer to our brightness example, in which every pixel in the input image was multiplied by 2.0 to produce the output image.

Now that we have our terminology in place, we're ready to start discussing some of the basic image processing tools we can use when compositing a scene. We will go over a number of different tools, but we will not yet discuss how these tools can interact with one another or how they can be combined to produce a

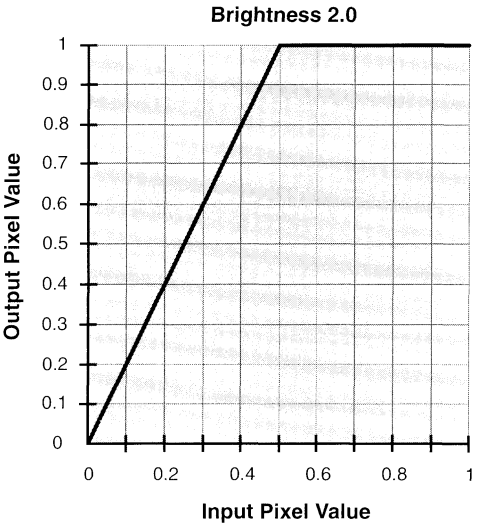


Figure 3.2 *Brightness 2.0 graph.*

multiple-step compositing script. Chapter 11 provides more information on this topic.

Single-source image processing operations can be broken down into three basic categories: color manipulations, spatial filtering, and geometric transformations. Our example, the brightness operation, would be considered a color manipulation, and we will discuss this category first.

COLOR MANIPULATIONS

As mentioned, we consider the brightness operation that we've just covered to be a color manipulation. While the layperson might not consider an apparent brightness change to be a shift in color, we'll see that it is really just a simplified case of an operation that *will* produce a visible color change. Thus, in the image processing and digital compositing world, operations that affect brightness and contrast are all classified as color-correction tools. Brightness is a specialized version of a more general tool, called simply the "Multiply" operator.

RGB Multiply

The brightness example had us multiply every pixel by a constant value to produce a new result. But if you recall from the last chapter, we often find it useful to work with digital images as a group of channels instead of whole pixels. If this is the case, we can actually apply a multiplication to each component of the pixel separately, and not necessarily equally. Thus, instead of multiplying each channel by 2.0 to create an overall brightness shift, we could apply a multiplier of 0.1 to the red channel, a multiplier of 1.25 to the green channel, and leave the blue channel unmodified (i.e., multiply it by 1.0). The result of such a procedure is shown in Plate 8. As you can tell, we've dropped the amount of red in the image significantly and boosted the green a bit, producing an image that is much more greenish blue than the original. The graph of the equivalent function is shown in Figure 3.3.

Most of the color-correction operators that we discuss can be applied either to all the channels of an image equally or to individual channels in varying amounts. When applied equally, the result will tend to be an overall brightness or contrast modification. When different amounts are applied to different channels, a visual color shift will usually take place as well. For the most part, our examples show the operators as they apply to the image as a whole, but you should assume that they can also be limited to individual channels as well.

Add

Instead of affecting the apparent brightness of an image by multiplying, we can instead add (or subtract) a constant value from each pixel. Consider the situation

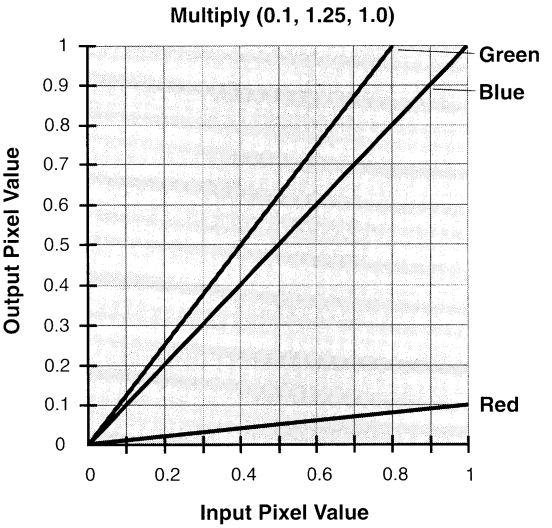


Figure 3.3 *Graph of RGB multiplication.*

in which we wish to add a value of 0.2 to every pixel in the image (O = I + 0.2). The resulting image is shown in Plate 9, with the graph shown in Figure 3.4.

Notice that, unlike the multiplication operation (which keeps the deepest blacks at the same level), the blacks in this example have gone to gray, or "washed out."

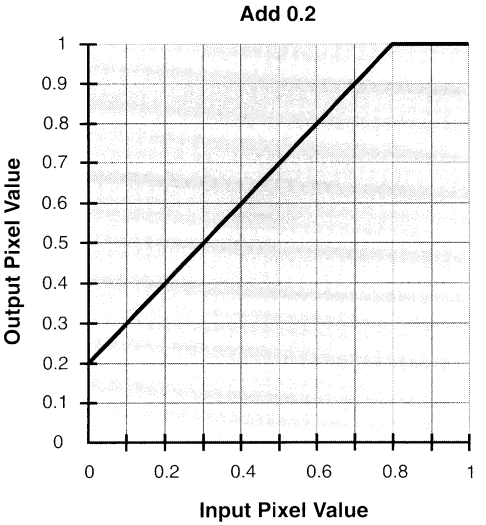


Figure 3.4 *Graph of add 0.2 operation.*

The reason should be obvious: With the Multiply operator, a value of 0 will remain 0 when multiplied by any other number, and small numbers tend to stay small. In the case in which we *add* a value to every pixel, then even the pixels that start as 0 will end up being changed to whatever value we added to the image, in this case 0.2.

We've just discussed two different tools that can change an image to be brighter than it was to begin with. Phrases such as "make it brighter" are ambiguous at best, and it generally ends up being the job of the compositing artist to decide which tool would be best for producing the desired result. Using a multiplication is probably more common, simply because the loss of rich blacks when using the Add operator is usually an undesirable artifact. However, you will notice that both of these tools will push pixels to pure white (a value of 1.0), causing bright areas of an image to look "blown out" or "burnt." Fortunately, there is another tool that is commonly used for making an image appear brighter, yet which doesn't exhibit the same sort of problems, either on the high *or* the low end. This tool is known as the Gamma operator.

Gamma Correction

The Gamma Correction operator uses the following exponential function:

$$O = I^{1/\text{Gamma}}$$

In other words, we raise the value of each pixel to the power of 1 divided by the gamma value supplied. If you have a calculator handy, you can see how different values of gamma will change the brightness of a given pixel. For instance, a pixel whose initial value is 0.5 will end up with a new value of 0.665, if a gamma of 1.7 is used.

The real reason why the Gamma operator is so popular becomes apparent when you examine what happens when you raise 0 to any power: It stays at 0.[1] A similar thing happens when 1.0 is raised to any power: It stays at 1.0. In other words, no matter what gamma correction you apply to an image, pixels with a value of 0 or 1.0 will remain unchanged. The only effect that the gamma operator has will be on nonblack and nonwhite pixels.

Don't worry too much about the math behind the Gamma Correction tool. Rather, examine the graph of its behavior. A gamma correction of 1.7 is depicted

[1] The exception, of course, occurs when you raise anything to the zeroth power, which is defined as producing a result of 1. However, since applying a gamma value of 0 to the above equation would produce a divide-by-zero problem, the generally accepted convention is that applying a gamma of 0 to an image will produce a black frame.

in Figure 3.5 in our graph format, and the effect on our example image is shown in Plate 10. As you can see in the graph, the endpoints of our brightness range remain fixed, midrange values are affected the most, and the effect gradually tapers off as you reach the extremes. Examining the resultant image will show that it is primarily pixels that were of medium brightness that have been boosted. The image does indeed appear brighter, but there are none of the problems we've seen with earlier tools, where the blacks become washed out or bright areas are clipped to pure white. Not only does the image tend to look more natural, but there is also much less chance for the types of data loss that can plague other operators.

Incidentally, please be aware that "gamma" is one of the most overused letters in the Greek alphabet. Ambiguity and confusion can arise because the term is commonly used to refer to a variety of totally unrelated nonlinear functions. Many other disciplines outside the image processing field use the term, and even *within* the field of digital compositing, "gamma" may be used to refer not only to the specific image processing operation we've just discussed but also to a nonlinear data encoding method (as discussed in Chapter 2), or even to the characteristic response of a piece of film to light exposure.

Invert

An extremely simple operator, yet one that will be used a great deal, is the Invert operator. The basic math behind the tool is merely

$$O = (1 - I)$$

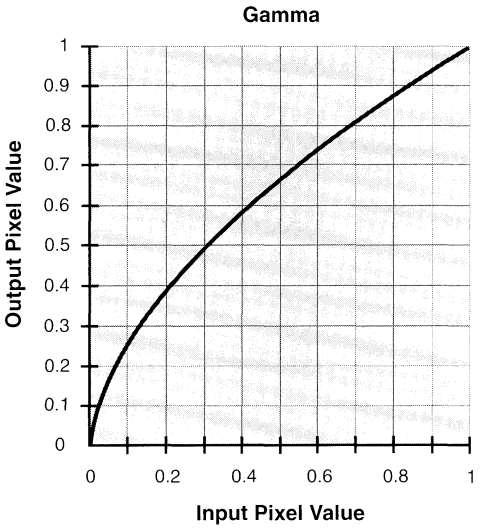


Figure 3.5 *Graph of gamma correction of 1.7.*

In other words, every pixel is replaced by the value of that pixel subtracted from 1.0. When working with color images, the result is an image that appears to be similar to the photographic negative of the original. Colors are converted to their complementary colors, bright areas become dark areas, and vice versa. The Invert operator is also referred to as the Negate operator on many systems.

Although the need to produce the negative of an image may seem limited, a much more common use of the Invert operator is to modify images that are being used as masks or mattes. Such images are discussed further in Chapter 4.

Contrast

The Contrast operator is a tool for changing the brightness relationship between the upper and lower color ranges of an image. Increasing the contrast causes dark areas to get darker and bright areas to get brighter. Decreasing the contrast will bring the apparent intensity of lights and darks closer together. There is no universal standard for exactly how the contrast operator should be implemented, so we'll look at the graphs of a few different possibilities. A very rudimentary contrast could be implemented using a combination of the tools that we've already discussed. For instance, if we wanted to increase the contrast of an image, we could first subtract a constant value and then multiply the result by another constant. The graph of this (from the equation $O = (I - 0.33) \times 3$) is shown in Figure 3.6.

Contrast as applied in this manner is a less than ideal operator, since both the low end and the high end treat the image's data rather harshly. A better system

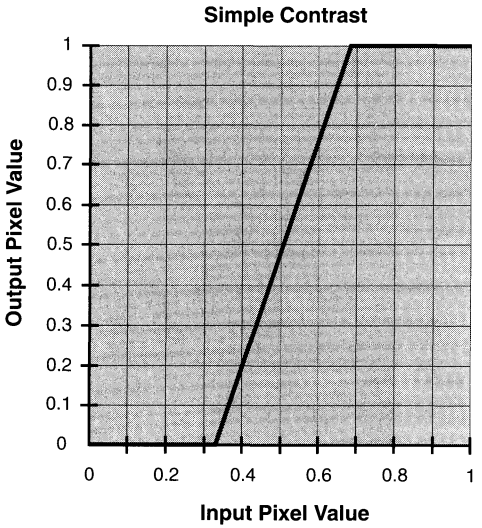


Figure 3.6 *Simple contrast operation.*

is to apply gammalike curves to the upper and lower ranges, as in Figure 3.7. This method tends to give a much cleaner image, particularly at the low and high ends of the brightness spectrum. Plate 11a shows an image that has been affected by the original contrast we showed in Figure 3.6, whereas Plate 11b was modified by our smoother contrast.

Ideally, our operator will also allow us to explicitly choose the boundary between what are considered the highs and the lows in the image, giving us a "biased contrast." In the graph in Figure 3.7 you'll notice that the point at which the curve goes from being concave to convex happens at exactly 0.5. Everything above 50% brightness is made brighter; everything below is made darker. This is fine if our image is relatively normal in its distribution of light and dark areas, but if the image tends to be very dark or bright overall, we may wish to adjust the midpoint where the transition occurs. Given a dark image, in which nothing in the frame is brighter than 0.5, we might want to set our bias point to be at 0.25 instead. If we *didn't* do this, then everything in the scene would become even darker, since all values fall below the threshold for what is considered the "dark" areas of the frame. Figure 3.8 shows a graph of a biased contrast in which the bias is set at 0.25.

Channel Swapping

There may be situations in which one actually wishes to reorder the channels that make up an image—placing the blue channel into the green channel, for instance. Simple reordering within the R, G, and B channels is not used a great deal, since it is a fairly brute-force and nonintuitive method of modifying an image's color.

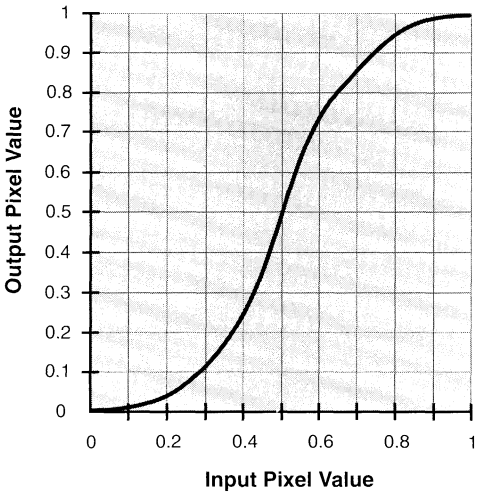


Figure 3.7 *Smoother contrast graph.*

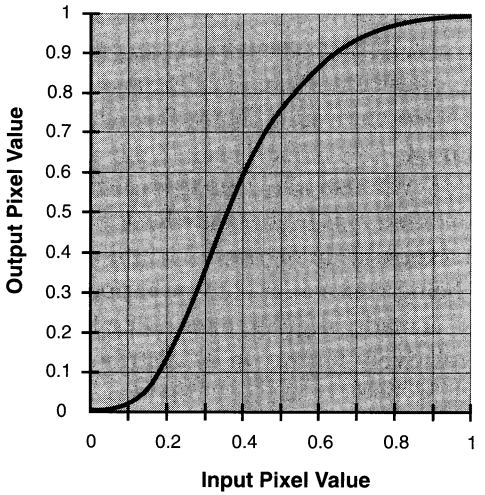


Figure 3.8 *Graph of biased contrast (bias = 0.25).*

More common is the need to move data to and from the matte channel. Channel substitution techniques are also commonly used as the basis for certain color-difference methods, as discussed in Chapter 5. It is usually more common to use channel swapping in conjunction with some sort of expression language, which we will discuss later in this chapter.

HSV Manipulations

As mentioned in Chapter 2, there is an alternate way to represent the colors in an image, known as the HSV model. This model is particularly useful as a tool for color correction, since it can be used to deal with concepts that would normally be very inconvenient were we limited to viewing color as simply a mix of red, green, and blue. For instance, affecting the saturation of an image merely by adjusting RGB values can be rather cumbersome, but using the HSV model allows us to directly access the saturation component, making manipulation trivial. An example is shown in Plate 12, in which the image's saturation has been reduced by about 50%.

Because HSV represents hue in a circular fashion (as shown earlier in Plate 6b), we can now "rotate" all the colors in an image by a certain amount. Plate 13 shows our test image after it was rotated by 180° through the color spectrum. You'll notice that every color in the image has been shifted to its complementary color, while still preserving the brightness and saturation relationships.

Look-up Table Manipulations

We've shown a variety of graphs for various operators in the last several pages, where we plot the output color as it compares with the input color. But instead of merely viewing the results of an operation with such a graph, we ideally should be able to create a graph manually and then allow the software to apply this function to the image directly. In essence, the software will evaluate the value of the graph for every input value and determine what the corresponding output value should be. This input-to-output mapping is known as a **look-up table** (or a **LUT**), and consequently such a feature is usually called a LUT-manipulation tool. Such a tool can give an extremely fine amount of control, even allowing specific color corrections across a narrow range of values. It will also often prove useful as a tool for creating and manipulating mattes. The sample image shown in Plate 14 has had all three channels modified by the set of curves shown in Figure 3.9.

This method of using user-defined curves to manipulate the colors of an image is not limited to merely modifying the brightness of certain regions in a given channel. Given a flexible enough software package, one can choose to arbitrarily modify one channel based on another channel. This operation can be particularly useful if you are able to include HSV color space in the mix. For instance, we may wish to modify saturation based on a particular hue. The diagram in Figure 3.10 shows a tool for performing this operation. The *X*-axis maps out the full range of hues available in an image, while the *Y*-axis controls the relative saturation. In

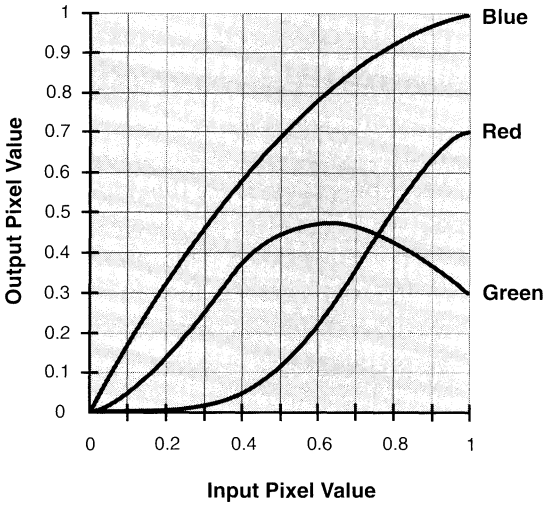


Figure 3.9 *Graph of arbitrary look-up table curves.*

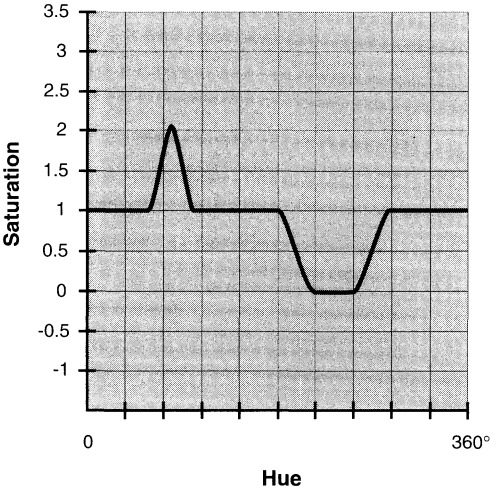


Figure 3.10 *Graph of hue versus saturation operation.*

the example shown, the center of the *Y*-axis is defined to be the point where saturation is unchanged. Pulling the curve upward will increase saturation, while moving it downward will decrease it. As you can see, we have chosen to desaturate anything that is blue, while at the same time we supersaturate a narrow range of green values. The resulting image is shown in Plate 15.

Expression Language

Finally, if all the other built-in techniques are not able to achieve the result you desire, a good compositing system should allow the user the ability to define arbitrary mathematical expressions to modify the color of an image. This type of tool is essentially a simplified computer language with support for a wide variety of expressions and mathematical constructs. It should be robust enough that the user can create very complex expressions. Since this type of tool can vary widely between different packages, we will define a few basic conventions and then give some examples that follow those rules. This pseudocode should be easily translatable to a number of different systems. In general, the syntax for most of these manipulations is assumed to be C-like, and certain built-in keywords and functions are assumed to be predefined.

As was the case with the LUT manipulations, we will use this tool to modify the value of the channels in our image. But instead of creating a user-defined curve, the color change will instead be based on some sort of algorithm or equation. Let's start off with a very simple example, in which we modify the entire image by multiplying each of the R, G, and B channels by a constant value. In this case

we'll modify each channel by 0.75. The expression for such an operation might look like this:

$$R = R \times 0.75$$
$$G = G \times 0.75$$
$$B = B \times 0.75$$

The resulting image would be darkened by 25%. This isn't necessarily the best use of an expression language, since the identical result could have been obtained much more simply by merely applying a brightness of 0.75 to the entire image.

Let's look at another example, one that produces a monochromatic image from the color source by averaging the three channels together:

$$R = (R + G + B)/3$$
$$G = (R + G + B)/3$$
$$B = (R + G + B)/3$$

A more proper expression, which takes into account the fact that the human eye perceives the brightness of red, green, and blue differently, would be obtained by the following:

$$R = (R \times 0.309) + (G \times 0.609) + (B \times 0.082)$$
$$G = (R \times 0.309) + (G \times 0.609) + (B \times 0.082)$$
$$B = (R \times 0.309) + (G \times 0.609) + (B \times 0.082)$$

Again, however, similar results could have been obtained by using other operators, in this case some sort of desaturation or monochrome effect.

Instead, consider a situation in which we wish to have a conditional requirement carried out by the expression. In particular, examine the next scenario, in which only the green channel is being modified. The red and blue channels remain unchanged:

$$R = R$$
$$G = R > G \ ? \ (R + G)/2 : G$$
$$B = B$$

For those of you unfamiliar with the shorthand syntax used in this expression, the English translation is as follows:

> If a pixel's red value is greater than its green value, then set the pixel's green value to be equal to half of the red value plus the green value. If the pixel's red value is not greater than the green value, then leave the pixel's green value unchanged.

The result of the application of this expression is shown in Plate 16. As you can see, well-thought-out equations can quickly produce results that would otherwise have required the combination of several different methods.

SPATIAL FILTERS

So far, all of the image manipulations that we have talked about involved a direct mapping between a given input color and a given output color. This color mapping could be described by a simple function, and the only thing necessary to compute a pixel's output color was the equation and the original color of the pixel. Now we will look at a new class of tool that takes into account not just a single input pixel, but also a small neighborhood of pixels surrounding the input pixel. This type of tool is known as a **spatial filter**.[2]

Convolves

The most common spatial filter is known as a **spatial convolution**, or simply a **convolve**. It iteratively considers a specific pixel, as well as a specific number of adjacent pixels, and then uses a weighted average to determine the value of the output pixel. This group of pixels that will be considered is known as the **kernel**, and is usually a square group of pixels that has an odd number of rows and columns. By far the most common kernels are 3×3 and 5×5, but there are occasions where larger kernels may be used. As we said, the convolve operator uses a weighted average of the existing pixels to determine a new pixel value. To control this weighting, we define a specific **convolution filter** that is used for the computation. This filter is the same size as the kernel we plan to use and contains a series of different numbers in the shape of a square matrix. The numbers in this matrix will dramatically affect the resulting image.

Although usually the compositing artist won't need to do much more than choose a specific filter from a list of options and then allow the computer to apply the convolve process, we'll take a moment to look at an example of how the math is actually applied for a simple 3×3 kernel. We start with the following matrix:

$$\begin{matrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{matrix}$$

[2] The term "spatial filter" is actually rarely used in the digital compositing world. Very often you will see packages try to group the operators that we discuss in this section under either color-correction tools or geometric transformation tools. Since they do not really fit into either category, we have decided to refer to them in a manner that is consistent with most image processing books.

Think of this as a mask that will be laid over our source pixel, where the source pixel is aligned with the center of the matrix and each of the eight neighboring pixels is aligned with its positional counterpart. We then multiply each pixel by the coefficient with which it is aligned. In this example, the value of every neighboring pixel is multiplied by -1, and the source pixel itself is multiplied by 8. Now we add together all of these values to get a new number. This number becomes the value of our new pixel in the destination image. (Numbers above 1.0 or below 0 are clipped.) This process is repeated for every pixel in the source image until we have created a new image, point by point.

If we consider a section of an image whose pixels have the values shown on the left in Figure 3.11, applying the convolution filter we've just discussed will produce the results shown on the right. The particular filter we applied in this example is one that is designed to detect edges in an image. As you can see, it produced a bright pixel wherever there was a transition area in the original image, and produced dark pixels wherever the source image had a constant tone. To see this more clearly, examine the result of applying this same convolution filter to a larger image. Figure 3.12a is our original image, with some very high-contrast transition areas. Figure 3.12b is the resulting image after applying our edge-detection convolve.

There are a large number of different convolution filters that are commonly used for processing images. A few more of these will be mentioned in a moment, but you will probably need to consult a book on image processing if you are looking for a specific effect. Most compositing software will also come with a large number of prepackaged filters already implemented.

Blurring

There are many different algorithms for blurring an image, all of which produce slightly different results, but the basic concepts remain the same. The colors from neighboring pixels are partially averaged with the current pixel to produce a new image. Visually, the end result is an apparent reduction in sharpness.

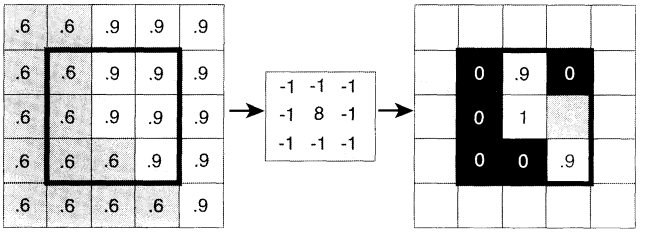
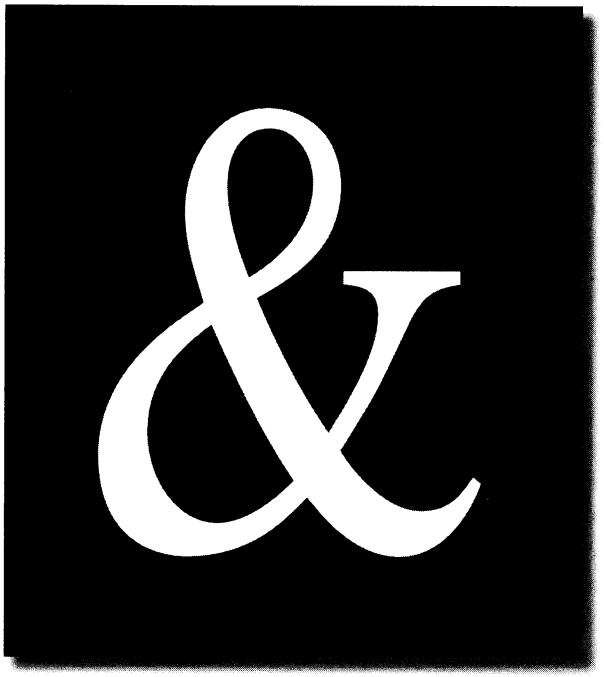
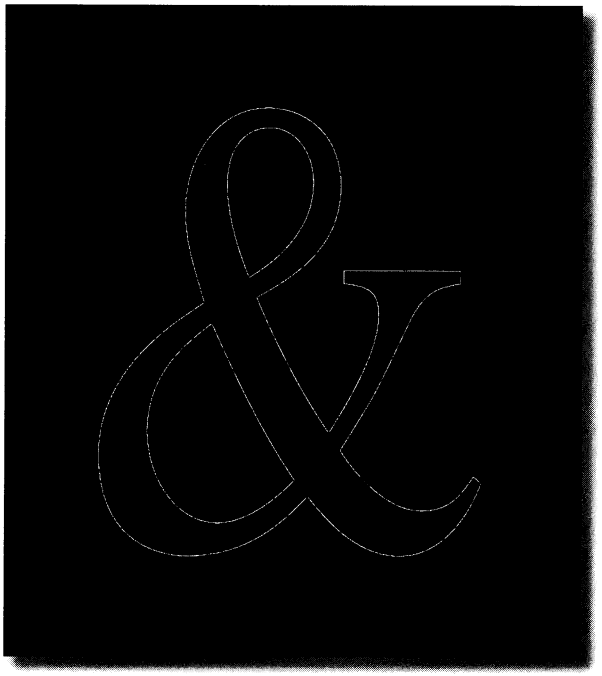


Figure 3.11 *A convolve operation.*



(a)



(b)

Figure 3.12 *(a) Original image. (b) Image after applying an edge-detecting convolve.*

Probably the most common blurring algorithm uses a simple convolve. For instance, a 3 × 3 convolve using the filter

1 1 1
1 1 1
1 1 1

will result in a slight blur, as shown in Figure 3.13. Using a larger kernel, and thus averaging a larger area together, can cause a much greater amount of blurring, albeit at the expense of speed. Convolve-based blur algorithms can be extremely slow, particularly for large amounts of blur.

Also be aware that certain blur algorithms can animate *very* poorly. Visual steps between blur levels can be noticeable, causing blurs that change over time to be unacceptable. As usual, you may want to test the particular system that you are working with to see if it suffers from these problems.

Finally, although simple blur algorithms are often used to simulate the effects of an out-of-focus lens, a visual comparison between a "real" defocus and a simple blur will show decidedly different results. Chapter 12 discusses this topic in greater detail.

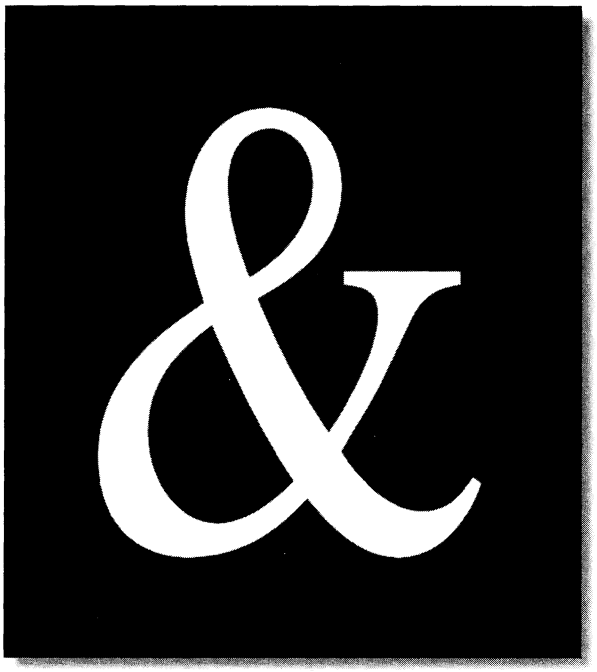


Figure 3.13 *Slightly blurred figure (3 × 3 convolve).*

Sharpen

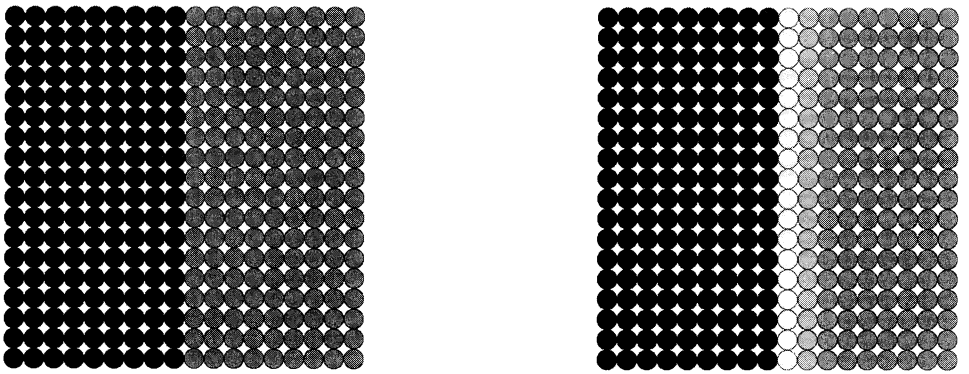
Given the fact that we can blur an image, it seems that one should be able to sharpen an image as well. To a certain extent this *is* possible, although it's really somewhat of a trick. The Sharpen operator actually works by increasing the contrast between areas of transition in an image. This, in turn, is *perceived* by the eye as an increased sharpness. Keep in mind that sharpening tools can never actually create or restore lost information. There is really no detail being added to the scene, only the perception of it. The trick only works up to a certain point, and results can often include undesirable artifacts.

Sharpening can either be done by using a convolve with a filter such as

$$\begin{matrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{matrix}$$

or via other techniques, such as **unsharp masking**. It doesn't really matter which specific mathematical technique you use—the principle is the same, as illustrated by the simplest-case scenario discussed next.

Figure 3.14a shows an enlarged look at an edge in an image—in this case a straight-line transition from dark gray to light gray. Figure 3.14b shows the same area after some sharpening has been applied. If we create a graph based on the values of a horizontal slice of the original image, it would look like Figure 3.15a, whereas Figure 3.14b corresponds to Figure 3.15b. Notice the boosted contrast at the transition juncture: The bright area gets slightly brighter, the dark area slightly darker. A good sharpening tool should let you choose the amount of the sharpening, as well as the extent of the sharpening. If we happen to be using a convolve



(a) **(b)**

Figure 3.14 *Close-up of a sharpen operation. (a) Original edge. (b) Same area after sharpening has been applied.*

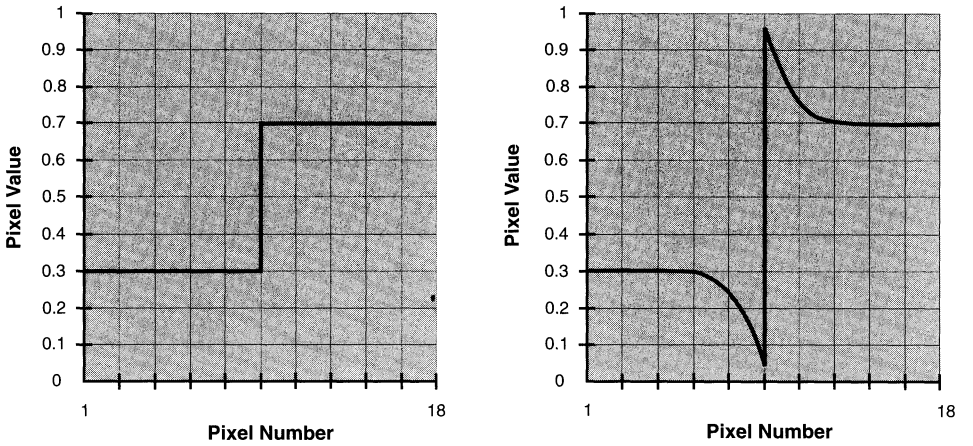


Figure 3.15 *(a) Graph of Figure 3.14a. (b) Graph of Figure 3.14b.*

to do our sharpening, increased sharpening amounts would be produced by increasing the weights of the coefficients in our matrix, and an increased extent would be produced by using a larger matrix.

Plate 17a is an original image, with no sharpening applied. Plate 17b has had a slight sharpening applied, with subtle but noticeable results. Plate 17c has had a far greater amount of sharpening applied, to demonstrate the type of problems that can show up. Now that you are familiar with what sharpening is really doing, it should make sense that oversharpening will cause you to see noticeable **ringing** along strong transition areas, such as the edges of the stones against the sky in the example. You should also be able to see that the sharpening has increased the amount of apparent grain in the image—an artifact that would be even more noticeable if it occurred on a sequence of images.

Median Filter

Certain spatial filters do not use a specific weighted matrix to determine the way pixels are averaged together. Instead, they use other rules to determine their result. One such filter is the **median filter**. Quite simply, it ranks all of the pixels in the kernel in terms of brightness and then changes the value of the pixel in question to be the same as the median, or center value, of this ranking. The net result is that the median filter does an excellent job of removing single-pixel noise artifacts, while causing only a slight reduction in the sharpness of the image.

Figure 3.16a is an image with some noticeable noise spikes—small bright and dark spots. Figure 3.16b is the same image after the application of a median filter.

(a) (b)

Figure 3.16 *(a) Image with noticeable noise spikes. (b) Same image after application of a median filter.*

As you can see, the bulk of the noise has been eliminated. Applying the filter again in a second pass would probably eliminate the rest of the noise. This process does not come without a price, however, since each time a median filter is applied the image will be slightly softened. For this reason, median filtering is usually applied only within a certain threshold. The only pixels that will be replaced with the median of their neighbors are those pixels that vary by more than a certain amount from the original image. Areas that do not have any noise are not changed, and the overall image suffers far less softening than a normal median would produce.

GEOMETRIC TRANSFORMATIONS

The next class of operations we will be discussing all fall under the category of geometric transformations, or simply "transforms." A transform operation causes some or all of the pixels in a given image to change their existing location. Such effects include **panning**, **rotating**, **scaling**, **warping**, and various specialized distortion effects. We'll first consider the three simplest transformation effects: pan, rotate, and scale.

Whenever we talk about moving images around, we must realize that they have to be moved relative to something. It's easiest to just consider that the image is being moved relative to some predefined frame. In most day-to-day compositing work, the first thing we tend to do is define our working resolution, and everything is moved around inside of this frame. What we call a **working resolution** is typically the resolution of the image that will be produced once we are finished with our compositing operations.

For the purposes of the next few examples, let's assume our working resolution is 1200 pixels wide by 900 pixels tall. Let's also assume that the input or source image that we'll be dealing with in these examples is a slightly smaller resolution, say 900 × 600. There is at least one more item we need to know about the system we are working in, namely, what is considered the *origin* of our system. In other words, what part of the frame is considered to have the (*X*,*Y*) location of (0,0). This point may vary between different software packages, but the most common assumption is that the origin is located in the bottom left corner of the frame. Thus, our example image when placed (without any additional transformations) into our working resolution frame is shown in Figure 3.17.

Panning

Let's say we wish to apply a simple translation to the image, offsetting it in both *X* and *Y*. Such a translation is usually referred to as a **pan**.[3] In this case, we will

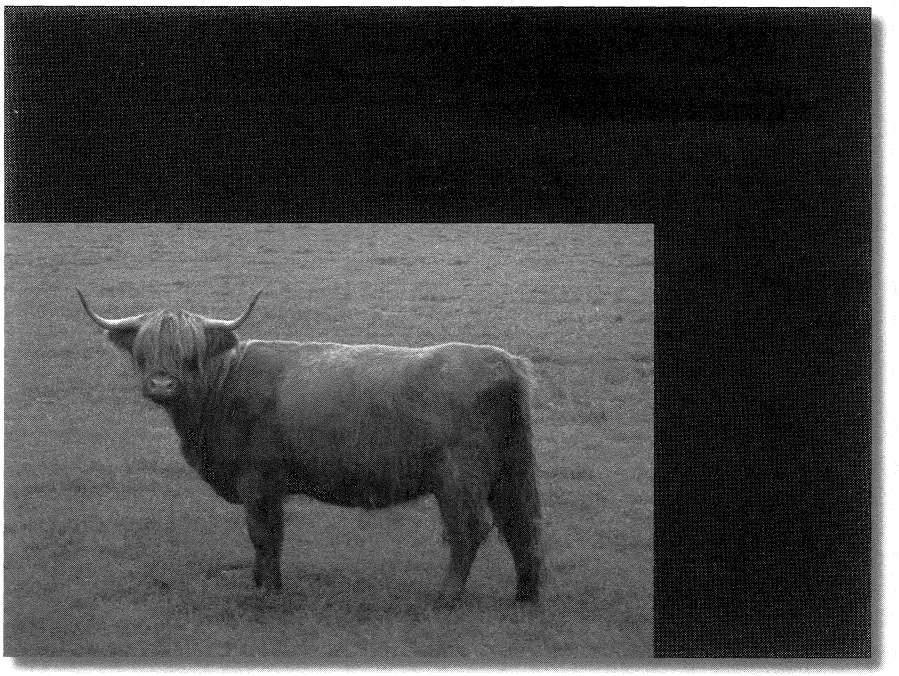


Figure 3.17 *Image in a working resolution frame.*

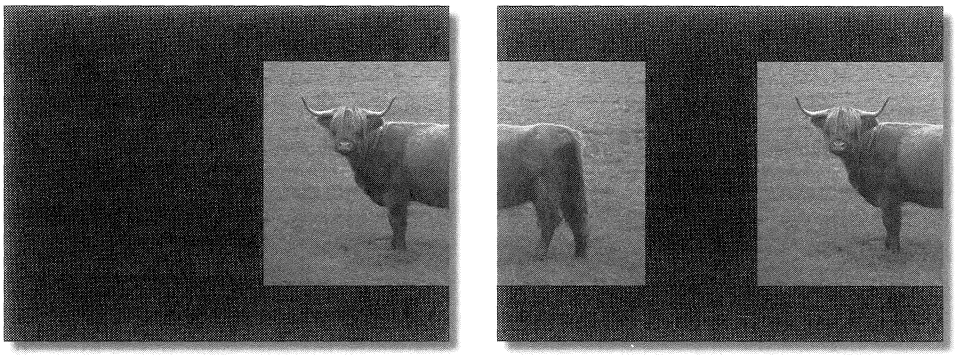
[3] As usual, different software and artists may refer to these operations using different terminology. Some people would call this a simple reposition, or "repo." The term "pan" would then be reserved for a reposition that animates over time. Rather than have a term that is dependent on whether the

pan the image by 150 pixels along both axes. The new image produced is shown in Figure 3.18, with the original image more or less centered within the frame of our new output image.

What happens if we move the input image 700 pixels in *X* instead, causing part of the input image to be moved beyond the borders of our working resolution? The result will usually be something like what is shown in Figure 3.19a. However, the issue of what is done with the rest of the image that has moved out of frame is dependent on the compositing system that is being used. On most systems, the rest of the image will be cropped, or discarded. Any additional transformations to this image will be performed with the truncated image, and any portion of the image that was moved out of the working area will be unrecoverable. A few systems are able to deal with this problem in a more robust fashion, allowing the off-screen information to be preserved so that it can later be brought back into frame if needed. There is also a common option in most systems that lets the user

Figure 3.18 *Image from Figure 3.17 panned by 150 pixels along both axes.*

effect is applied over time, we have decided to refer to either a static or a dynamic reposition as a "pan."



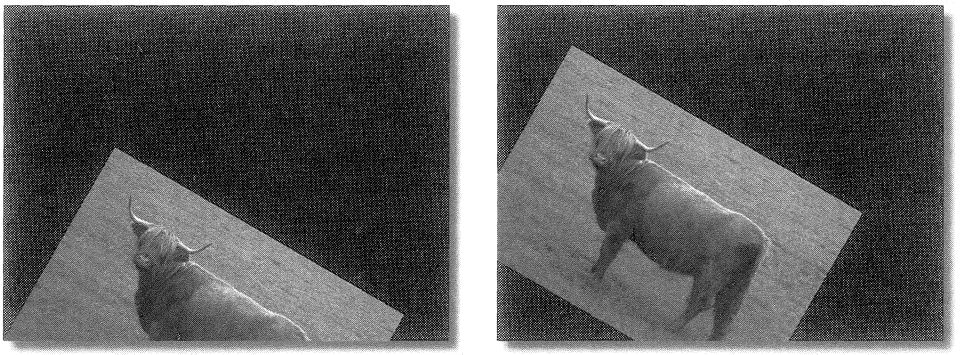
(a) (b)

Figure 3.19 *Panning. (a) Image panned off-screen. (b) Image panned with wrapping.*

specify that the image "wraps" around to the other side of the frame, as shown of this is shown in Figure 3.19b.

Rotation

Let's **rotate** an image now. The two parameters needed to control a simple rotation are the *amount* of the rotation (usually specified in degrees) and the *center* of the rotation. Changing the center of rotation can dramatically affect the result. Compare Figure 3.20a and Figure 3.20b. Both images have been rotated 30° clockwise, but in the first the center of rotation was the origin, (0,0), whereas in the second the image was rotated about a point that is approximately its center.



(a) (b)

Figure 3.20 *Rotation. (a) Image rotated 30° clockwise about the origin. (b) Image rotated 30° clockwise about its approximate center.*

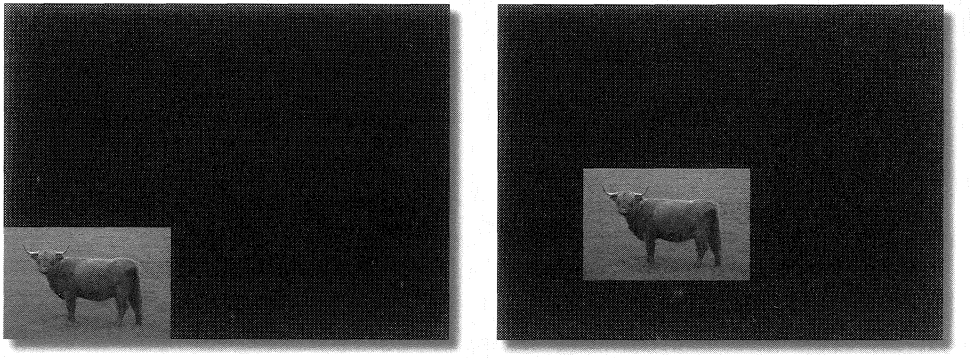
Scale

Now we will look at the case of **scaling** an image. Again, if your compositing system supports it, the ability to scale around a user-defined point is useful. Figures 3.21a and 3.21b show an image scaled down by 50% (or scaled by 0.5) around the same two origins that we used for our rotation examples. If for some reason your compositing system does *not* support scaling around a user-defined origin, don't be too concerned, since it is really just a shortcut that can easily be duplicated by panning the resulting image into the position we desire.

Although this example shows an image scaled by the same amount (0.5) in both X and Y, we can certainly scale nonuniformly instead, say by 0.3 in X and by 1.2 in Y. Note that we can also "flip" or "flop" an image by scaling it by -1 in the X or Y direction, respectively. Remember that flipping an image is *not* the same as merely turning it upside-down (i.e., rotating it by 180°). Instead, flipping produces a *mirror* image along the X axis. This can be very useful when faking shadows or reflections in a composite. Figure 3.22 illustrates the difference between an image that is flipped (3.22a) and one that is merely rotated (3.22b).

3D Transforms

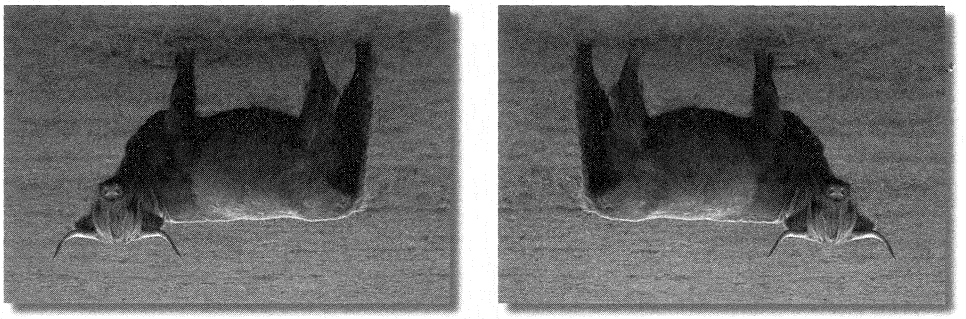
So far we've confined our discussion of various transformations to what are considered to be 2D transforms. We can also choose to rotate an image as if it were being moved about within a 3D environment. Various perspective[4] effects



(a) (b)

Figure 3.21 *Scaling. (a) Image scaled by 50% around the origin. (b) Image scaled by 50% around its approximate center.*

[4] Perspective is discussed in greater detail in Chapter 12.



(a) (b)

Figure 3.22 *(a) Flipped image. (b) Rotated image.*

will consequently be introduced. Without going into great detail to define the information needed to extend our discussion into this third dimension, let's just look at a simple example of transforming an image in 3D space. Figure 3.23 shows an image that has been partially rotated around the *X*-axis. You can see the obvious

Figure 3.23 *Three-dimensional rotation around the X-Axis.*

perspective effects, whereby foreshortening causes the bottom of the image to be compressed and the top of the image to be enlarged.

Even if your compositing system doesn't support the concept of perspective transformations, it may support something known as "corner-pinning," in which the user can manually reposition the four corners of an image to create any arbitrary tetrahedron. The end visual result should be similar.[5] In general, most 3D transformations can be emulated using 2D transforms.

Warping

An even more sophisticated method of distorting an image is known as **warping**. Conceptually, it is easiest to think of warping as if your image were printed on a thin sheet of flexible rubber. This rubber sheet can be pushed and pulled by various amounts in various areas until the desired result is obtained. Image warping is usually controlled by either a grid mesh or a series of splines. (Spline-based systems ultimately create a grid as well—they just do a better job of hiding it from the user.)

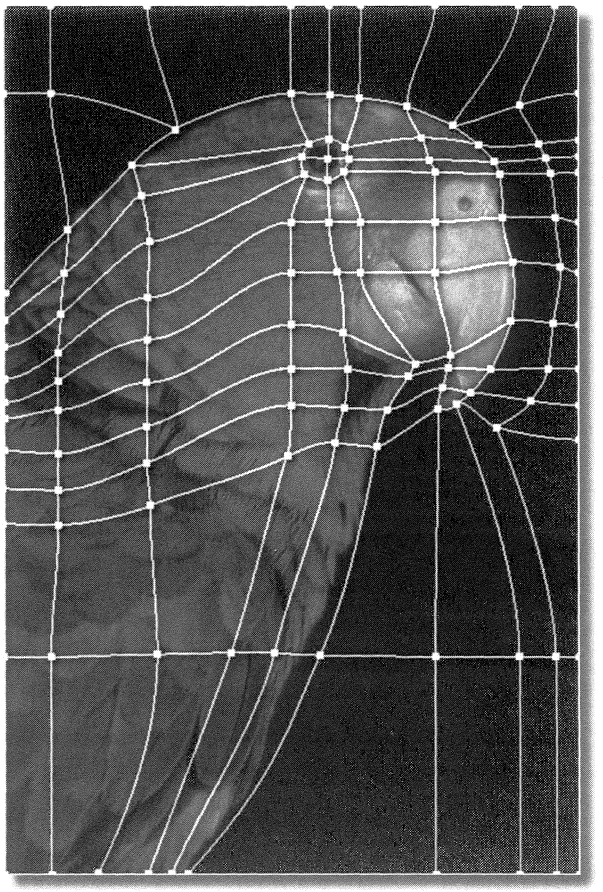
Although warping is a powerful tool for manipulating images to obtain effects that would otherwise be impossible, for illustrative purposes we will use it for a slightly less serious result. Consider Figure 3.24a, which shows our sample image with a grid laid over the top. In order to control the warping of this image, we will manipulate this grid; the corresponding warp will be applied to the image. Thus, if we stretch our grid as shown in the right half of Figure 3.24b, the resulting warped image would be similar to that shown in Figure 3.24c. Later, in Chapter 15, we will also touch on the technique of **morphing**, a sophisticated combination of warping and dissolving between two images over a period of time.

Expression Language

Just as we saw with the color-correction tools, the ideal compositing system will allow you to resort to mathematical expressions to define warping parameters. The syntax will follow the same format as we defined for our color-correction example, only now we will be dealing with *X* and *Y* transformations. The degenerate case, in which we are merely mimicking something simple like an *X* translation, could be represented as

$$X = X + 30$$

[5] The primary difference is that the perspective may not be exactly the same. A good 3D transform will let you choose a true 3D position, which will in turn affect the resulting perspective of the image.



(a)

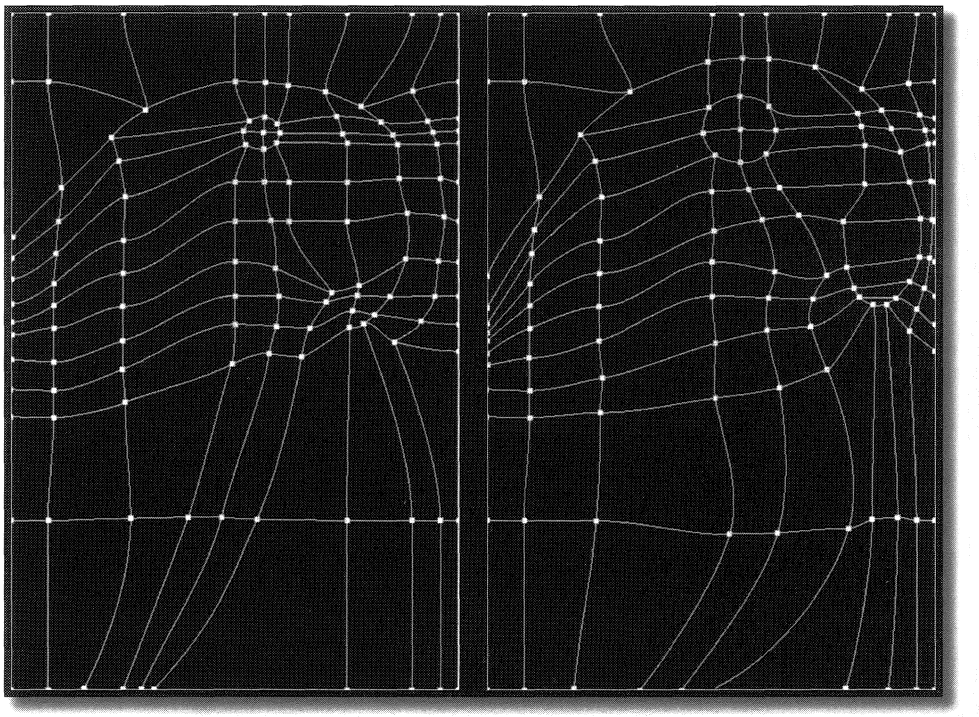
Figure 3.24 *Warping. (a) Sample image with grid laid over it.*

This transformation would shift every pixel to the right by 30 pixels.

More complex equations can easily be extrapolated. For instance, using a periodic function like sine, we can create a wavy offset:

$$Y = Y + 50 \times \sin(X \times 0.02)$$

In this example, the sine function returns a value between -1 and 1 depending on the value of X. The multiplication by 50 magnifies the effect so that pixels are offset in the range of -50 to 50. (Multiplying X by 0.02 before passing it to the sine function will control the frequency of the wave.) The result of this warp operation is shown in Figure 3.25.



(b)

Figure 3.24 *(b) Grid before and after distortion.*

Filtering Algorithms

Every time a geometric transformation is applied to an image, there is a step in which the computer samples the original image and creates a new one. Within this step, there is a choice of what type of **filter** will be used to produce the new image.[6] Consider the situation in which we want to scale an image down to one-tenth its original resolution. There are a number of ways that we could go about this. We could simply use every tenth pixel, for instance. This technique is a very simple filter, generally known as an **impulse filter**. It is a very fast method of reducing an image, but unfortunately not a very good one, since it completely ignores 90% of the data that makes up the original image. Consider the worst-case example of how this can be undesirable. If pixel number 1 and pixel number

[6] From a mathematical point of view, these filters are essentially the same as the spatial filters that we have already described. However, their implementation here is different, and the compositing artist will use them differently.

(c)

Figure 3.24 *(c) Resulting distorted image.*

10 in the image are black, but pixels 2 to 9 are white, our scaled-down image would use a black pixel to represent the new pixel number 1, even though the average value should be closer to white. Better filtering algorithms look at all of the pixels in the original image and selectively average these pixels together to obtain a new pixel that more accurately reflects the data in the original image.

Choosing a specific filter is not just something that we do when scaling an image down (or up). Every geometric transformation (from a rotation to a warp) will use some kind of filter to compute the resulting image. Presumably, your compositing system will allow you to choose a specific filter for most geometric transformations. (If it doesn't, it is probably time to get a new compositing system.) Advanced systems may give you a variety of different filters to choose from by

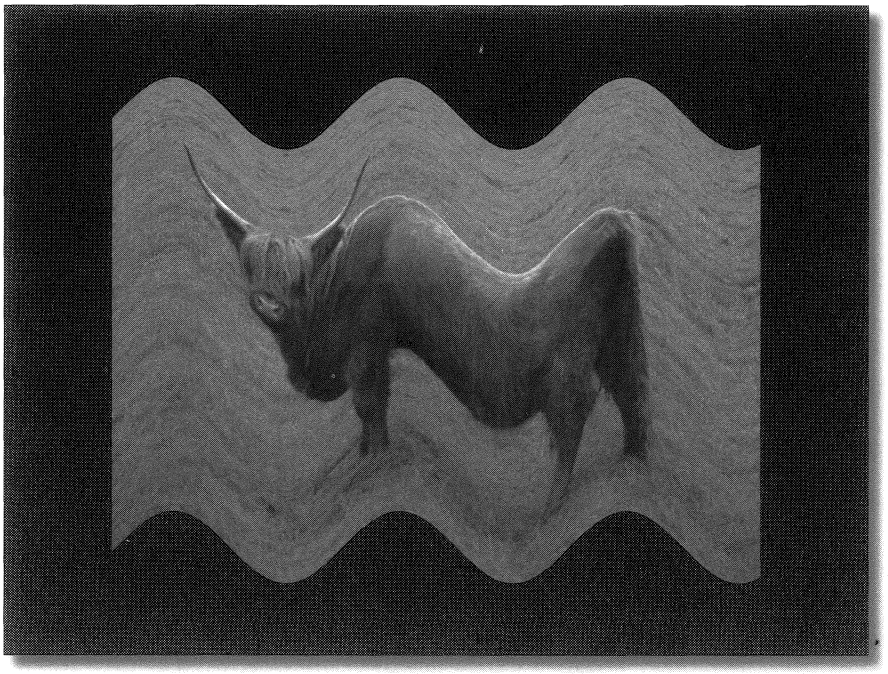


Figure 3.25 *Warping operation performed using a sine function.*

name, whereas less sophisticated systems may just limit your choices to simple options such as "low-quality" or "high-quality."

Different types of filtering can produce vast changes in the quality of the resulting image, particularly when dealing with a moving sequence of images. For instance, when animating a pan be sure to choose a filter that is able to deal with increments of less than a pixel. Such a filter, known as one with **subpixel** accuracy, is designed to prevent your image from popping to whole-pixel boundaries as it moves—a visual artifact that is surprisingly noticeable, even when working at high resolutions. On the other hand, if you are using a geometric transformation to merely reposition an image (i.e., without any scaling, rotation, or animation), then you would be better off choosing a filter that does *not* resample, but instead preserves the original data exactly.

Although the perfect filter does not exist, there are a variety of different filters that are typically used with image processing algorithms. The following list mentions a few of the filters that would normally be available for image resizing, with a look at the pros and cons of each. A resizing filter can act in the same fashion as a sharpening operator, where subtle application will appear to increase sharpness but at the risk of introducing ringing artifacts. Other resampling filters may

introduce noticeable softness into an image or may result in **aliasing**. Higher-quality filters are more expensive in terms of memory and CPU usage, not surprisingly; therefore you should be aware of what type of filter you are using for a given operation. The use of a lower-quality filter for intermediate test work may be justified, as long as you intend to switch to a higher-quality one for the generation of your final imagery.

- The **impulse filter** is the fastest method for resampling an image, since it will sample only a single pixel in the original image to determine the value for a given pixel in the new image. It is also known as a **Dirac filter** or a **nearest-neighbor filter**. Although fast, it generally produces a significant amount of aliasing in just about any situation. In general, it should only be used when the need for speed is paramount.
- The **box filter** is of slightly better quality, but still tends to produce a lot of artifacts. When scaling an image up in size, it will result in a boxy look.
- The **triangle filter** considers slightly more area when resampling, and is used a fair amount as a quick method for resizing an image for viewing purposes.
- The **Mitchell filter** is a good balance between sharpness and ringing, and is often the best choice when scaling an image to a larger resolution.
- The **Gaussian filter** is a common filter that is virtually free of aliasing or ringing artifacts, but tends to introduce noticeable softening in the image.
- The **sinc filter** does a very good job of keeping small details without introducing much aliasing. It is probably the best filter to use when scaling an image from a larger resolution to a smaller.

CHAPTER FOUR

Basic Image Compositing

Now that we've taken a look at a number of different methods to modify a particular sequence of images, we can start to discuss the manipulated combination of *multiple* sequences. This process, combining two or more image sequences, is the true heart of digital compositing. It requires the use of special operators that are capable of dealing with more than a single input sequence, and so we will refer to these operators as "multisource operators." As was the case with our discussion of basic image manipulation tools, we will only take an in-depth look at a few of the most important image-combination tools. There are numerous others, for many of which we provide a bit more information in Appendix A.

We'll start this discussion with one of the most common multisource operators. It is also one of the simplest, and we'll refer to it as simply an Add. If you recall, in the last chapter we discussed a single-input Add operator that added a constant value to each pixel in an image. Although we'll be using the same name for the multisource operation we're about to discuss, the difference should be obvious. By using the same names, we're following typical nomenclature used in the industry; because of the obvious difference in the situations in which the two operators will be used, there should be little room for confusion. A number of the basic mathematical single-source operators, such as Subtract or Multiply, have dual-input equivalents.

Adding two images together involves, not surprisingly, the addition of each pixel in the first image to its corresponding pixel in the second image. Plates 18a and 18b show the two source images we will be using to illustrate some of our multi-input operators. Combining these two images using an Add produces the image shown in Plate 19. As you can see, the result is similar to a photographic double exposure.

While this effect is certainly useful in a variety of situations, it does not give us the impression that any sort of layering has occurred. There is no sense that certain portions of one image are actually occluding the second image. To accomplish this, we need to introduce the concept of a **matte**.

THE MATTE IMAGE

Combining different image sequences needs to be a process with as much control as possible. We need to be able to limit which portions of the various images will be used, and which will not. We need to be able to control the transparency of the various layers so that they don't completely obscure everything that they are covering. And we need to have a method for defining and controlling these attributes that is intuitive and consistent with the rest of the image processing we will be performing. This is what the matte image gives us.

First of all, understand that a matte image is no different from any other image, in terms of the data used to represent it. It can generally be treated and manipulated in exactly the same fashion as other images, but it is considered to have a different *purpose* than those images. Instead of providing a visual representation of a scene, it is more like a utility image, used to control individual compositing operations.

Mattes are used during compositing when we only wish a portion of a certain image to be included in the result. You may also hear the term "mask" used when referring to mattes, and it is not uncommon to find the two terms used interchangeably. For sanity's sake, we will try to limit the use of the word "mask" to refer to a general image that is used to control or limit certain parameters in an operation, such as a color correction.

To complicate things even further, both "mask" and "matte" may also be used as either nouns or verbs. The terms can refer to the image used in the process of protecting or excluding a section of an image, or they may refer to the process itself. Consequently, we may "matte out" a section of the foreground so that the background is revealed, or we may "mask off" the bottom third of an image when we color correct it so that the correction doesn't affect that area.

Mattes are generally considered to be single-channel, grayscale images. There is no need for three separate channels, as there is when specifying color, since the transparency for any given pixel can be described by a single numerical value in the range of 0 to 1. Many systems and file formats support single-channel images, whereas others will simply place a copy of the same information into all three channels of an RGB image. While this method is redundant (and wastes disk space), it does sometimes provide a simpler model for both the user and the programmer.[1] Ideally, the compositing system will store a single-channel matte

[1] Matte images tend to contain large areas of identical pixels, usually black or white, and as such will compress dramatically using one of the better lossless compression schemes. Consequently, there

image as efficiently as possible, yet still allow the compositor to treat it as if it were a normal RGB image when necessary.

Depending on the software package you are using and the file format you have chosen, a matte can also be bundled along with a three-channel color image as a discrete fourth channel. When the matte image is part of a four-channel image, it is known as the **matte channel** or the **alpha channel**.[2] In the next section we will discuss four-channel images in greater detail, but for the time being we will consider the case in which our matte is a separately stored image.

Let's look at a very simple example of how a matte image is used, given our original two images (Plates 18a and 18b) and a third matte image shown in Plate 20. We will use this matte channel to isolate or extract a foreground piece of Plate 18a and will then place it over the background of Plate 18b. The resulting image is shown in Plate 21.

This example is simply to give you an intuitive idea of how a matte channel might typically be used. As you can see, areas that are white (have a pixel value of 1.0) in the matte channel are used to specify that the corresponding area of the foreground image is kept at full opacity. This is said to be the "solid" area of the matte. Conversely, the black areas of the matte are used to specify that the corresponding pixels in the foreground image will be transparent, or effectively removed, when it is placed over the background. Intermediate gray levels of the matte channel provide a continuum of transparency, with brighter (higher-valued) pixels specifying more opaque areas and darker pixels specifying more transparent areas. The same behavior can be seen if we look back at the image from *James and the Giant Peach* that we discussed in Chapter 1. Plate 2e shows the matte that was used to extract the peach from Plate 2a in order to place it into the scene shown in Plate 1.

In the preceding examples, we have only shown matte images as being distinct entities, completely separate from normal color (RGB) images. Any operation that is used to combine two images would need to reference the matte image as a third image in order to control varying levels of transparency. This need not always be the case if we are working with a system that supports four-channel images.

THE INTEGRATED MATTE CHANNEL

As stated earlier, very often an image will be stored with not only the three basic color channels, but also a fourth channel, the matte channel. But there is more to

---

is often less concern about the amount of disk space that is taken up by a matte image.

[2] The concept of a matte image that could be integrated as a fourth channel to a normal color image was developed and formalized by Ed Catmull and Alvy Ray Smith at New York Institute of Technology in the late 1970s.

the process than simply attaching that matte channel to an image. Usually, when a fourth channel is added to an image, the color channels are modified as well, to include some of the information that comes from the matte channel. In fact, the standard definition of a four-channel image assumes that the red, green, and blue channels have already been multiplied by the integrated matte channel. Such an image is referred to as a **premultiplied image,**[3] and it is most commonly produced as the output of a 3D rendering and animation package. (Occasionally there are situations in which you may have a three-channel image that has already been multiplied by an external matte. This too could be referred to as a premultiplied image, but this scenario is much less common.)

Plate 22 shows an image whose color channels are the result of multiplying Plate 18a by Plate 20 to produce a premultiplied image. As you can see, everywhere that the matte was black (having a digital value of 0), the color channels have become black, or 0. Wherever the matte was a solid white, the color channels are unmodified. Less obviously, in areas where the matte had some intermediate gray value, the corresponding pixels in the RGB image have been darkened by a proportional amount.

Premultiplied images can be a great source of confusion, primarily because in certain situations this multiplication step is done automatically but in other situations (or using different software packages), the process must be dealt with explicitly by the user.

Whether or not an image is premultiplied can significantly affect the compositing process and the resulting imagery. Using premultiplied images with tools that aren't expecting them can be disastrous, as can the reverse. This is particularly true when dealing with certain layering operators such as the Over tool, which we will discuss in a moment. Toward the end of this chapter we will spend even more time discussing these concepts.

Before we take a look at some additional multisource operators, it will be useful to stop and describe a few notations that we will be using in conjunction with our definitions.

MULTISOURCE OPERATORS

Many of the multisource operators that we will be discussing will support images that carry an auxiliary alpha channel. Unless otherwise specified, assume that the operation discussed is independent of whether the images contain integrated

---

[3] The premultiplied image (as well as a number of the multisource operators that can take advantage of a premultiplied image) was first described in a classic paper that Tom Porter and Tom Duff presented at the 1984 **SIGGRAPH** conference.

matte channels. For those operators where it is applicable, we will first discuss the process as it relates to images without any integrated matte channel and then will look at the same operator's behavior with four-channel images. When we do need to make a distinction about separate image channels, we will use the following notation. For any image A,

A_{rgb} = The RGB channels only.
A_a = The alpha, or matte channel, only.
A = All the channels (be there three or more) of the image.

Finally, just so that we can put some of these descriptions into equation form, we'll define "O" to represent our output image, just as we did in Chapter 3. We will also occasionally use "M" whenever we need to denote an image that is used exclusively as a matte channel. Such an image should generally be thought of as a single-channel (grayscale) image.

Using these notations, the Add multisource operator that we originally discussed could be simply represented as

$$O = A + B$$

A and B are our two source images and O is the resulting image. Since the Add operator behaves the same with either three- or four-channel images, there is no need to mention any specific channels in this particular equation, and it should be obvious that in actuality,

$$O_{rgb} = A_{rgb} + B_{rgb} \quad \text{and} \quad O_a = A_a + B_a$$

Now that we have a common language that we can use to discuss some of these concepts, we can finally begin to talk about some of the more powerful compositing operators. Most of these are not just two-source operators such as the Add we just saw, but instead can accept several inputs, including matte inputs that may be used to control which portion of one image is combined with another image. By far the most common operator for selectively combining two images is the Over tool.

Over

Since the Over operator is such a crucial, often-used tool, we will examine it in great (some may say excruciating) detail. Even if you feel you are very familiar with using Over for compositing, it is worth understanding the exact algorithm that is used, since a number of problems can be diagnosed when you are armed with this knowledge.

The Over operator takes two images and, using a third image as a controlling matte, lays a portion of the first image on top of the second. It was the operator we used to illustrate the matte channel early in this chapter, producing the image in Plate 21. Intuitively, people understand compositing with the Over tool as if the matte were a cookie cutter that removes all excess information from the foreground image. Once this cutout is created, the result is then pasted on top of the background.

Here's what really happens, mathematically, when we place image A (the foreground) over image B (the background), using image M as the matte for image A.

$$O = (A \times M) + [(1 - M) \times B]$$

Let's break this down into the specific steps as they occur. First, the foreground image is multiplied by the matte image ($A \times M$). In our example, this was already shown to produce the intermediate image shown in Plate 22. Again, everything outside of the matte has become black, and the portion of the image that is located within the solid, or white, area of the matte remains unchanged.

The second step is to take the inverted matte image (Plate 23a) and multiply that with the background image. This multiplication produces a new intermediate image with a black hole where the foreground will go (see Plate 23b). To complete the process, these two intermediate images are then added together, creating the final output, which we already saw in Plate 21.

This example underscores an important point about most image-combination tools. They are often just a group of even more simple operations applied sequentially. If you find yourself trapped on a desert island with a compositing system that has only a limited number of tools, you will usually be able to recreate a large number of more complex tools as necessary.

For those of you who are accustomed to working with images that already have an integrated matte channel (such as those rendered by a 3D animation package), the equation describing the Over operation may seem to contain an additional step. In fact, the first stage of our Over equation (described by ($A \times M$)) is designed to produce a normal premultiplied image that would behave identically to an image produced by 3D rendering software.

If we look again at the Over operation, this time simplifying for the case of an image with an integrated matte channel, the equation becomes

$$O_{rgb} = A_{rgb} + [(1 - A_a) \times B_{rgb}]$$

Note that the color channels of the output image are independent of the background image's matte. However, the output image's matte channel is composed as follows:

$$O_a = A_a + (1 - A_a) \times B_a$$

We could also write the simplified equation for all four channels as

$$O_{rgba} = A_{rgba} + [(1 - A_a) \times B_{rgba}]$$

As mentioned earlier, be sure you understand exactly what type of scenario your compositing operator is expecting in terms of premultiplied images, so that you can avoid the problems we'll be discussing at the end of this chapter. Let's look now at a few more common image-combination tools that you will probably find useful.

Mix

A mix is the weighted, normalized addition of two images. In other words, the two images are averaged together, often with one of the images contributing a larger percentage to the output. Plate 24 shows the result of mixing 75% of image A with 25% of image B.

The equation for such a mix, where "MV" refers to the mix value (the percentage of the first image that we will be using), is as follows:

$$O = (MV \times A) + [(1 - MV) \times B]$$

This operation is usually known as an "additive mix," since the weighted averages are added together.

To "dissolve" from one image to another over a given period of time, one merely animates the mix value so that it initially displays 100% of image A and then eventually displays 100% of image B.

Subtract

The Subtract operator causes every pixel in image A to be subtracted from its corresponding pixel in image B. Quite simply,

$$O = A - B$$

Note that Subtract is not a symmetrical operator. The order in which the two images are specified is important to the result. Be aware of which multisource operators are symmetrical and which are not, since it will determine whether or not you need to be concerned about the order in which you combine images. Thus, (B + A) equals (A + B), but (A − B) does *not* equal (B − A).

Most implementations of the Subtract operator will allow you to choose whether to clip all values that go below zero, or to take the absolute value of the result, in which negative numbers are converted to positive. The absolute value method is particularly useful for difference matting, which we'll discuss in Chapter 5. A subtraction that returns the absolute value of the result becomes a symmetrical

operation, so you no longer need to worry about the order of the two image sequences that you specify.

In

There are certain multisource operations that require only a matte image for the second input. (The matte image can either be a single-channel image or the alpha channel of a four-channel image.) The In operation is one of these. As you can see from the equation, it pays no attention to the color channels in the second image, even if it is a four-channel image:

$$O = A \times B_a$$

When we describe the use of the In operator, we usually say we are placing image A *in* image B; the result is an image that consists of only the areas in image A that overlapped with the matte of image B. An example of this is shown in Figure 4.1. On the left, we see image A (the dark gray circle) placed in a certain position relative to image B (the light gray square). Both images are assumed to have an integrated solid matte channel that corrresponds to the shape you see. "A in B" is shown on the right.

Out

The Out operation is the inverse of the In operation—it results in a new image that consists only of pixels from image A that did not overlap the matte of image B. We say that A was "held out" by B.

$$O = A \times (1 - B_a)$$

This result is illustrated in Figure 4.2.

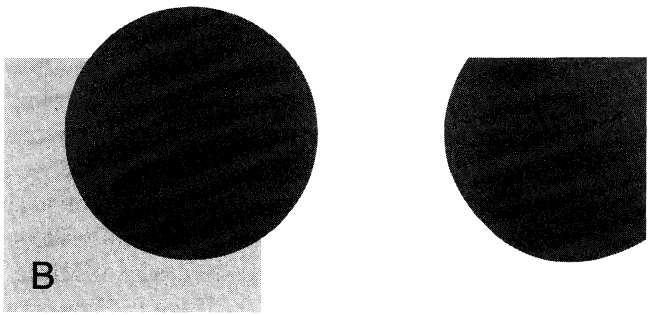


Figure 4.1 *A in B.*

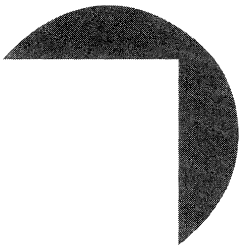


Figure 4.2 *A held out by B.*

Atop

As mentioned, many image-combination tools can be used in tandem to produce new tools. Most of these combinations are given different names by different software vendors, so we will not attempt to list them. But to give an example, we show a fairly common one, usually referred to as an Atop operator. It places image A over image B, but only in the area where image B has a matte. We say we are placing A "atop" B.

$$O = (A \text{ in } B) \text{ over } B$$

An example of this operation is shown in Figure 4.3.

Although most of these image-combination operators are really just very simple mathematical calculations that work with images and mattes, more complex operators certainly exist. **Morphing**, for instance, combines the animated warping of two images over time with a controlled dissolve between the two sequences.

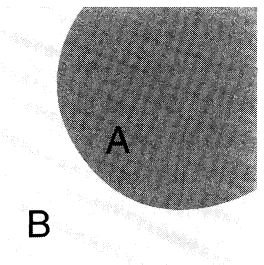


Figure 4.3 *A atop B.*

MASKS

There are times when we wish to limit the extent of a certain operator's effect. A particularly useful method for doing this involves the use of a separate matte as a control image. Whenever a matte is used in this fashion, it is usually referred to as a **mask**. In essence, the use of masking with a single-event operator implies a multisource scenario, with the original sequence as the first source and the mask as the second. Just about any operator should be something that can be controlled by an external mask, and so we'll look at a fairly simple example.

Consider the matte image shown in Plate 25a. Let's say we wish to decrease the brightness of our parrot image, but we want to limit that attenuation as a function of this mask. The result of this is shown in Plate 25b, where we used a brightness value of 0. If we hadn't used a matte to limit this operator, the entire image would have gone to black, but instead the darkening was only applied where the mask allowed it to be. In the areas where the mask was completely solid, the resulting pixels remained unchanged. Where the mask was transparent, or black, the full attenuation could be applied. In any areas where the mask had intermediate gray values, the brightness was applied by a proportionally smaller amount.

If your system does not support masking to control operations, you can still accomplish the same effect manually, using some of the image-combination tools that we have already discussed. You would first apply the effect to the entire frame and then use the mask to isolate the proper area and combine it with the original, unaffected image.

COMPOSITING WITH PREMULTIPLIED IMAGES

It is extremely important to understand exactly what type of image (premultiplied or not) your system's compositing operators expect. Some systems assume that they will always be dealing with premultiplied images; others may require the image and matte to be brought in separately so that they can be recombined during the operation in question. If you do not use the proper type of image, you run the risk of introducing a wide variety of unacceptable artifacts. Let's look at some different scenarios. We'll use an example based on the Over operator, but the same problems can show up with other tools as well. The matte in our example is intentionally very soft-edged, since this is the most problematic situation.

Consider Plate 26a, the image that we will use for our foreground. It features a dog standing in front of a gray wall. Plate 26b is a soft-edged matte to extract the dog from the scene. Plate 27 shows the background image that we will be using. It features the same wall (shot from an identical camera position), only with a cat standing in front of the wall.

If we premultiply our foreground image (Plate 26a) by its matte (26b), the result is shown in Plate 26c. Now, if this result is placed over the background in a system that assumes that the images you are providing are all premultiplied, the proper result is obtained, as shown in Plate 28. The soft edges blend well with the background and there are no noticeable artifacts.

However, if we combine Plate 26a with Plate 27 *without* premultiplying the color channels, we have an image that will not be handled properly if our Over operator is expecting a premultiplied image. The result of placing such an image over the background is shown in Plate 29. Note that the foreground element, in areas where its matte is supposed to be 0, is still contributing to the result. If you were to check out the math of what is happening, you'd see that in those areas of the result image, it is exactly as if we had simply added the two images together.

Equally damaging problems can occur in the other direction, that is, if we feed premultiplied images to a system that is expecting to combine the image with the matte itself. Such a system will automatically multiply the image by its matte channel, even though this has already been done in a normal premultiplied image. In this situation, we have effectively multiplied the image by its matte channel *twice*, thereby darkening all areas of soft-edged matte. The result of such a mistake is seen as a dark halo around the foreground, as shown in Plate 30.

Although the preceding examples resulted in some very obvious problems, an improper image/matte relationship can be the cause of some very subtle artifacts as well. Such artifacts can show up even if we have properly synchronized our image types with what the operators are expecting. One of the more common places these will occur is when we are trying to color correct images that have already been premultiplied.

Color-Correcting and Combining Premultiplied Images

Whenever we premultiply an image by a matte, there is a very specific brightness relationship between the pixels in the color channels and the pixels in the matte. Systems that assume you are working with premultiplied images will rely on this image/matte relationship; consequently, the brightness of any color channel can no longer be modified without taking the alpha channel into account (and vice versa). Thus, any time you apply a color correction to a premultiplied image, you run the risk of producing a new image whose matte/image relationship is no longer "legal." If you look at the math that is used when we premultiply an image by a matte, it should be obvious that the brightness of any given red, green, or blue channel in such an image can never exceed the value of the alpha channel. This can sometimes be used as an indicator that there has been some kind of operation performed on the image after the premultiply occurred. The result of compositing with this image will vary, depending on the extent of the change

made to the different channels, but generally it will manifest itself as a slight brightening or darkening of the composite in areas where the foreground matte was semitransparent. More extreme changes to the color or matte channels will obviously result in more extreme artifacts.

A particularly common problem will occur when some color correction has been applied to a foreground premultiplied image that causes the blacks in the scene to be elevated above a value of 0. Now, even though the matte channel may still specify that the surrounding field is black, the RGB channels may have some small, often visually undetectable, value. The problem will show up when one attempts to layer this element over a background. The resultant image will show a noticeable brightening of the background in the area outside the foreground's matte. As one becomes a more experienced compositing artist, artifacts like this become the clues that let one track down exactly which layer in a composite is causing a problem. In this case, the clue is the fact that the background gets brighter. As you can imagine, it may be very difficult to determine if this problem exists, since at first glance an improperly modified image may appear perfectly correct.

Because inevitably you will find yourself with a need to color correct a premultiplied image, there is usually a tool on most systems to temporarily undo the premultiplication, at least in the critical areas of the image. We will refer to this tool by the rather unwieldy name of "Unpremultiply." Essentially, the tool redivides the image by its own matte channel, which has the effect (except in areas where the matte is solid black, or 0) of boosting the areas that were attenuated by a partially transparent matte back to approximately their original values. An example of this is shown in Plate 31, which is the premultiplied image from Plate 26c after redividing by its matte. (Areas where the matte was equal to 0 have been set to be pure white.) Although this is obviously not a completely perfect recreation of the original image, it restores enough information in the edges so that appropriate color corrections can be applied.

If your system does not have an explicit tool to perform this operation, you can try using some other tool to simulate it. For example, if there is a simple expression language available, you can modify the image so that

Red = Red/Matte

Green = Green/Matte

Blue = Blue/Matte

Once the image has been unpremultiplied, any necessary color corrections can be applied without fear of corrupting any semitransparent areas. The result should

then be remultiplied by the original matte channel, which will restore the element to one whose image-to-matte relationship is correct.

For slight color corrections, or when using images that have very hard-edged mattes, you may get perfectly acceptable results without going through this process. As usual, use your best judgement, and if it *looks* correct, then by definition it *is* correct. But if there is any doubt in your mind, or if you are having problematic edges, you may want to take a moment and compare the results of performing your color correction before and after the image is multiplied by the matte.

By default, all the major 3D packages will render elements that have been premultiplied by their matte channel. As we've seen, this is not always the ideal scenario and in many situations may actually be counterproductive. Assuming that the 3D element is perfectly ready for integration into the scene, having it delivered already premultiplied by its matte channel will be fine. But as often as not, this conceit is not really the case. You may want to determine if there is some way of overriding this premultiplication behavior in the 3D package, so that you can have unpremultiplied images available for your compositing work. This ability will be particularly important if you are in a situation in which you will be performing a good deal of color correction on the 3D element.

Luminosity

When working with four-channel premultiplied images and using the Over operator, there is an additional effect that can sometimes be useful. It involves selectively manipulating the alpha channel of the image to intentionally modify the image-to-matte relationship. This operator is often referred to as a "Luminosity" tool.[4] By decreasing the value of the alpha channel before placing an object over a background, the object will appear to become brighter in the result. In addition, the background will become more visible through the foreground element.

An example of this effect is shown in Plate 32. In this case, the matte channel for the foreground image was multiplied by 0.6 before being placed over the background. In fact, as the matte channel is decreased toward 0, the result becomes more and more like an Add operation instead of an Over. This can be proven by examining the equation for Over again. If the matte of the foreground object in a premultiplied image is set to 0, the Over operator will produce the same results as an Add.

---

[4] The other common name for this on many systems is the "Opacity" operator, even though this name is less descriptive of the visual result this tool produces.

CHAPTER FIVE

Matte Creation and Manipulation

In the last chapter, we introduced the concept of a matte image and gave some examples to show what an important role these images play in the digital compositing process. But unlike most other images that you will be dealing with when compositing elements together, matte images are not scanned into the computer from some outside source. Rather, they are almost always generated within the computer, as a part of the compositing process. There are many different methods used to generate mattes for compositing, just as there are many ways that a matte image can be used. The process of generating a matte, particularly when automated, is referred to by a variety of terms, including "matte extraction," **pulling a matte**, and **keying**. Keying is actually a broader term, in that it is often used not only to describe the process of creating a matte but also to include the process of combining one element with another. Thus, you might "key" a blue-screen element over a background. This chapter will look at methods for manipulating as well as creating mattes.

An example of a situation that would require a very simple matte would be a split-screen composite. This is a situation in which (for example) the left half of the image comes from one source and the right half of the image from another. The matte would be a fairly simple shape that defines the boundaries of the split—often just a straight line that separates one plate from another. Such a matte is easily created using simple paint or shape-generation software. More often, however, we need to place an object whose outline is much more complicated into a scene. We need a matte that accurately represents the boundaries or outline of the object and that fully encloses all solid areas of the object's interior.

In the case in which this object does not move, it is still conceivable that such a **static matte** can be hand-painted. But even with an unmoving subject it can be

difficult to accurately paint something that properly captures the desired edge qualities (such as transparency and softness) of a given object. Instead, one will tend to use certain software algorithms, discussed later in this chapter, to help isolate an image from its background.

When compositing sequences of images, situations involving static mattes are fairly rare. Far more often we find the need to create a matte for an object that is moving within or through the frame. We require the use of a moving matte, or **traveling matte**. There are two approaches we can take to generate these mattes. The first would continue the methods we've talked about already, that is, hand-drawing a matte for the object in question over every frame of our sequence. This process, known as **rotoscoping,**[1] is still done, but only after all other options have been exhausted, since hand-drawing a matte for every frame of a sequence is time-consuming and error prone. Common problems produced by a series of hand-drawn mattes include edges that slide, jitter, or crawl.

One slightly less brute-force method of manually generating a traveling matte involves the use of splines to rotoscope a basic outlined shape for an object. The rotoscope artist specifies certain key shapes, and the software smoothly interpolates any in-between frames. Unfortunately, objects that move or change shape a great deal may end up needing a key shape defined for every frame anyway! In any event, this technique still does not do a very good job of isolating extremely fine detail (such as hair or fur) and has less control over areas of partial transparency than is usually desirable.

Ultimately, we will need to rely on more *procedural* techniques—semiautomated processes in which some initial parameters are determined that are capable of extracting a matte, and then the software is allowed to apply these same parameters over a sequence of images.

PROCEDURAL MATTE EXTRACTION

We will briefly touch on a number of different methods that may be employed to procedurally extract a traveling matte from a sequence of images. The efficacy of these methods is highly dependent on the image in question, and often you will find it necessary to try a number of different tools before obtaining a result that you are happy with.

The best methods of extracting a matte for an object rely on the fact that we (usually) know in advance that we are planning to place the object in question into a different scene. Consequently, we can choose to photograph this subject in

[1] The **rotoscope** was a device patented in 1917 by Max Fleischer to aid in cel animation. The term is now used generically to refer to the process of creating imagery or mattes by hand on a frame by frame basis.

a manner that greatly simplifies its matte extraction. Typically, this involves the use of a special background that is placed behind the subject we wish to isolate. This background (or **backing**) should be a uniform color, ideally a color that is not significantly present in the subject itself. The exact choice of what color to use will be determined by the extraction technique that will be employed, but by far the most common color that is employed is blue. Thus, the process of shooting in front of any colored background is sometimes generically known as **bluescreen photography**. Not all of the extraction tools we will discuss rely on a uniform backing, however, and many of them can be used to help extract mattes for objects that were not shot with any special attention or intentions. We will look at these tools first.

Keying Based on Luminance

One of the most common methods used to extract a matte for a given item is based on manipulating the luminance values in a scene. This is usually known as **luma-keying** and involves the application of some basic image processing operators to choose the luminance values that one wants to include or exclude from the matte.

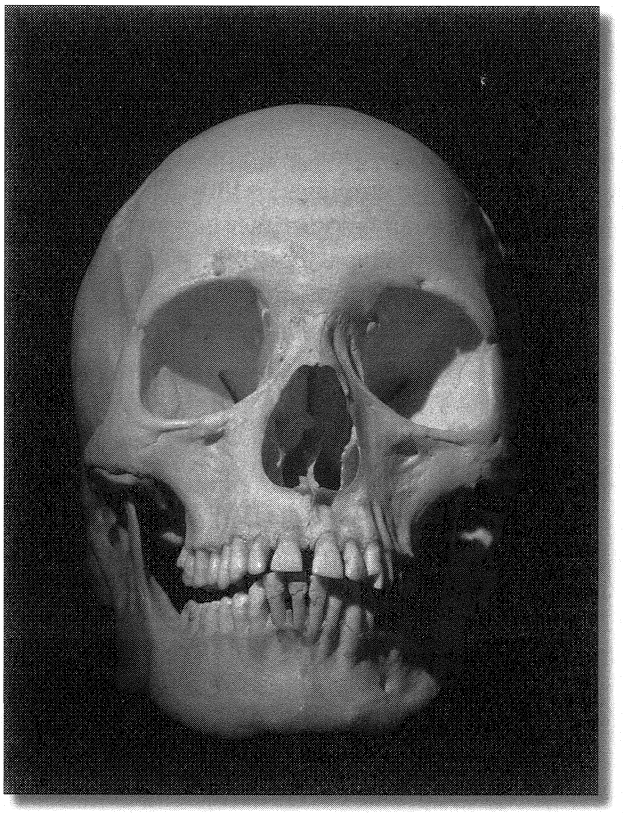
The technique is generally most useful when the feature you wish to extract from the scene is significantly brighter or darker than the background from which you wish to separate it. The ideal case would be a bright object shot over a black background, but you'll find that there are often situations in which the brightness difference between foreground and background is enough to extract a decent matte.

Consider Figure 5.1a. A bit of manipulation can quickly produce the image shown in Figure 5.1b. As you can see, we now have something that could easily be used as a crude matte, and additional digital manipulations could continue to refine the result. This situation would also benefit greatly from the use of an interior **garbage matte**, which we will discuss in a moment.

Keying Based on Chrominance

Chroma-keying is a bit more sophisticated, but it still makes use of some of the basic tools that we discussed in Chapter 3. In the case of a simple chroma-keyer, the process is to pick a certain range of colors or hues and to define only the pixels that fall within this range as being part of the background. Generally, a good chroma-keyer will have additional tools that let you specify not only a specific hue range, but also a particular saturation and luminance range to further control the matte.

A good chroma-key tool can be used to pull a matte from a bluescreen or a greenscreen, but it should not be confused with some of the more sophisticated color difference methods that we'll discuss in a moment.



(a)

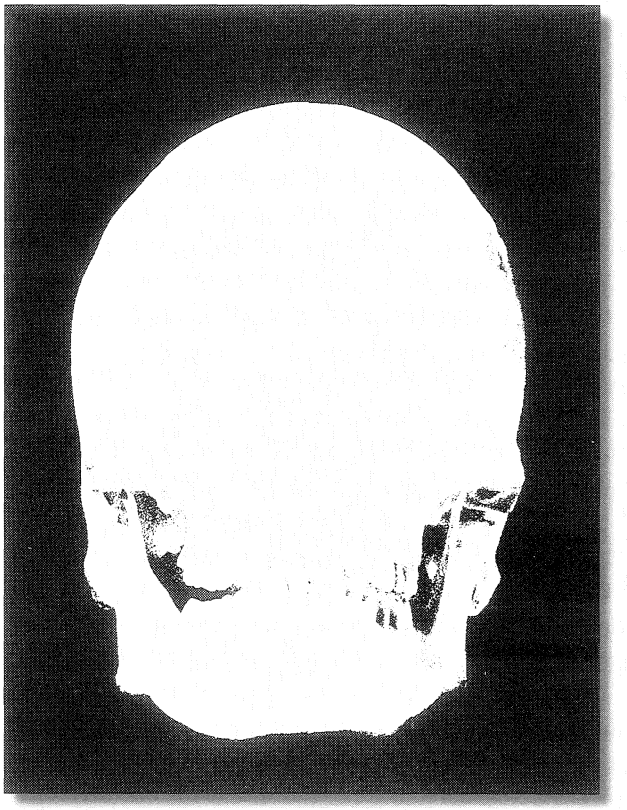
Figure 5.1 *Luma-keying. (a) Skull image on a dark background.*

Incidentally, the terms "chroma-key" and "luma-key" originally came from a process that used specialized hardware for real-time keying on a video signal. A standard video signal natively represents image information in terms of both luminance and chrominance values, and this keying hardware would use modified luma or chroma components to key elements together.

Difference Matting

In theory, if you have an image that contains the subject you wish to isolate, and another identically framed image that does *not* contain the subject, subtracting one from the other will produce an image that consists of information only where the subject was present. This process is known as **difference matting**.

In practice, slight lighting discrepancies, shadows, and grain make the difference between the two images unpredictable and the results less than perfect. Difference matting is thus usually not considered a complete solution, but rather



(b)

Figure 5.1 *(b) Matte extracted by luminance-based keying.*

as a very useful first-pass method that can then be cleaned up using other methods. Since a difference matte requires two separate images with the same lighting and camera setups, it is a limited tool, and using it to produce a traveling matte from a sequence of images would require either an unmoving camera or some method of perfectly synchronizing the camera movements between the two plates.

In spite of these limitations, difference matting is an extremely useful tool in certain situations because it can be used to extract a matte from just about any background. Consider the example image shown in Plate 26a as compared to Plate 33a. As you can see, the dog is present in the first image but not the second, yet everything else is identical. If we subtract Plate 33a (which is known as a "clean plate") from 26a, the result is shown in Plate 33b. While this is by no means a perfect matte, it is not a bad starting point and can probably be digitally manipulated to produce something that is more acceptable.

The Color Difference Method

By far the most popular (and effective) method for procedurally generating traveling mattes is known as the **color difference method**. This method was actually developed and patented in the 1950s by Petro Vlahos as an optical matte extraction process, and was first used in a major theatrical production for *Ben Hur* in 1959. The steps used, which involved selectively combining the different color records (or channels, to use digital terminology) of a piece of film to produce the resulting image, can be easily converted into equivalent digital formulas. We will describe the basic steps here so that you can get a better idea of exactly how the technique is applied, but understand that this will be a very simplified version of a more complex process.

The first thing that we should mention is that the color difference method is not just a matte-extraction tool. It is actually a combination of steps that includes matte extraction, color correction, and image combination. Although the matte-extraction step *will* produce a matte that could be used in other situations, the basic definition of the color difference method must include all of the steps to be complete, and so we will describe the entire process in this section.

Assume we are working with an image shot on a blue background. The first step in the color difference method involves the creation of a new image in which this blue backing is suppressed to black. This is done by selectively substituting the green channel for the blue channel in every pixel in which the existing blue component has a greater intensity than the green component. In other words, for each pixel,

If Blue > Green
then New Blue = Green
else New Blue = Blue.

An example of this process is shown in the color section: Plate 34 is our original bluescreen element, and Plate 35 is the result after applying the operation. Since the green channel should have a value of 0 in areas of pure blue backing, the primary result of this substitution is an image in which the blue background has gone to black. Additionally, anything in the foreground that has a heavy blue component (i.e., blue or magenta areas) will be modified. Although this effect is a problem if there is any blue coloration in the foreground that we wish to keep, it is also a benefit in that it will neutralize any blue **spill**—blue lighting from the backing that has inadvertently fallen onto the foreground. This step, known as "spill suppression," is usually applied with a few more parameters than we have just described. Additional thresholds are added so that there is control over exactly how much blue is removed and to compensate for any change in brightness that the channel substitutions might cause. The use of spill suppression to better integrate an object into a scene will be discussed again in Chapter 14.

The second step in the color difference method involves the creation of the matte itself. This is simply a matter of subtracting the maximum of the red or the green component from the blue component. It is this difference between the blue channel and the other channels that gives the technique its name. To restate the step mathematically:

$$\text{Matte} = \text{Blue} - \text{Maximum}(\text{Green}, \text{Red})$$

This operation will actually produce what we would normally think of as an inverted matte—the foreground area is black and the background area is white. This result is shown in Plate 36.

We now simply multiply this inverted matte with our intended background—a step that results in a background with a black hole in the shape of our foreground element. The final step would be to add our modified foreground to this intermediate background, producing an integrated result. As you can see, the final steps of this process are identical to the way that the Over operator works.

Of course, the steps described are merely a basic template for the process. If you were to apply these *exact* steps to any given bluescreen image, the chances are high that you would not be terribly happy with the result. In our example images, you'll note that the background of Plate 35 did not become perfectly black, but rather a dark gray. The matte created in Plate 36 is also less than perfect—the areas that should theoretically be pure white are only about 85% white. In practice you would want to have additional control over the various steps as they are being applied in order to deal with such issues. You would adjust the relationships between the channels, thresholding certain values and multiplying others, until a visually acceptable result is obtained for the final composite. This type of control is exactly what the Ultimatte system, described in the next section, allows.

The color difference method always involves shooting the foreground object we wish to isolate in front of a uniform-colored backdrop. What's more, this method works best when the backing is a pure primary (red, green, or blue) color. Its effectiveness is greatly reduced if the backing is less pure. Chroma-keying tools have an advantage in this respect, in that they are not biased toward any particular hue. If you are confronted with an image that has an off-color background, these tools may be a better choice. Chapter 13 goes into much more detail about choosing a backing color for optimal matte extraction.

Specialized Keying Software

Above and beyond the standard compositing tools that are used to extract a matte from a scene are custom software packages that are designed explicitly for this purpose. These custom packages can almost always produce far better results, due to the specialized algorithms they employ.

The value of these custom packages lies not only in their excellent matte-extraction tools, but also in the methods they use to integrate an element into a scene. Color correction, spill suppression, and edge treatments can all be dealt with when using these packages. Of the packages in question, by far the most common are the tools provided by the Ultimatte Corporation. Ultimatte was actually founded by Petro Vlahos, the creator of the color difference method, and the Ultimatte software and hardware that it provides are extremely evolved versions of that process.

Yet another common tool for matte extraction and bluescreen compositing is the software known as Primatte. This software is essentially a very powerful chroma-keyer, allowing the user to choose a large number of different chroma values at the same time from which to key, and also allowing the user to deal with spill areas explicitly. Both of these tools are available as adjuncts to most of the common compositing packages or can be purchased as stand-alone software.

Many complete compositing packages will also provide their own custom matte-extraction tools, most of them based on some of the techniques that we have already discussed. Depending on the software, these tools can range in quality from barely acceptable to extremely powerful.

Ultimately, no single tool or technique is necessarily "better" than another. Each can be useful in a particular situation, and the choice of which tool to use will often be determined by a number of tests. Some packages or techniques work better for some images and worse for others, and it can be very difficult to predict this in advance.

MATTING TECHNIQUES

We first need to correct the common misconception among novice compositors that the process of pulling a matte off a bluescreen should be a simple, one-step process. In the real world, especially for high-end work, this is true so rarely that it is definitely the exception rather than the rule. Even the best tools can have problems with certain images, and typical production situations often deliver plates to the compositor that are less than perfect in terms of evenness, graininess, and freedom from objects that are not intended to be seen in the final composite.

A typical matte extraction usually involves *several* steps, a great deal of interactive testing, and often requires that specific tools be used for specific problem areas. You may also find that whatever tools you use to extract a matte will need to be modified at different points in time. Unavoidable changes to the backing's illumination (such as an inconsistent light source) may require that the settings be animated throughout the duration of the sequence on which you are working.

Garbage Mattes

Very often one will use **garbage mattes** to help simplify the process of creating a clean matte. A garbage matte is a loose-fitting shape designed to address specific

problem areas, and will almost always be used in conjunction with a more exact matte. Garbage mattes should be quick and easy to create; usually they will have very little to do with the edges of the object in question. By definition, their purpose is to remove any obvious "garbage," or extraneous information. This garbage will usually be something exterior to the chosen foreground subject, such as a piece of rigging that is hanging into frame or a defect in the bluescreen itself. The matte shown in Plate 2e was actually created with the aid of a garbage matte similar to the one shown in Figure 5.2. This matte, in conjunction with some more standard matte-pulling software, was used to quickly remove the support pole that is visible in the original bluescreen plate (Plate 2a).

Garbage mattes can also be used to explicitly *include* a portion of an image, such as areas that would otherwise be problematic due to their color similarity with the backing. Thus, it is not uncommon to create both an inner and an outer garbage matte for any given image. This underscores a particularly salient point: The most important thing that any automated matte-extraction technique will give you is a high-quality edge. Everything that is *not* an edge may very well be something that can be addressed with far less effort using other techniques.

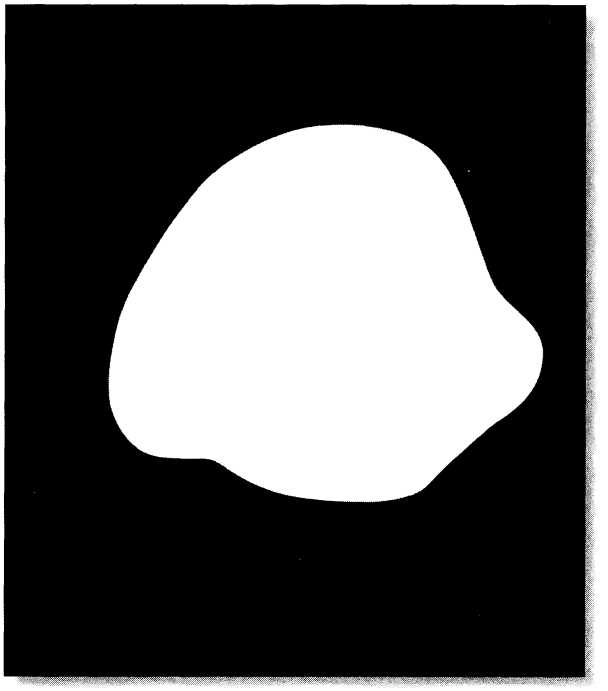


Figure 5.2 *Garbage matte.*

Edge Mattes

Since it is usually the edges that we spend the most time with when working on a matte, it is extremely useful to have a way of limiting certain operations so that they affect only these edges. One of the best ways to limit these operations is by using yet another matte, usually referred to as an **edge matte**. An example of an edge matte for the peach from Plate 2a is shown in Figure 5.3. As you can see, this is a matte that is solid only along the edges of an element and is transparent elsewhere. It can be created either directly from an image or, probably more commonly, from another matte. For instance, if we pull a very high-contrast matte for our foreground element, we can then use an edge-detection algorithm to define only the edge. In this situation, the original matte we pull should be as hard-edged as possible, since edge-detection algorithms will work more effectively with this scenario. Then, depending on the edge-detection tools you have available, you may need to do some additional processing to create a thicker edge.

Edge mattes are worthwhile for a number of reasons. First of all, when coupled with a few simple operations they can be used to create a very accurate inner and outer garbage matte. They can also be very useful as a mask for controlling additional effects that are used to help with the integration. For instance, one could use an edge matte to more easily control the softness or transparency of the foreground's edges. Or a soft edge matte might be used to help with spill suppression around the edges of the foreground element. There may even be occasion to use an edge matte to apply effects after the layering is finished. It

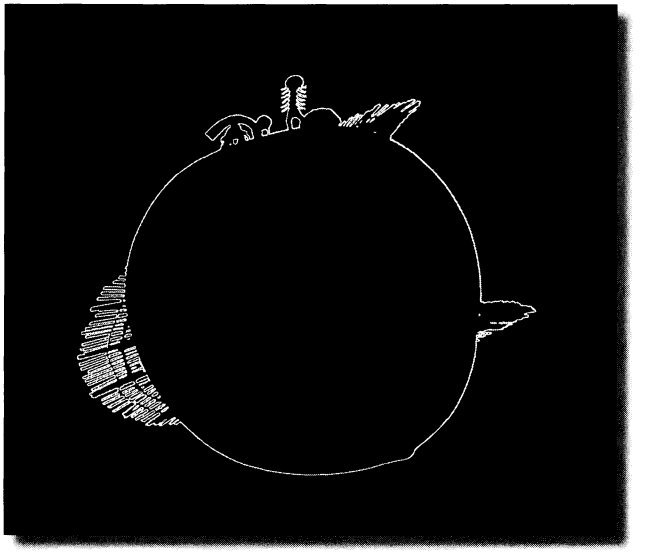


Figure 5.3 *An edge matte for the peach in Plate 2a.*

may be worthwhile, for instance, to apply a slight blur along the edge where the foreground and background meet *after* they have already been composited together. When done sensibly, over a very small area, this can look far superior to simply blurring the edges of the foreground element before creating the composite.

Combining Mattes

Instead of using a single technique to create a single matte, it is often the case that a number of different mattes are generated, each targeted to a certain portion of the image. For instance, certain settings may pull an excellent matte around a character's hands, yet these same settings may produce absolutely unusable results on hair blowing in the wind. A second pull may be tuned to deal with the hair, and the matte image that this produces can then be manually combined with the matte that works well on the character's hands.

Combining these mattes will require finesse in and of itself. Since the different settings will produce different edge qualities, a hard-edged split will tend to show this transition area. Instead, you will probably need to use an additional matte with a soft transition edge to split the different procedural mattes together.

Manipulating Mattes

Since a matte image is really just like any other image, it means that we can use our standard image processing operators to modify these mattes as needed. We already discussed this a bit when we looked at the luma-key, where we used a combination of operations to tune a matte. No matter what method we use to extract a matte, procedural or manual, there may be times that we wish to tweak that matte by a slight amount—to expand it or shrink it or soften the edges a bit. This practice is perfectly acceptable, but is abused so consistently in the industry that we want to be very careful about recommending it. Postprocessing a matte to soften edges is not a substitute for pulling a proper key in the first place. Even though some of these techniques can modify mattes so that they look acceptable at first glance, significant loss of detail may have occurred. This detail may have been something that could have been preserved had a bit more time been spent tuning the original matte-pulling parameters instead of quickly resorting to post-processing the result.

Having said this, we'll now assume that you have spent the time necessary to convince yourself that the matte you have pulled is the best possible using the tools you have available. If this is the case, yet your matte still does not feel acceptable, then it may be time to resort to other tools. We'll mention a few of the more typical problems that are seen with mattes that are procedurally generated and then suggest some possible remedies. These are only suggestions, and may be totally inapplicable to your particular situation. In fact, many of these

suggestions could easily cause more problems than they solve, depending on the matte in question.

Remember that *any* changes you make to the matte channel must also be reflected in the RGB channels if you are already dealing with a four-channel, premultiplied image. Do *not* simply blur or dilate the matte channel of a four-channel image, or you will most likely end up with a noticeable matte line when you go to composite your element over a background. We will assume that you are working with matte images that are separated from the RGB images and consequently will not remind you of the matte/image relationship for every situation in which it may be applicable.

Holes in the Matte

It is not uncommon to find that a particular matte pull has some undesirable "holes" in it. These holes are usually caused by grain or noise in the bluescreen element—grain that contains enough variance so that certain particles have coloring identical to the backing. The ideal way to eliminate this sort of thing is with the use of garbage mattes, but if this is not practical, you may be able to apply something like a median filter, which will remove single-pixel anomalies.

Another technique for dealing with excessively grainy bluescreen elements is to apply a very slight blur to the original bluescreen element, extract a matte from this blurred element, and then use this extracted matte in conjunction with the *original* bluescreen plate. By slightly blurring the bluescreen plate, the most egregious grain or noise will be toned down, and a cleaner matte can then be pulled. However, we don't actually want to have a softened foreground element in our scene, so we revert to the original, unblurred element for our image. This can be combined with our clean matte to produce what one hopes is a better composite.

Incidentally, there may be situations in which a few small holes in the foreground matte are not going to be noticeable. If the background is not significantly brighter or darker than the foreground, the holes will reveal something that is fairly neutral and innocuous. They will, at worst, appear to be a bit of grain on the element.

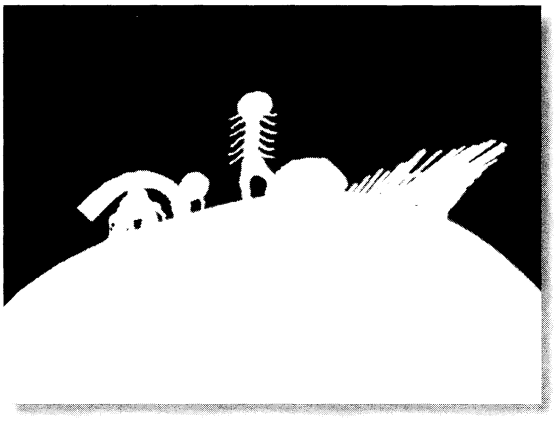
Hard Edges

Attempts to create a solid matte for the subject while completely removing the backing area will often involve a distinct threshold between the two areas. This can, as a side effect, produce a fairly hard edge to the matte. This is usually undesirable, since in the real world, even very sharp objects will have a slightly soft edge when viewed from a distance. The simple solution is to apply a slight blur to the matte channel, although by doing so you run the risk of introducing a matte line, since a blur will typically spread both outward and inward. You may end up doing some additional processing on the matte after the blur, to alleviate such problems. If your matte was too hard only in certain areas, you

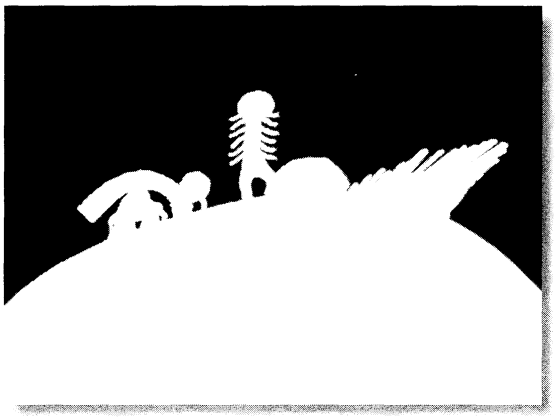
should use some sort of mask to control the blur, or limit its extent to only areas that are problematic.

Ill-Fitting Mattes

Very often we will find the need to slightly increase or decrease the coverage of a given matte or a portion of a matte. Many compositing packages will provide tools for doing this directly, using algorithms typically known as Dilate and/or Erode. These increase or decrease the relative amount of dark and light pixels in an image. Figure 5.4a shows a close-up of a small portion of a matte; the result after applying a simple Dilate tool is shown in Figure 5.4b.



(a)



(b)

Figure 5.4 *Fixing ill-fitting mattes. (a) Close-up of a portion of a matte. (b) The matte after applying a dilate tool.*

If your software does not have an explicit Dilate/Erode tool, you can usually come up with fairly similar results by using a combination of other operators. For instance, to dilate a matte, you can apply a very slight blur to it and then use an additional brightness or gamma to manipulate the medium-gray pixels that are created. There are any number of additional scenarios that might require explicit modifications to a matte. Some later chapters of this book will look some more at specific considerations that should be taken into account when creating a matte for an object.

CHAPTER SIX

Time and Temporal Manipulations

Although everything that we have discussed up to this point is related to the topic of digital compositing, most of it would also fit just as well into a book about using computer-based paint programs. We have talked about a variety of tools for combining and manipulating images, but primarily have looked at these tools only as they are applied to a single, static image. The focus of this chapter is to move beyond the single-frame mentality into a discussion about how the concepts that we've been discussing apply to a sequence of images.

Certainly a good deal of what we have already discussed is easily understood in terms of a moving sequence. Just about any compositing package (and even some paint packages) will allow you to apply the tools we have discussed to a sequence of images. But the reason digital compositing *must* be treated as a separate issue from simple digital painting is that techniques that would be perfectly acceptable when dealing with a single image become not only cost ineffective but sometimes completely unusable when the need arises to produce a sequence of images.

Before we go into more detail about methods for dealing with image sequences, let's take a moment to discuss the difference between viewing a single still image and a sequence of them.

APPARENT MOTION

The eye has a natural ability to perceive movement when shown a sequence of still images, assuming that the rate of viewing is high enough and the distance any given object moves within the scene is small enough. This illusion is known

by a variety of names, including "stroboscopic motion," the "phi phenomenon," or simply "apparent motion."

There is a common misperception that the reason we can see motion when viewing a succession of unmoving frames is that the human eye has a tendency to continue to transmit a signal to the brain for a few moments after the image is removed from view. This phenomenon is known as **persistence of vision**. Persistence of vision *does* explain why we do not see flicker when viewing a sequence of images at a high enough rate, particularly with a medium such as film or video, in which is a brief moment between each frame when the screen is actually blank. Persistence of vision will eliminate flicker only if the rate at which images are displayed is high enough. Film running at 24 frames per second (fps) is actually too slow to completely eliminate flicker; consequently, theatrical motion picture projectors are designed so that they show each frame *twice*, running at a rate of 48 frames per second. Even a rate of 30 fps is not really enough to completely eliminate flicker, which is why video was designed to show two interlaced field images at a rate of 60 fps instead. (This topic will be discussed in greater detail in Chapter 10.)

However, it is really a combination of both apparent motion *and* persistence of vision that allows us to be fooled into believing that we are seeing a moving scene when shown a sequence of still pictures. Apparent motion can actually happen at extremely slow frame rates, although generally a rate of about 16 frames per second is considered a reasonable minimum. This was the rate used for most of the original silent films.[1] Both the rate at which one views a sequence of images and the rate at which that sequence of images was acquired are important factors in digital compositing. We need to be aware of both, not only so that we can judge movement, but also because of the greater storage and processing requirements that arise when the number of frames increases.

TEMPORAL RESOLUTION

Just as is the case when dealing with a single image, in which we have a certain number of pixels to work with, when dealing with a sequence of images we will have a certain fixed number of these images, or **frames**, to work with. The more frames that are sampled over a given period of time, the greater the amount of information we have about the scene and the motion within it. In essence, this is yet another way of increasing the resolution of our imagery. We will refer to this as the **temporal resolution** of a sequence.

---

[1] Interestingly, the reason that the standard speed for film projection was increased to 24 fps was primarily to improve the *sound* quality for the new "talkies."

For the time being, let's assume that the images we are dealing with were captured at the same rate as that at which they will eventually be displayed. This rate is equivalent to the temporal resolution of a sequence and is usually measured in frames per second, or simply **fps**.

Most current feature films are captured and displayed at a rate of 24 fps. This is actually a fairly slow rate, bordering on the limits of the eye's ability to perceive flicker. Many artifacts, such as the distracting "strobing" one gets when the camera pans quickly, are often more pronounced in film than when dealing with higher-speed media.

Video in the United States is considered to run at a rate of 30 fps, although this is somewhat of a simplification, as we'll describe in Chapter 10. Various other countries may have different video standards: PAL, for instance, which is the video standard used in much of Europe, specifies a rate of 25 fps.

Other specialized film formats, such as the one known as Showscan, are designed to be played at even greater speeds. (In the case of Showscan, the rate is 60 fps.) The advantage of a format that is designed to display frames at a higher rate relates to the perceived increase in the "realism" of the experience for the viewer. The artifacts caused by a low temporal resolution (such as flicker, strobing movement, and noticeable grain) are all minimized. High-speed formats are usually coupled with increased spatial resolution as well, to give a much more satisfying image. The disadvantage of high-speed systems from a digital perspective is that they require significantly more data for any given amount of time. Even if you choose to scan and store your images at a similar spatial resolution as you would for regular film, the fact that the projection speed is 60 fps instead of 24 means that you need more than twice the amount of storage space for the same length shot.

Frames, once digitized, are usually numbered sequentially for the sequence, although often they will not necessarily be used exactly the way they were created. Two sequences of images need not be of the same length to be combined, and they need not be combined with their frame numbering, or even their acquisition rate, synchronized.

TEMPORAL ARTIFACTS

As mentioned, there are certainly a number of issues that start to become problems once we are dealing with sequences of images instead of just a single frame. Some of these problems, such as the large amount of data that must be managed, are reasonably well defined. If we want to work with 100 frames instead of just one frame, the amount of disk space will probably be about 100 times greater. But there are other problems that arise as a result of our moving image. Many of these problems will fall into the category we refer to as **temporal** artifacts. The

definition of this term should be fairly obvious—it refers to some visually noticeable problem that is a result of viewing moving images over time.

Very often you will find yourself putting together a composite and testing a number of different frames in the sequence. Each individual frame may look perfect, with nothing to belie the fact that it is actually a composite. With full confidence you process the entire sequence and send it to a viewing device. However, once you look at the result at the proper speed, everything falls apart. Matte edges crawl, objects moving around the scene are jerky and irregular, elements flicker, and you realize that a composite is more than just a collection of stand-alone images.

The eye is very sensitive to small movements. This is almost certainly a survival trait, a way of sensing danger in one's surrounding environment. Any sudden anomaly in the field of view, any unusual motion, will trigger a warning somewhere in the back of the brain. Unfortunately, this means that many small artifacts that are unnoticeable with a single image can become dramatically more visible when viewed as a sequence. Just about any movement can be a problem if it is not smooth and regular. An object's edge that does not have good frame-to-frame continuity due to an irregular matte will become immediately obvious when played at speed. Small changes in the rate of movement—accelerations and decelerations—will be noticeable and distracting. There is no single magic solution to these problems. The compositor should constantly test short sequences, not just single frames. Every effort should be made to ensure that techniques are smooth and continuous over time, and always make sure that you look at the complete moving sequence before you show it to your client or supervisor.

Now that we're aware of some of the problems that can arise when working with image sequences, let's go ahead and look at some of the ways we can work with and modify the timing aspect of image sequences. The next section will cover some general-purpose methods for modifying the speed of an image sequence. The reader should also be aware that a very specific timing conversion—the conversion between 24-fps film and 30-fps video—will be covered in Chapter 10.

CHANGING THE LENGTH OR TIMING OF A SEQUENCE

Whenever a sequence of images is captured, there is a certain rate associated with it, and this rate determines the number of frames that are captured for a given shot. The person capturing the images may have some control over the rate at which they are shot, but once a particular event is captured it will have been recorded using a specific number of frames. If, for some reason, the number of frames proves to be inappropriate for the desired shot, it will be up to the compositor to modify the sequence so that it is useful.

A number of tools and methods exist that can be applied to a sequence of images to modify its length or **timing**.[2] The most simple method entails the removal of certain frames from the sequence in order to produce a new sequence. For instance, we may wish to use only the first 120 frames of a 240-frame sequence. Or we may use some range of frames from the middle of the sequence, say frames 100 to 200. Note that choosing a continuous range of frames out of a larger sequence will certainly give you a shorter shot, but it does not actually change the *speed* of the action in the shot. Say we have a 100-frame sequence of images that shows a ball rolling from the left side of the frame to the right. At frame 1 the ball is just entering the frame; at frame 100 it is just exiting. If we need a shot that is only half this length (50 frames long), then using only frames 1 to 50 from this sequence would seem to fit the criterion. But the new shot will be that of a ball that starts at the left frame line and rolls only to the middle of the screen. The action of the shot is quite a bit different, and the ball itself still appears to be rolling at the same rate.

Instead of this shot, let's say that we need a 50-frame shot in which the ball's action is the same: It should still be seen to enter and exit frame over the course of the shot. The simplest method would be to reshoot the scene and just roll the ball twice as fast (or run the camera at half its normal rate). This is often the best solution, but may be impractical for a number of reasons. As we'll see in many other places in this book, the "simple method" for solving problems (reshooting the scene) is often *not* an option, which is why digital compositing becomes necessary in the first place. So, let's assume that we are not able to reshoot our scene and therefore need to come up with a solution that makes use of the existing frames.

The most basic method for increasing the apparent speed of the action in a sequence of frames is to remove, or "drop" selected frames. For instance, if we wish to make the sequence appear to be twice as fast, we could just remove every other frame. In our rolling ball example, we will be left with 50 frames, but they will contain the full action desired (the ball rolling from left of frame to right), and consequently the ball will appear to be rolling at double the original speed. By the same token, if we wish to change a sequence so that things appear to be moving twice as slow, we can double every frame. Unfortunately, these methods often result in a new sequence in which the action doesn't appear quite as smooth

---

[2] "Timing" is yet another potentially ambiguous term, particularly if you work in the film world. Film, when being developed and printed, goes through a color-correction step that is known as "color timing." In conversation especially, you may find people abbreviating this term and referring to the "timing" of a shot when they are discussing its color balance. *Usually* the context of the discussion will make it obvious whether they are talking about color or temporal issues, but not always.

as it should. The eye can perceive the artificiality of the process. In the case in which we double the speed of the sequence by dropping frames, for instance, we may see a noticeable strobing or jerkiness in the motion. This effect is mostly due to the fact that, had the moving object actually been traveling at twice the speed, the camera would register a larger amount of **motion blur**. Motion blur is literally a blurring or smearing in an image that is caused by the distance an object moves while it is being captured. An example of motion blur is shown in Plate 45, and the phenomenon is described further in Chapter 12.

In many situations, the lack of motion blur can be compensated for by adding a subtle blur to the shot, although you may find that it is necessary to restrict the blur to the proper areas of the frame, depending on the type of action. For instance, if the camera is unmoving and all the action is coming from an object moving across the frame, then any additional blur should be applied *only* to the moving object.

Slowing down the action in a scene by doubling frames is also subject to visual artifacts. In this situation, since the motion is effectively being reduced from (in the case of film) 24 frames per second to 12 frames per second, we are approaching the threshold at which apparent motion no longer works. The brain may start to realize that it is seeing individual images instead of a scene in motion.

Depending on the software you have available, there are a number of methods to help alleviate some of the artifacts inherent in simply doubling or dropping frames. One of the most common methods involves selectively mixing, or averaging, frames together. Let us look again at the case in which we wish to have a sequence move at twice its original speed. Figure 6.1a shows our original frames, numbered sequentially. Instead of dropping every other frame, we do a 50/50 mix between every pair of frames to produce a new frame. Our new sequence would be as follows:

1&2, 3&4, 5&6, 7&8, 9&10, 11&12, . . .

Thus, our new first frame is composed of both frames 1 and 2 of the original sequence, averaged together. Frame 2 is a mix of frames 3 and 4, and so on, as shown in Figure 6.1b. This method produces a sequence that is half the length and visually twice the speed of the original. It has the additional benefit of

Figure 6.1 *Averaging frames to change length and timing. (a) The original sequence of frames. (b) Double-speed image sequence.*

producing images that will appear less artificial. The process of mixing two frames together essentially gives a crude approximation of a motion blur effect for objects that are moving.

The ability to produce a new sequence by mixing together additional frames becomes even more important when you need to create sequences that are not whole-number multiples of the original sequence. Consider a situation in which we want to increase the speed of the animation by only 50% instead of 100%. Using an intelligent averaging tool, we can produce a sequence whose frames consist of the following:

1&2, 3, 4&5, 6, 7&8, 9, 10&11, 12, 13&14, 15, . . .

Most of today's compositing systems will allow the user to either enter a compression ratio (e.g., "reduce speed by 33%") or specify a frame-range mapping (e.g., "change the timing so that frames 1–8 of the new sequence are composed of frames 1–12 of the original sequence"). The system may even allow you to create a curve that controls the weighting of the various frames that make up your new frame. If your system does not support any of these tools, then you may find it necessary to create the intermediate averaged frames manually—potentially an extremely tedious task!

Finally, there is an even more sophisticated tool for modifying the timing of a sequence of images. It is currently available almost exclusively on high-end systems and is usually an extremely compute-intensive process. Using sophisticated algorithms, it can analyze the motion of individual areas (or even pixels) within a frame and actually produce new intermediate frames that are created via some specialized interpolation and warping parameters. It is fully expected that this technology will become increasingly available and will soon be a standard option on all compositing packages.

KEYFRAMING

Now that we have discussed some of the ways that we deal with sequences of images in our compositing system, let's go to the next step and discuss how the various tools that we defined in earlier chapters are extended to work on these sequences. Digital compositing is based on the notion that just about any sort of manipulation you may use to produce or enhance a single image can then be similarly applied to a sequence of images. This is not to say that such an application is necessarily trivial, and certain techniques, such as those involved when using a digital paint system, do not easily translate from a single image to a sequence.

Even with tools that can just as easily be applied to a sequence of images as to a single frame, we will soon realize that instead of applying the same settings to our entire image sequence, we will want to modify certain parameters so that

they change, or animate, over time. This raises the question of what the best way to animate these parameters would be. Obviously it could be quite painful to manually specify a different value for a particular effect on every frame of our sequence. Fortunately, there is a better way—a way that makes the job of animating values over time a relatively painless task. This technique is known as **keyframing**. Keyframing is a process in which, instead of explicitly defining a value for every frame of the sequence, the artist chooses certain "key" frames, assigns values to those frames, and then allows the computer to interpolate the values for the remaining frames. **Interpolation** is the process of using certain rules or formulas to derive new data based on a set of existing data. With keyframe animation, we are deriving values that fall between the existing keys that we have chosen and that will appear to make smooth transitions as we move through time.

Most books on computer animation techniques will give a great amount of detail about the process of using keyframe animation, but for the purposes of our discussion we will work with a simple example that explains the basic concepts. We'll look at how we might use keyframe animation to vary the brightness of an element in order to properly integrate it into a scene. Consider the case of an actor shot on bluescreen who is walking toward the camera. Let's assume that we've already done a wonderful job of pulling a matte for the actor, using some of the techniques discussed in Chapter 5. Now all we need to do is color balance him to the background plate in which he's supposed to appear. The only slight difficulty is that the plate dictates that he is walking from the far end of a shadowy corridor into a well-lit room. The bluescreen element was shot with constant lighting, so obviously we'll need to deal with animating the brightness of the character as he walks toward the camera. (Note: Animating the brightness of an element to simulate a change in lighting is common, but not ideal. A better composite would result had the element actually been shot correctly, with lighting that mimicked the background plate.)

Using basic keyframing techniques, we'll start with determining a proper brightness for the character at the first and last frames. At frame 1, we determine that applying a brightness of 0.4 to the foreground produces a pleasing, visually integrated result. At frame 300 (after he has completely entered the well-lit room), we decide that a brightness of 1.1 is most appropriate. By default, our software will probably linearly interpolate these two values to determine values for the intermediate frames. (There are a number of different ways to interpolate data between control points. We'll look at another in just a moment.) Figure 6.2 shows a graph of the linear interpolation between our keyframes. As you can see from the graph, at frame 150 (halfway through the sequence), the software has chosen a value of 0.75 (halfway between 0.4 and 1.1). Unfortunately, when we go look at frame 150, we notice that the actor has still not entered the room and consequently appears to be too bright for the scene. On closer examination, we realize that the

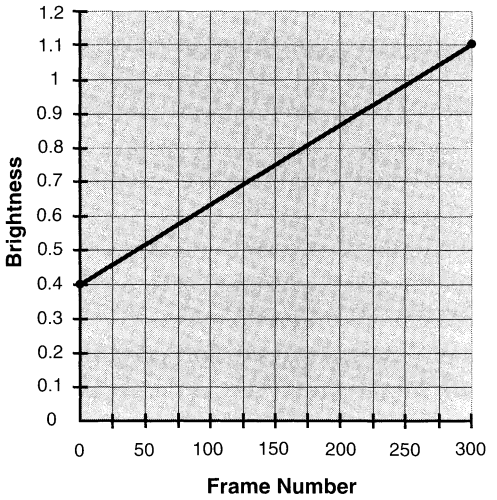


Figure 6.2 *Graph of linear interpolation between keyframes.*

actor doesn't enter the room until about frame 200, and should therefore remain fairly dark up until that point.

We then go back and look at a few more frames and determine proper brightness values for those frames. Figure 6.3 represents a graph of those new values, where we have added keyframes at frames 200 and 250. Note that we have changed our

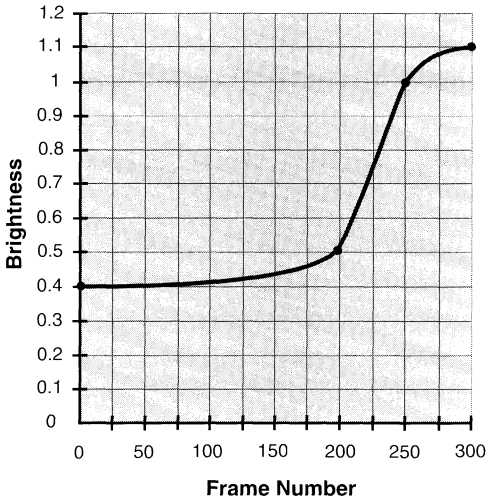


Figure 6.3 *Graph of values interpolated from keyframes using a more complex (nonlinear) method.*

interpolation method, so that instead of drawing straight lines between our key points, we are now using a smooth spline to create our curve. As you can see, the foreground element's brightness slowly increases as the actor approaches the room. Then, as he steps through the doorway, which occurs at about frame 220, there is a more rapid increase in brightness, lasting a few frames. Finally when he is fully in the room, where the lighting is constant, the value essentially levels off for the rest of the sequence.

This situation is, of course, a very simple example of how a single parameter might be animated over a range of frames. More complex scripts may involve animated parameters for many different layers.

There are a number of different ways to animate a parameter over time. Instead of using keyframes, we might simply use an equation, or we may import data from an outside source. If we are animating the movement of an object, it might be simpler to draw the motion path of the object and then control the speed at which the object moves along that path. Not all software will support all methods, but usually the same results can be achieved using a variety of methods.

CHAPTER SEVEN

Image Tracking and Stabilization

If you examine the history of visual effects in feature films, you'll notice that the vast majority of composites, be they optical or digital, occur in scenes in which the camera isn't moving or zooming. This setup is known as a **locked-off camera**, and until fairly recently it was almost a mandatory requirement for visual effects shots because it was extremely difficult to synchronize multiple elements in the same scene if their camera moves weren't exactly the same. However, it is often not possible, or even desirable, to use a locked-off camera. Multiple shots without camera moves can become boring and lifeless, and with today's sophisticated audiences may even cause the viewer to *expect* a visual effects shot.

Some of this problem has disappeared since the invention of the **motion-control camera,** which gives the camera operator the ability to repeat any given camera move multiple times. A background plate can be shot with a motion-control (or "moco") camera move, and then at a later time the foreground element can be shot with the same move. Motion control has a number of disadvantages, however, including the fact that it requires setting up a good deal of extra equipment (such as camera dollies rolling on a prelaid track). It also tends to be limited in terms of what type and speed of moves it is capable of performing. In many cases motion-control equipment can also be fairly noisy while in use, thus limiting its acceptability when shooting scenes with dialogue in them.

In situations in which the need arises to composite together elements that were shot *without* identical camera moves, one can now turn to a technique known as **tracking**. Tracking is the process of selecting a particular region of an image and determining that region's movement over time (i.e., on a sequence of images). Once the information about the movement of a region (or group of regions) is

obtained, a variety of operations (which we'll discuss in a moment) can be applied much more effectively.

As we've seen with many other digital compositing tools, even a process such as tracking is something that was possible to do within a traditional optical compositing system, although the time it took to manually track an object was considerable. Additionally, it took a very skilled, patient operator to accurately hand-track an object into a scene. Today's digital tools, however, make it much easier to approach a shot in which the camera was not locked down. A great deal of work has been done to automate the tracking process,[1] and almost all compositing packages now include the ability to exactly determine the movement of a particular feature over a sequence of images.

TRACKING AN ELEMENT INTO A PLATE

There are a variety of situations in which tracking can be used, but probably the most common is when you need to synchronize the movement of an object you are adding to the scene with something already in the scene. The object in the scene may be moving, or the camera may just be moving relative to the object.

Consider the sequence of images in Figure 7.1. We'll use this image sequence as our example footage to help us explain the specific steps that are involved when tracking an element into a scene.

These steps will vary somewhat depending on what software you are using, but the concepts are fairly universal and the tools for automatically tracking a certain region of an image are all based on the same basic principles. Let's say we intend to add an object into the scene that appears to be attached to the front

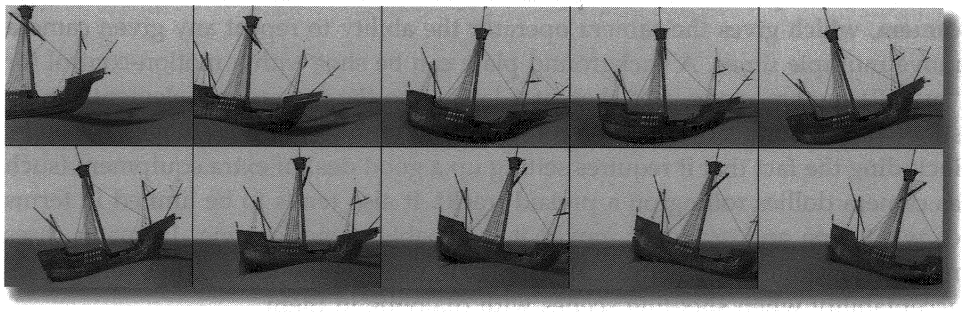


Figure 7.1 *Image sequence for tracking example.*

[1] Tracking as an automated process was actually developed by the U.S. Defense Department to aid in missile guidance.

of the ship as it moves through the frame. First, you choose the feature in the image that you wish to track. (The next section gives some suggestions on how to go about choosing a feature that will track effectively.) Specifying the feature is done by outlining a **region of interest** (usually abbreviated as **ROI**) with a bounding box. An example of this is shown in Figure 7.2, where we have placed our ROI so that it outlines a small outcropping on the bow of the ship.

This region will now be used as a pattern that the software attempts to locate in all the other frames in the image sequence. Let's assume that you specify this region while looking at the first frame of the sequence, as shown in our example. The tracking software takes this reference ROI from this frame and begins by comparing it with the same location in the second frame. But not only does it compare the sample with the *same* location in the next frame, it also compares it with regions that are slightly *offset* from the original location. For instance, it pans

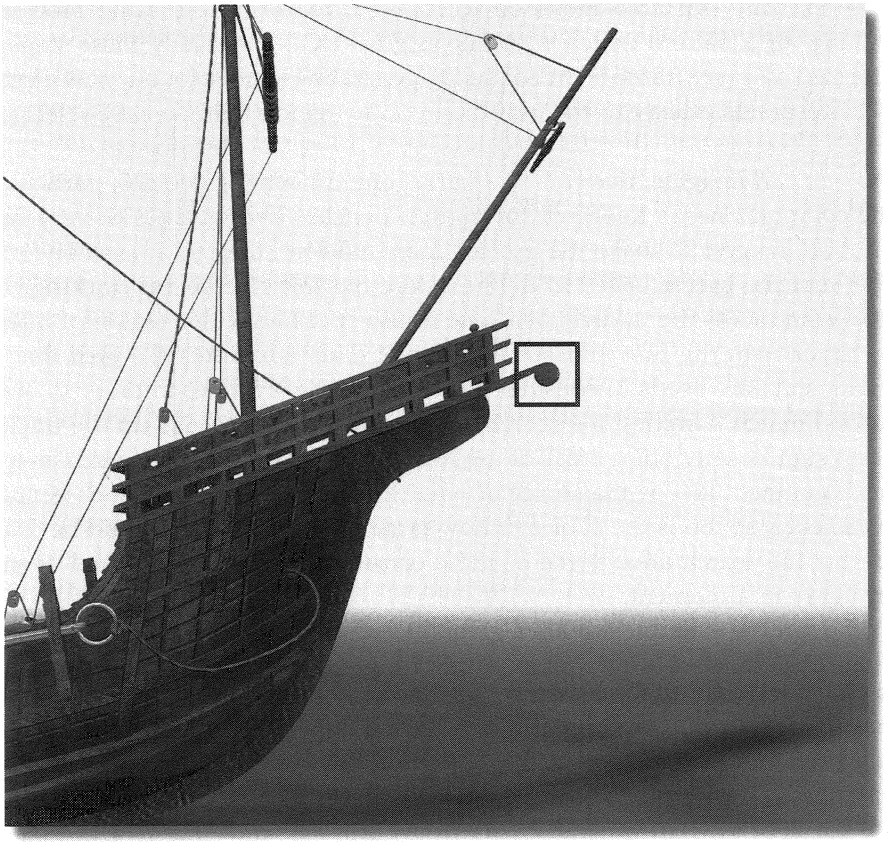


Figure 7.2 *Choosing an area to track using a bounding box.*

the source ROI by one pixel to the left and does a similar comparison. Then it pans two pixels to the left and does the comparison. Every time it tests a new offset, it keeps a record of how much that tested area deviates from the original sample. Eventually, after doing an exhaustive search, there will almost always be a region that matches far better than any of the other tests with the original pattern. This region is then accepted as the new location of the feature in frame 2. For instance, our tracking software may have gone through its exhaustive search and concluded that the best match with the original ROI from frame 1 occurs when it moves the sample two pixels to the right and three pixels down.[2] In order to properly match this movement, the element you are adding into the scene will need to be moved by an equivalent amount—two pixels to the right and three pixels down. Usually this movement is done automatically by the tracking software, once it determines the proper new location. The value of this new location is also entered into a table that keeps track of the movement of our pattern for every frame that we are tracking.

The software can now move on to frame 3, repeating the same process of searching for a pattern match with our original ROI. Eventually these steps are performed on every frame of the sequence, generating a move for our new element that corresponds exactly to the movement of the region that we have selected to track.

As you can imagine, this can be a fairly time-consuming process, particularly if the computer needs to search for pattern matches over a series of very high-resolution images! To make the process more efficient, there is at least one more region that the user is asked to define before the software begins tracking. This region is known as the "search area" and is also specified with a region-of-interest box, larger than the first and surrounding it. This box limits the area that the tracking software needs to consider on the subsequent frames in order to obtain a pattern match. Limiting the search area in this fashion speeds up the tracking process significantly, since areas of the image that are outside the search area are never examined. Just as the choice of what feature one wishes to track is not an exact science, so too is the choice of how large a bounding box should be drawn to define the search area. Some of these considerations will be looked at more thoroughly in a moment.

Although the example we have given discusses the process as if it were only testing areas that are whole-pixel offsets from the original, in actuality a good tracking system will exhaustively check subpixel offsets as well. Very few things will conveniently move a whole-pixel offset between every frame, and the ability to track accurately at the subpixel level is one of the things that distinguishes a

[2] Incidentally, the algorithms used to determine what exactly constitutes a "match" can vary quite a bit between different pieces of software; consequently, the accuracy can vary quite a bit as well.

mediocre tracking system (of which there are many) from an excellent one (of which there are very few).

Choosing the Feature to Track

Probably the most important thing that experienced trackers have developed an instinct for is how to choose the right area to track. The first step of this decision-making process is to visually examine the entire sequence with which you are working. There are a number of reasons for this step. Obviously, one of the primary reasons has to do with the fact that you are looking for a feature that remains visible throughout the necessary duration of the shot. If the feature you are tracking becomes obscured or leaves frame, you will need to expend some additional effort dealing with this problem, as described in the section entitled "Human Intervention."

Secondly, you are looking for a feature that remains fairly constant in terms of its shape and color. The tracking algorithm will usually be trying to find something that matches the original sample, and if the pattern changes over time it will no longer be such a match. Fortunately, there are methods for tracking objects that vary over time. Generally they involve choosing to periodically update the sample region you have chosen, replacing the original pattern with the most recent successful match. This change needs to be fairly gradual for the algorithm to deal with it, and a sudden, drastic change in the region that occurs over only a few frames will probably still cause the tracker to lose the region in question. Depending on the situation, this process of redefining the sample pattern may occur any number of times throughout the duration of the shot. As usual, the degree to which this process is automated will depend on the software that you are using.

Not only do we need to worry about the visibility and constancy of our ROI over the length of the sequence, but we also need to make sure that the ROI moves in a way that is appropriate for the element we are adding. In our example we chose to track the tip of the ship because we were planning to attach an object to this point. But we don't always need to track the exact attachment spot, as long as it does not move *relative* to that desired location. In other words, tracking some other feature on the front of the boat would probably also give us data that was reasonably accurate. But the farther away from the actual attachment point we move, the less accurate the data becomes, since these areas are moving differently. If you look at the example images in Figure 7.1 again, you can see that the rear of the boat is actually rising as the bow is descending. Obviously, if we wish to attach something to the front of the boat, it would be useless to track the back of the boat.

Finally, there are some specific visual characteristics that we look for in the region that we are choosing to track. Generally, you will want to pick a high-contrast area, with noticeable variations in color and brightness. Certain software

or scenarios may work better if you preprocess your element to increase the necessary qualities. Usually this entails some kind of overall contrast manipulation in an attempt to emphasize specific features that you would like to use as a tracking target. Be careful not to emphasize undesirable artifacts (such as noise) at the same time, or you may actually decrease the accuracy of the resulting track.

The feature chosen should have variations along as many axes as possible to minimize the chance of the software coming up with a false match. For example, choosing an ROI like that marked as A in Figure 7.3 could prove to be a problem, because all the test regions that lie along this section of the mast look extremely similar to the original sample and will probably cause the software to be unable to accurately lock to a particular point. The ROI marked B has the same problem in a different direction; if you attach an object to this point, you will probably notice that your object slides up and down along this diagonal. A much better

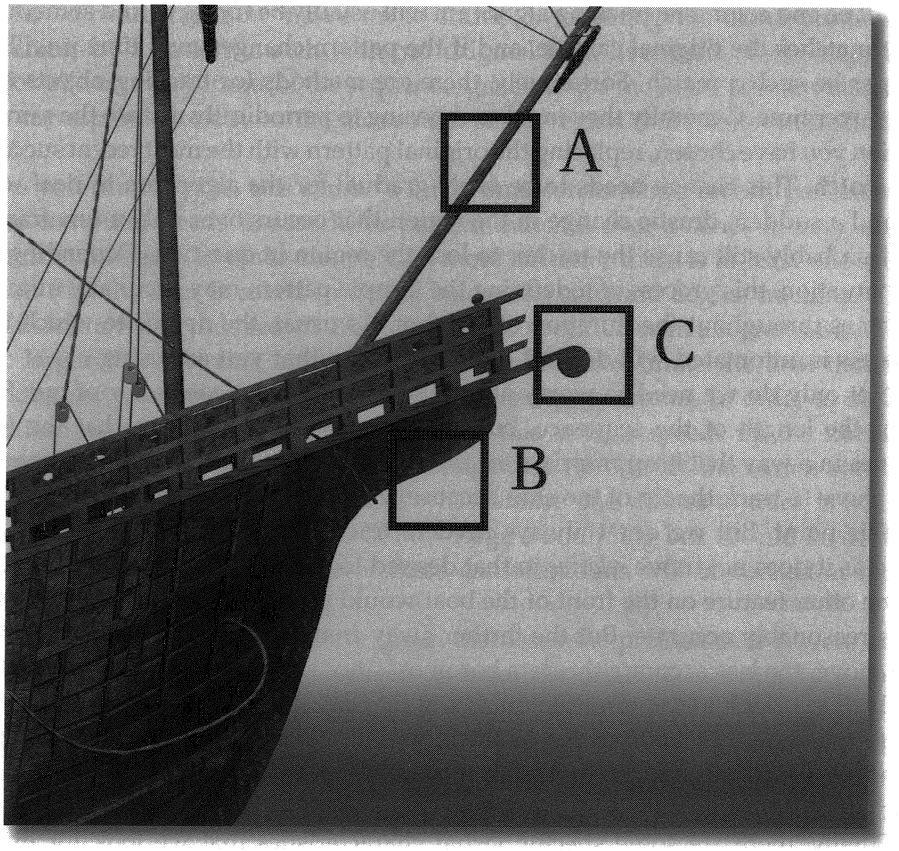


Figure 7.3 *Three possible tracking locations.*

area to choose would be something like that shown in choice C, where movement along any axis would be unique.

If you know ahead of time that you intend to track something in a scene, you may even be able to add something to the scene that will be easier for your tracking software to identify. By placing specific **witness points** into a scene, you will be assured that there are high-contrast, uniquely shaped references that the software can track to. Ideally, you will be able to place them in locations that will not be obscured or suffer problems with lighting variations. Depending on the scene, these witness points may or may not need to be removed from the final shot. They may be covered by the new element that you are tracking into the scene, or they may be innocuous enough (such as strategically placed props in a scene) that they can remain. If you do need to remove them, the tracking data they provide may simplify the process of tracking a neutral piece of imagery to cover them.

Limiting the Search Area

Another important reason to examine the entire sequence ahead of time is so that we can get a rough idea about the maximum amount our feature will move between any two frames. This information is important because it will help us to choose a search area that is no larger than necessary. This choice is generally a trade-off, since searching a too-large area will result in the search taking far longer than necessary, but choosing too small a box runs the risk of not including the proper area in certain frames. For features whose motion relative to the camera is fairly small, your search area box can also be fairly small, but if you are tracking an object that is moving significantly between every frame, then you'll need a wider search area (and consequently the tracking process will take longer).

Ultimately, you don't want to spend a great deal of time worrying about the size of this search area. It's better to err on the side of caution and choose a search area that is larger than you think necessary. These days, tracking algorithms are fast enough that the extra time it will take to search an overly large area will probably be negligible.

HUMAN INTERVENTION

Although most people would like to think that tracking an area in a scene is a completely automatic process, this is rarely the case. There are any number of reasons why the software may not be able to track certain things properly. Consider the case in which something else in the scene moves to obscure the object you were trying to track. Suddenly there is nothing in the search area that even remotely matches your sample. It need not be something as drastic as having an

object move in front of our area of interest. Something as simple as a quick shift in lighting in the scene, caused by a light turning on or a surface catching a reflection, can confuse a tracking algorithm. Film grain may even cause enough variation between frames to introduce inaccuracies.

It is not uncommon for problems like this to show up several times during the process of tracking an area through a sequence of images. Generally, one has two choices. If the problem only happens for a few frames, you may be able to just manually enter in your best guess for where you think the feature would be on those frames. It will be much easier to make this guess if the object's movement is smooth and predictable. Erratic movement that varies significantly on every frame or a feature that is obscured for a larger period of time will require a second method. In this case you should try to choose a *different* point in the scene from which to continue your tracking, making sure that the new region is moving equivalently with the original ROI. Sophisticated software will usually be able to make a transition to this new point with little or no difficulty, but on some systems you may need to manually deal with the offset in the tracking data that happens when you switch points. Usually this intervention merely involves subtracting a constant X and Y offset from each frame's data after the changeover occurs.

Using Tracking Curves Manually

Although most software will allow you to simply specify the element you wish to attach to the feature in question and will automatically deal with tracking *and* moving it, there may be times when you need to interact with the tracking data directly.

Once a feature has been tracked, your software should have in storage a sequence of either pixel locations or pixel offsets for every frame of the shot. Some software returns the data based on the absolute pixel position of the image you tracked; others return the data as relative offsets from your original position (now defined to be [0,0]). Ideally, you have some way of examining a curve that is a plot of this data. This visual test can immediately show any gross problems or mistrackings that have occurred. For instance, Figure 7.4 shows a graph of the X movement that the tracking software has calculated for an object. The chances are high that the tracker encountered some problems around frame 40, indicating that you may wish to either retry the tracking or manually intervene.

Instead of having the software automatically apply any tracked movement to an element, you may wish to do it yourself. In this case you will need to attach the movement curves to a general Pan operator explicitly. By doing so, you will have additional flexibility, and you can easily modify the data first or use it in conjunction with a secondary move.

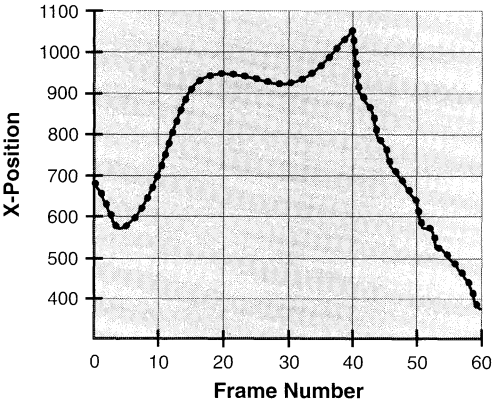


Figure 7.4 *Graph of tracking curve with a potential problem.*

STABILIZING A PLATE

Tracking techniques are also commonly applied when one needs to **stabilize** a sequence of images before they are used within a composite. This need tends to arise most often when the sequence was originally intended to be locked off but something happened during the shoot that compromised that intent. Any number of factors could conspire to produce such an "unstable plate." The camera might have been bumped or jarred while the film was rolling, a strong wind may have caused a slight wobble, or there may have been mechanical problems with the camera itself that caused the normally stable registration of the film as it moves through the gate to be less than acceptable. In this situation, the motion of the plate itself is tracked and analyzed, and then the unwanted movement is removed.

To do this, we still begin by tracking some feature in the frame to determine its motion over time. But instead of choosing a moving object in the scene, we will choose an object or feature that we believe is supposed to be stable and unmoving. By tracking this allegedly stable point, we can determine how far it deviates from a locked-down condition. Now we can take the resultant data and invert (negate) it so that we can move the plate in the *opposite* direction on each frame. We are effectively subtracting the plate's natural motion from itself to produce a new plate that no longer moves. (Again, applying our tracking curves in this fashion is assuming that our software isn't being used to automatically do this process for us.)

A modification of this technique can even be used to remove high-frequency jitter from an otherwise desirable camera move. Consider a tracking curve such as that shown in Figure 7.5. We can see that the motion has a lot of small, erratic

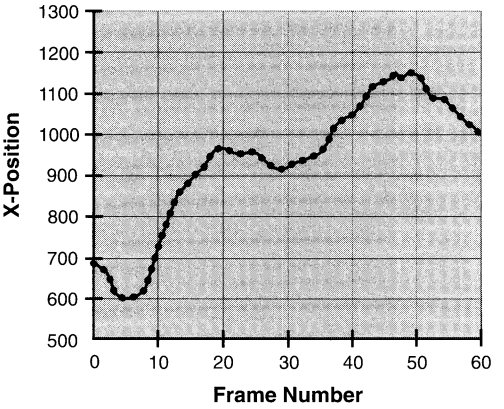


Figure 7.5 *Tracking curve with high-frequency noise.*

moves over time, yet there is also a large overall camera move. In this situation, where we are looking at the *X*-translation curve, we see that the high-frequency noise is moving by up to 20 or 30 pixels in *X*, whereas the overall curve covers a range of nearly 600 pixels over the 60 frames shown. If we want to remove only the high-frequency artifacts, we need to create a curve that does the same basic overall move but has none of the noise. Figure 7.6 shows such a curve.

Depending on your software, you can either create this second curve manually, matching the overall move as much as possible, or use your software's built-in tools for smoothing a curve to remove high-frequency noise. In either case, once this new, smooth curve is available, we subtract it from the original. (Essentially,

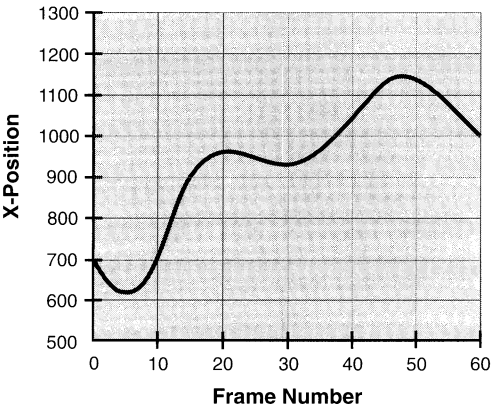


Figure 7.6 *Smoothed tracking curve (without noise).*

we subtract the value of the curve at each frame from the value of the original curve at that frame.) The resulting curve is shown in Figure 7.7; it is now a curve that contains *only* the unwanted high-frequency noise from the original.

If we invert this curve and use it to pan the original plate, we will end up with a plate that still has the required move yet doesn't exhibit any annoying extra jitter.

TRACKING MULTIPLE POINTS

So far, we have talked only about one-point, or single-point, tracking. It is capable of giving us the *X* and *Y* position for a single point on an image so we can attach an element to it or stabilize the image based on that point. Single-point tracking only gives us enough information to deal with simple, overall positional changes.

If, instead, we track *two* points in a scene, we now have enough information to accurately reproduce any rotational changes that occur between the two points. What's more, by measuring the change in distance between the two points, changes in scale for a given object can also be computed and mimicked. (Again, most systems would automate this process, but if not, you could also compute this information manually.)

Taking this concept even further, four-point tracking gives enough information to calculate simple warps that mimic perspective shifts. We can track the four corners of a moving object and lock an image or element inside those points, often creating a perfect match for any transformations caused by object or camera moves. The logical extension of four-point tracking is a system in which multiple points in a scene are tracked and then used to control some sort of warping

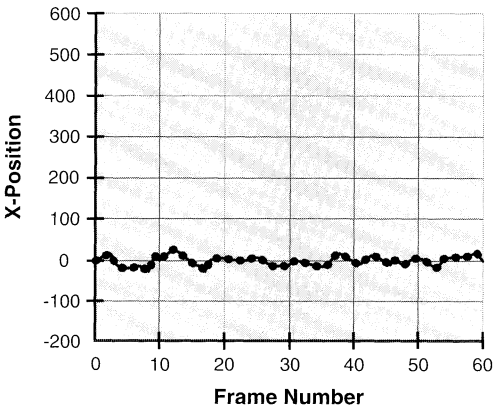


Figure 7.7 *Noise-only curve.*

tool. Every moving point modifies the warp on the image, so that very complex movement and distortions can be matched.

Finally, tools are now becoming available that can take tracking data from multiple points and, sometimes in conjunction with proper survey data for the scene in question, recreate the full 3D camera move for the shot. This data can then be fed back to a 3D animation system that will be able to render elements that exactly match the moves from the real scene.

CHAPTER EIGHT

Interface Interactions

Although all digital compositing systems share the same basic concepts and algorithms, the method by which a user interacts with the system can vary widely. This interaction is primarily determined by the **user interface** that is provided with the software in question. These days, this user interface is typically a graphical front end to the tools, and hence is usually referred to as simply the **GUI**, the **graphical user interface**. It would be pointless (and impossible!) to try to discuss the specific quirks of all the various interfaces out there, so instead we will attempt (somewhat arbitrarily) to define certain user-interface categories, discuss the different philosophies behind various working methods, and then spend some time looking at basic tools that have similar equivalents on all common platforms.

Because the process of digital compositing is an interactive one, requiring extensive feedback at every step of the way, it is very important that your compositing software does not hinder your efficiency. Certainly a great deal of this ease of interaction will have to do with how experienced and familiar you are with the particular paradigm that your software employs. Just about any system, including a completely text-based command-line compositing system, can be used to produce high-quality images. But the best systems recognize that the operator's time is a significant cost factor in producing imagery, and consequently will provide as many tools as possible for optimizing the process.

We will break user-interface-related issues into two primary categories. The first is the more complex of the two and deals with how a compositing artist combines and controls the various tools that were discussed in Chapters 3 through 7. The second category, discussed in Chapter 9, encompasses the use of specific tools for viewing and analyzing images and sequences of images.

Different software packages may have extremely different methods of interacting with certain tools. Even the hardware that is used may be different: Certain compositing software may support a keyboard-only interface, other software may expect that your primary interaction will be with a mouse, and some packages dictate that you work primarily with an electronic pen and tablet. But the concepts *behind* the hardware are well defined and vary little from platform to platform. These concepts are what we will focus on in this chapter (and the next). The most important issues are related not to the interfaces used for specific tools, but rather to the overall working methodology that the software allows you to develop—the workflow.

WORKFLOW

Up to this point, we've primarily treated the various image manipulation and combination tools as if they were isolated entities. In other words, we have mostly discussed how a single operator affects a single image. In the real world, the compositing process is one of applying *combinations* of these tools, each modifying some aspect of the image sequence until the final, intended imagery is produced. There may be situations in which dozens or even hundreds of these tools are used. To see an example of this, examine Plate 93, which is a graphical representation of all the operators that were used to produce a single specific shot for the movie *Titanic*. Each box represents a single operator, and the overall structure and grouping of the boxes is based on the order in which these operators were applied. Essentially, these numerous discrete operators were combined in an intricate fashion to produce the complex, custom-designed machine that was needed to produce the finished shot. For the bulk of our discussions in this chapter, we will try to use slightly less complex examples than the one shown in Plate 93.

Whether one is working with a large, complex composite or a small, simple one, the basic workflow is essentially the same. The flowchart in Figure 8.1 covers the basic steps. Note that this process is fairly recursive, since newly created image sequences will often be treated as source elements for the next step in a composite.

Let's look at the individual steps as they occur. In step 1, the images with which we will be working are imported into our system. Files of various types may be brought in, and, depending on the software that you are using, there may be a need to do some specialized processing to format the images in a particular way. (Chapter 10 discusses format issues in much greater detail.) Certain high-speed compositing systems may require you to preformat the images so that they can be rapidly stored and retrieved from a special area of the disk, for instance. Other systems will simply keep track of the image sequences' locations in a general file system and will read images into memory as needed. To speed up

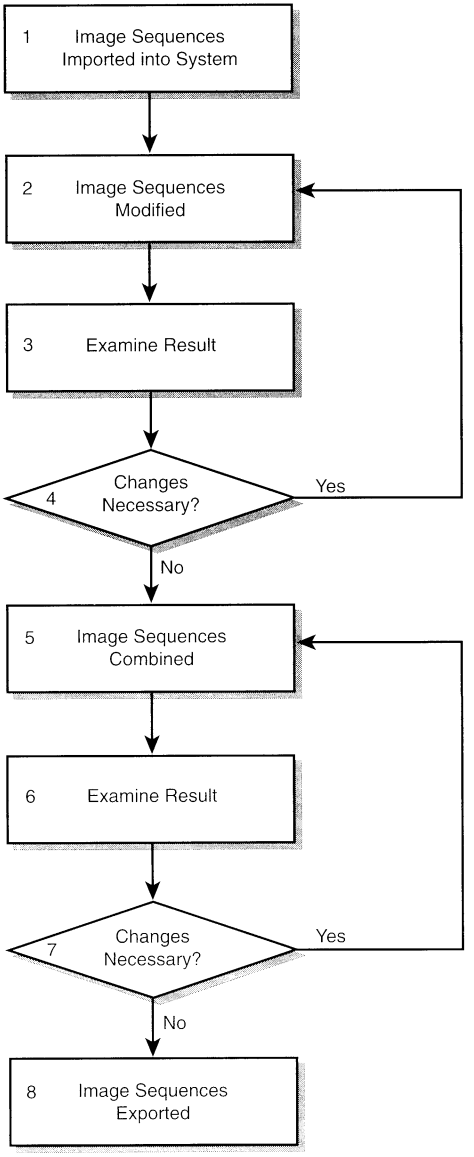


Figure 8.1 *Flowchart of compositing workflow.*

the interactivity of the process, the software may let you work with low-resolution **proxies** of the original frames. This option is discussed in a later section of this chapter.

In step 2, we start applying the image processing operators from Chapter 3 (and Appendix A). This process could include any number of color corrections, geometric transforms, or spatial filters—everything from brightness to blurs to rotations.

Now we take a look at the result of the operations we've applied to the sequence and decide whether we're satisfied with the result. This is step 3, the "visual feedback" step, which may cause us to return to step 2 for additional adjustments. We may bounce back and forth between steps 2 and 3 several times until we have a result with which we are happy. (Understand that you do not necessarily need to decide on all the image processing operations that you will use before you start to combine images. Most systems will allow you to add additional effects to the original layers as necessary, although your flexibility may be more limited with certain types of systems.) Also, although the steps are represented as single boxes in our flowchart, in a real composite we may go through these same steps with any number of original source sequences.

At some point we will need to combine some of these modified sequences, using tools such as those discussed in Chapter 4. This process is represented by step 5 in our diagram and is immediately followed by an analysis (usually visual) of the result. Once again we go into a feedback loop, with potentially a large number of iterations in which changes are made and the results are examined. In this case, we have the added complexity that the changes may need to be made to the operator that combines the images (step 5) or to one of the original image modification tools (step 2).

The flowchart obviously represents a very simplified view of the process. The actual method of interacting with your compositing package will depend heavily not only on the specifics of the user interface, but also on the basic paradigms your software uses. One of the most important distinguishing factors is whether your system is considered to be a **batch compositing** system or an **on-line** (or interactive) **compositing** system.

ONLINE VERSUS BATCH

Although it is convenient to try to divide compositing software into the categories of either "batch" or "on-line," the distinctions between the two are no longer all that clear. Historically, a batch compositing system was one with virtually no interactivity. It would generally rely on text-based scripts, include no graphical user interface, and would provide little or no immediate feedback for any image processing operations. On-line systems, on the other hand, usually consisted of

some dedicated hardware that was capable of applying a limited set of operations to an image sequence, creating a new sequence with each step. If the compositor needed to apply several operations, each one would be done as a discrete step, producing a modified sequence that would then have the next operator applied to it. These days, pure on-line systems tend to exist only in the video world, where dedicated hardware is used to rapidly process the imagery in question. Such systems are fast, but lack a great deal of functionality and flexibility.

More general-purpose compositing systems are now available that combine functionality from both paradigms. This evolution is not surprising, since ultimately the actual operations that are applied are no different between the two systems. It is the way in which the compositing artist interacts with the software that is different. Even today, different systems will tend to enforce different workflows. The most important distinction is whether or not the system is designed primarily to create a compositing script or to iteratively produce new image sequences with every step. This distinction is an important one, and consequently it is still useful to categorize compositing systems as either on-line or batch. Let's look at the typical workflow for each category in a bit more detail.

On-line systems tend to work in the following fashion. First, a sequence of images—or several different sequences—is brought into the system. The sequences are imported, either from videotape or from the general file system of a computer, into a specialized disk that allows for much quicker access to the images. The system is capable of viewing these sequences (often called **clips**) interactively, displaying them at the proper frame rate as needed.

The user then decides on the first compositing effect that is necessary (a color correction, for instance) and applies it to the image sequence in question. The clip is then processed frame by frame (but usually fairly quickly) with this effect, producing a new image sequence. The compositor then uses this clip as the source for the next operation (combining it with a different clip, perhaps), and yet another clip is generated. The process continues, with a new sequence of images produced at each step, until the final composite is complete. The intermediate image sequences are not destroyed at every step, but rather remain available should the need arise to redo any of the earlier steps in the process. These clips can, at some point, start to take up a good deal of disk space and memory, and thus one will often need to dispose of sequences that are less likely to be needed again. This will be a judgement call, and disposing of a particular sequence may require the user to manually redo a number of steps if that sequence later proves to be necessary.

Batch systems take a different tack. Although one still works by manipulating and combining elements, there is no explicit processing of these operators to produce a new sequence at each step. Instead, one is working to produce a specialized script that, once perfected, can be run as a batch process. Each operator

that is to be applied to an image sequence is tested on a few select frames. Once the desired parameters are determined, an instruction is recorded into the script that describes this operator and the parameters it is using. The script will continue to grow, and any number of effects or layers can be added, but the only processing that is taking place is being done on just a few selected frames, probably at a lower-than-normal resolution in order to speed up the testing process. The lower-resolution image that is being used for testing purposes is known as a **proxy** image, which will be discussed in a moment.

Eventually, the compositor has built a script that describes all the steps that will be needed to create the completed composite. Only then will the compositor choose to execute the script. The compositing software reads the script and applies all the operations that are necessary to produce the final composite. This step generally does not require any user supervision, since all the steps are already contained within the script. The process also does not necessarily need to run on the same machine as that on which the compositing artist created the script, and in fact it often does not, particularly if there are a number of machines in a network that can be used to share the workload. Ultimately, the script finishes executing and the new images that make up the final composite are stored on the computer's disk.

The process of building a script instead of actually processing and viewing each layer as it is created can be cumbersome, although this is as much a product of poor interface design as anything. The benefits of a script-based system are many, since complex, multistep processes can be modified and automatically rerun as necessary. Many on-line systems would require the operator to manually redo several of the same steps if there arose a need to change one of the original elements or effects that had been applied.

In general, on-line systems tend to be set up on machines that have a good deal of CPU and graphics-processing horsepower, which they will need in order to be interactive. Batch systems, on the other hand, tend not to need such extreme interactivity and consequently can run on much less expensive hardware platforms. The distinction between the two types of systems is starting to become less and less extreme, however. Most systems these days are, in fact, hybrid systems, with varying levels of interactivity and scriptability. Very few on-line systems do not offer at least some ability to group multiple operations together before computing a new result, and very few batch systems do not offer a number of tools that increase interactivity and provide reasonably rapid feedback. Ultimately there is no reason why software cannot be written that is both highly interactive as well as being fully scriptable.

For the duration of this chapter, as well as in certain other places in this book, we will discuss compositing systems primarily from a script-based point of view. This is done for a number of reasons. First, even if you are working on a system

that has no ability to create a batch-processing script, you should still think of compositing in terms of the entire *process*, as opposed to individual steps. Second, script-based systems are much more common and tend to be less expensive, and as such are available to a much wider audience. Third, on-line systems that have poor or limited script-generating ability are really missing a significant piece of functionality, and we cannot ignore that functionality if we wish to give a complete overview of the compositing field. Finally, it is much easier to write about compositing in terms of a script-based system, since it allows us to represent the entire process of creating a composite in the same fashion as most script-based systems represent their data to the user—via a set of graphs or flowchartlike diagrams.

METHODS OF REPRESENTING THE COMPOSITING PROCESS

We need to define a few conventions here so that we can better discuss large compositing scripts. Let's walk through the creation of a very simple composite, defining a few terms and conventions as we go along. We begin with two source sequences, labeled "Foreground" and "Background." Our goal is to combine the two images so that the foreground is over the background and the two sequences are well integrated. Let's say that the foreground image is initially too dark, so a brightness layer is added to it. The background needs to be defocused a bit, and so a blur layer is added to that. We also decide that the background is a little too bright, and so a brightness modifier is added there as well. Finally, the modified foreground is layered over the modified background.

There are any number of ways to formally define this process. We could use an interface known as "English," since it is presumably something that most readers of this book are familiar with. To describe this process as succinctly as possible in English, we might say:

> Read in an image sequence labeled "Foreground," apply a brightness of 1.2 to that and then place the result of this operation over the image sequence that is created when you read in a sequence labeled "Background," apply a blur of 4.0 to it, and apply a brightness of 0.8 to that.

As you can see, the English language is not necessarily the best tool for describing compositing operations. The above description is fairly wordy and is probably not as precise as it should be, since the brightness of 0.8 could potentially be interpreted as needing to be applied *after* the Over operation occurs.

Now, if we were to represent this process using a simple text-based compositing system, the syntax might look something like this:

```
Output = Over(
                Brightness(
                        Foreground,
                        1.2),
                Brightness(
                        Blur(
                                Background,
                                4.0),
                        0.8))
                )
```

For those of you who are accustomed to working with a compositing system that has a nice GUI, this may look like a crude and cryptic way of representing a simple compositing script. But it certainly contains all the important information. We can see that a brightness of 1.2 has been applied to the sequence called "Foreground." We can see that a blur value of 4.0 has been applied to the "Background" layer, and then a brightness value of 0.8 is added above that. Finally, the Over operator takes the two sequences and combines them.

Consider now the chart shown in Figure 8.2, which is yet a third way of representing a compositing script—using a hierarchical "tree." For most people, Figure 8.2 is far more intuitive than either of the first two descriptions. It quickly shows the complex relationships between the images and operations in an easy-to-read format.

Each box in this tree is usually referred to as a "node" of the graph. The flow of this graph is from left to right. That is, when the operations are executed, the first nodes that are evaluated are those at the left of the graph, and then the operators to the right of that are evaluated, and so on, until the final node, at the far right of the graph, is executed. Any node that feeds directly into another node is considered to be the "parent" of that node; consequently, any node that is immediately *to the right of* the parent node is considered to be a "child" node. Thus, the Over operator is the child of both of the Brightness operators.

One thing that you will immediately notice about the diagram in Figure 8.2 is that you do *not* see the values of the variables assigned to each operator. This is

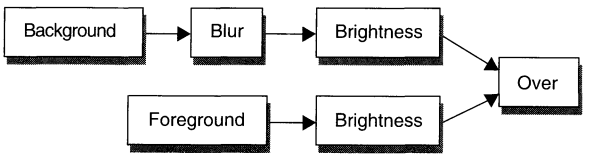


Figure 8.2 *Simple compositing script represented by a hierarchical tree.*

intentional, and common with most systems. Once we start to create larger scripts, we will find a need to limit the amount of information that our GUI is presenting us in order to better grasp the overall flow of the script. There is usually some method (such as double-clicking on one of the node boxes) that gives an expanded view of the specifics of that node, including the value of any associated variables. Your particular software may have many additional features; you should try to become as familiar as possible with them. Some software may allow you to explicitly name each node or to attach notes to them. This information can be helpful if your script grows to be large or if someone else will also be modifying parts of the same script.

Throughout the rest of this book we will use this simple tree format to depict groups of compositing operators. Again, we must stress that this is merely one of many possible ways to look at this kind of data: Your interface and mileage may vary. Most good compositing systems will give the user a multitude of different ways to view the data that is being manipulated, allowing the user to choose which method is most efficient for a given person or situation.

Take a moment now to look again at the script shown in Plate 93. This is a large script, but not unprecedentedly so. Composites produced for film, where attention to detail is critical and the increased resolution allows for a huge number of relatively small elements, can easily take months to set up and finesse.

Compressed Trees

As you can imagine, it can quickly grow difficult to keep track of where individual nodes are located in a large compositing script. This certainly underscores the importance of organizing your data efficiently and naming your nodes coherently and consistently. There are, fortunately, some other tools that can make the task a bit easier. One of the most important is the ability to select a portion of a graph and "compress" it, that is, to replace it (visually) with a simplified representation.

Consider a tree like the one shown in Figure 8.3. As you can see, our background image has several image processing operations layered on top of each other. Let's say that we've precisely tuned these layers to where we want them and that we no longer want to worry about them as individual objects. We can select the five

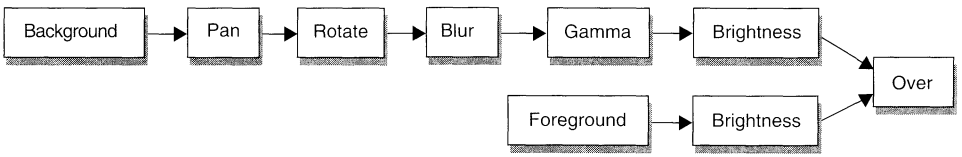


Figure 8.3 *More complex compositing script.*

layers and group them as a new compressed layer, which we'll call "BG_fixes." The simplified graph would look like Figure 8.4.

As usual, remember that not all software packages may support this sort of functionality, and even if they do, the paradigm may be different. If your software doesn't support this type of grouping directly, you can sometimes accomplish a similar effect by the use of precomposites, discussed more in Chapter 11.

Timelines

While the tree-based graph is very useful for viewing complex scripts, there is one piece of information that such a display does *not* address well, namely, the timing relationship between nodes, particularly for image sequences that are offset in time relative to one another. To better represent this information, we will use a **timeline graph** instead. The timeline lets us lay out the various events not only relative to each other, but also to an absolute time. Depending on what sort of work you are doing, you may find it useful to display the units of this graph either in common real-world units, such as seconds or minutes, or simply as individual frames. If you wish to display your timeline with real-world units, your system will need to know what the frame rate of your output medium will be—24 fps for standard film, 30 fps for NTSC video, and so forth. We will show all our time-related graphs with the units representing frames.

When dealing with layers from a timeline point of view, we are really only interested in two pieces of information: the length of the image sequence in question, and the starting point of this sequence. (A third useful piece of information, namely, when a sequence ends, is obviously an easily computed byproduct of the first two.)

Consider the case in which we want to have an object not appear in the scene until several seconds have elapsed. Perhaps the object is an explosion that is only 48 frames long, and it needs to begin after our background has been running for about 100 frames. Figure 8.5 shows a way in which we might represent this scenario. As you can see, we have a representation of our two layers, as well as a scale at the bottom of the diagram that shows us an absolute range of frames. The layers that we have available are positioned relative to this scale, so that their starting points and their lengths can be accurately measured. This sort of view gives us a great deal of information at a quick glance. We can see the relative

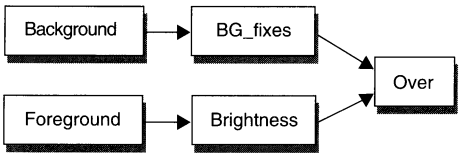


Figure 8.4 *Complex script represented by a hierarchical tree with a compressed branch.*

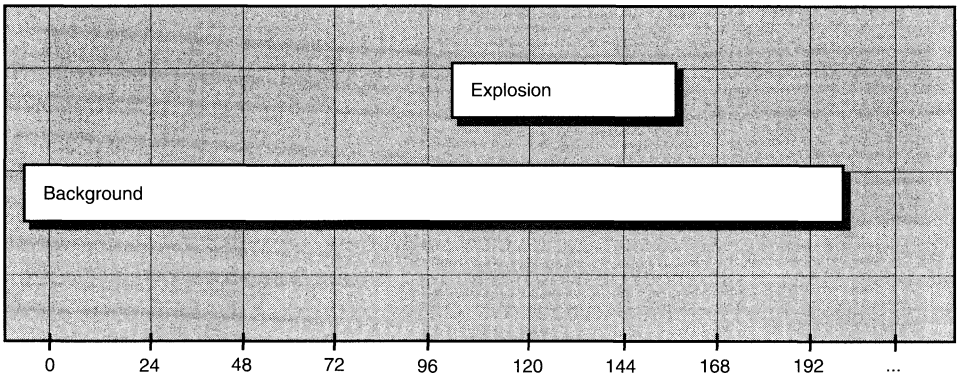


Figure 8.5 *Timeline graph.*

length of the two sequences—how the background is about 200 frames long and the explosion element is 48 frames long. We can also see that the explosion won't be used until about frame 100. This information would be far more cumbersome to determine using any of the other display tools that we've discussed so far.

The timeline view is useful in a variety of different situations and should be considered an essential adjunct to the standard node-based view of a script. Not only is it useful for viewing information, but it can also be used for manipulating the script as well. For instance, to change the point in time at which a certain layer appears, you could simply grab the layer in the timeline view and drag it to the desired starting frame. The timeline view makes it much easier to visualize and manipulate operations that involve a transition between two different layers. A dissolve between two different sequences can be indicated as in Figure 8.6, and adjusted by simply sliding the second layer so that the amount of overlap is of the desired duration.

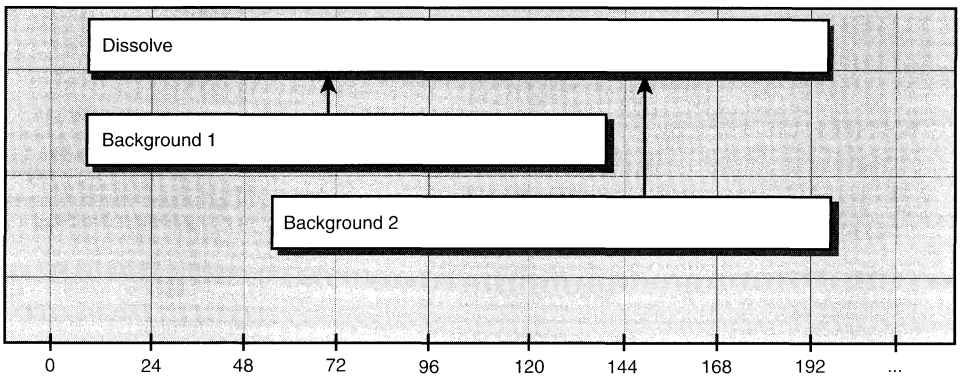


Figure 8.6 *Timeline graph showing a dissolve between two image sequences.*

CURVE EDITORS

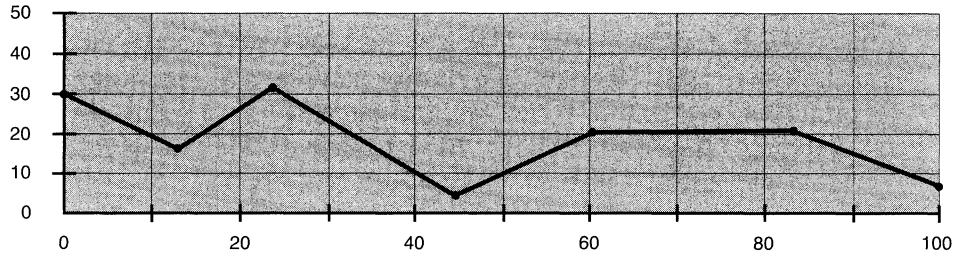
As we've discussed in several earlier chapters, there are situations in which it will be convenient for the user to define a curve to control some aspect of a composite. This curve may represent a variable that changes over time, or it may represent a range of values in a look-up table. No matter what the eventual use of such a curve will be, there are some basic tools that are used for creating and manipulating curves. We usually call a group of such tools a **curve editor**, and we will present an example that uses many of these tools in a fairly standard layout. Some form of curve editing is available in most graphic packages. We will not go through the concepts in a great deal of detail, other than to define some basic terms and to highlight specific features that should be present in an above-average editor.

Take a look at Figure 8.7. We will use this diagram to point out some of these specific features. The diagrams show a number of curves that reflect variables that change over time. Thus, the *X*-axis represents time and is measured in frame numbers. The *Y*-axis represents the value of the variable in question. A curve that is not based on time, such as a LUT curve, would have differently labeled axes, but would otherwise be dealt with in exactly the same fashion as a time-variant curve.[1]

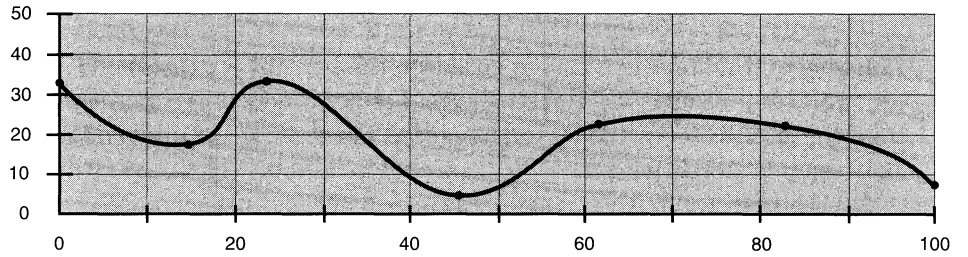
We are primarily concerned with a curve editor's ability to create, modify, and view a curve or group of curves. More specifically, a good curve editor should allow us to do the following:

- View multiple curves in the same window for direct comparisons. Multiple curves should be easily distinguishable, ideally through the use of color coding.
- Zoom in and out of a curve, scaling the *X* and *Y* axes nonproportionally as needed.
- View the specific numerical *X* and *Y* value at any location along the curve.
- Label individual curves with unique, user-defined names so that data can be better organized.
- View the **control points** on a curve and be able to directly access their specific numerical values.
- Create and manipulate individual control points.
- Insert and delete control points in an existing curve.
- Choose the type of interpolation used between control points. There are a wide variety of potential interpolation techniques, and different packages may vary slightly in what they offer. Figure 8.7a shows a curve with simple linear interpolation—the control points are connected with straight lines.

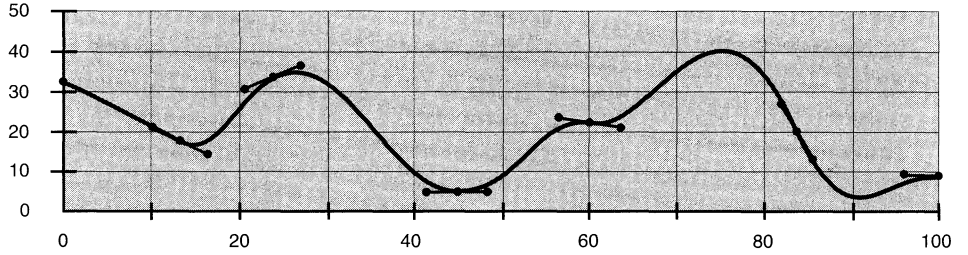
[1] Frame-based curves will usually limit you to nonfractional frame numbers, whereas other variables that can be controlled via a curve may not need such a limitation.



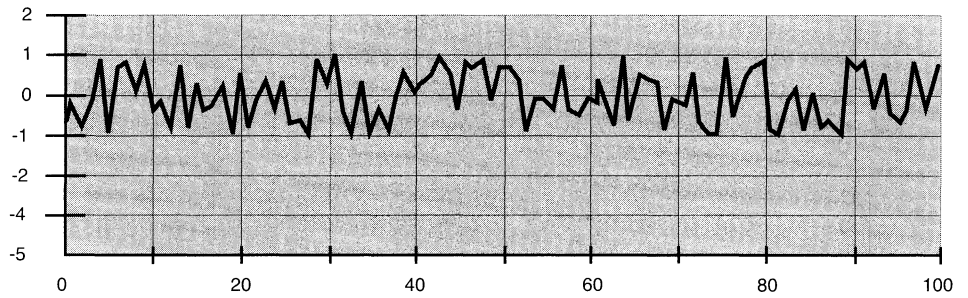
(a)



(b)



(c)



(d)

Figure 8.7 *Curves. (a) Linear interpolation. (b) Spline interpolation. (c) Hermite spline interpolation. (d) Function-based curve.*

Figure 8.7b uses the same control points, but interpolates with a smoother **spline curve** instead. Figure 8.7c uses a particularly powerful type of spline (usually known as a "Hermite spline") that allows one to control not only the position of the control points but also the tangents of the curve as it passes through these control points. These tangent controls are represented as additional "handles" that sprout from each control point.

- Create a curve based on a mathematical equation. The equation language should support standard functions as well as the ability to create a curve using random numbers within a given range. An example of this is shown in Figure 8.7d, a curve that is defined to be a random value between 1 and –1.
- Combine and manipulate curves based on mathematical expressions. This can work in a number of different ways—either a new curve is generated from the expression or the existing curve is maintained and the modifying expression is stored with it.

  For example, we may want to take a curve which defines some fluctuating values and tie it to two different effects. The range of values for the curve is fine for the first effect, but the second effect may need a much larger range. In such a situation, we should be able to mathematically define a new curve that is simply a constant value multiplied by our first curve. This is shown in Figure 8.8, where the bottom curve is our first curve and the top curve is our second.

  Notice that every value in the top curve is five times greater than the equivalent value in the lower curve for any given point in time. Ideally, the

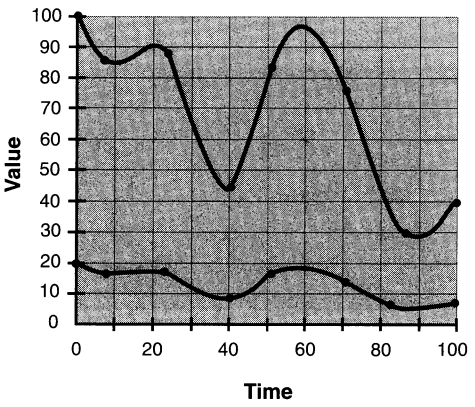


Figure 8.8 *Defining a new curve (top) mathematically based on a existing curve (bottom).*

second curve will still reference the first, and should we manually update the original, the second curve would automatically change as well.

- Apply arbitrary smoothing to an irregular curve. A number of different algorithms exist for smoothing a curve and removing unwanted noise or spikes. Figure 8.9a shows an unsmoothed curve, while Figure 8.9b shows the smoothed equivalent. Chapter 7 touched on some situations in which this capability is useful.
- Append two curves together to create a new curve. Ideally this operation includes the ability to apply some kind of smoothing to the connection point to prevent abrupt changes in value.
- Read and write a curve from or to a file on disk in a simple ASCII format. There is no industry standard for how a curve should be represented, but as long as the format that is used is text based and well documented, it is usually a trivial task to modify curves taken from one editor so that they can be read by another.
- Resample a curve to produce a new curve with a different number of control points. This capability is useful for a couple of reasons. First, you may wish to convert a simple curve that is defined by only a few control points into a curve that has a control point for every frame. This curve could then be written to a file and exchanged with any other package or device that works on a frame-by-frame basis—anything from a 3D animation system to a motion-control camera. It is also useful to do the reverse, that is, to import a

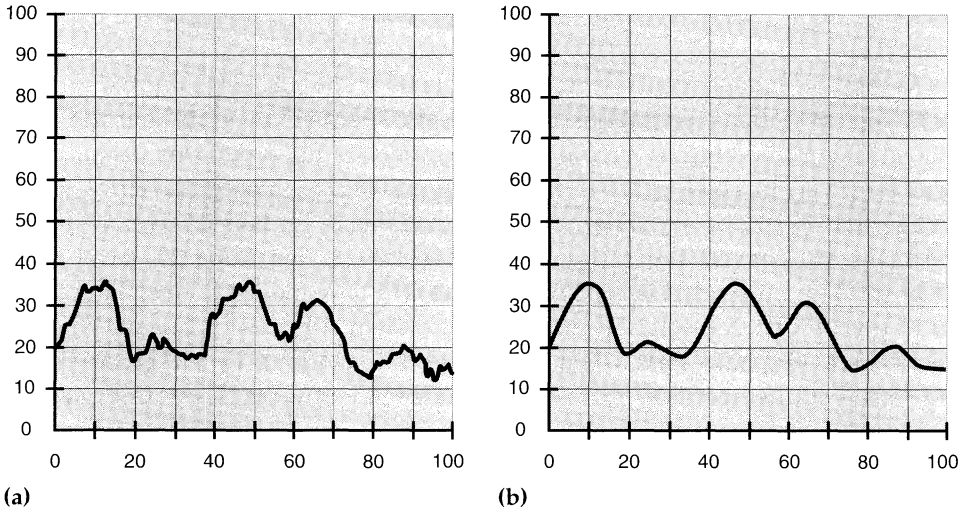


Figure 8.9 *Curve smoothing. (a) Original curve. (b) Smoothed curve.*

curve that is defined by an explicit value at every frame and convert it to a simpler representation that uses fewer control points. This curve will then be much easier to manipulate, as well as requiring less memory to represent it.

WORKING WITH PROXY IMAGES

If you are working on a computer that is not capable of displaying the images with which you are working in real time at full resolution, or if you do not wish to spend as much of your time waiting for the computer to process intermediate test images, you may wish to work with lower-resolution proxy images instead. A **proxy image** is a scaled-down version of the image that you are currently working on. The proxies are created either on the fly, as needed, or are precreated (manually by the user or automatically by the software). Working with proxy images usually requires (or at least implies) that you have some kind of batch-processing system available. The theory is that you can do the majority of your compositing work in low resolution; once you have taken it as far as possible, the steps you have defined will then automatically be converted and applied to the high-resolution equivalents. Some final tweaking will probably need to be done to deal with any unexpected problems that were not visible at low resolution, but the hope is that most of the work can be accomplished at low resolution in a much faster time. To best take advantage of the proxy image method, your compositing system should be **resolution independent**. This term means that the system can take a script created for low-resolution (proxy) images and apply it to the high-resolution equivalent images, while still producing the expected imagery.

Resolution independence is not as simple as it sounds, since a great number of image processing operations are normally specified in units that are pixel dependent. Something as simple as the parameters for a pan, for instance, are usually given in pixel units. If we specify a 200-pixel horizontal move for an image that is 400 pixels wide, we move that image by half its own width. If, however, we convert our script to high resolution, that same 200-pixel move applied to an image that is 2000 pixels wide will only move the image a fraction of its own width. A resolution-independent system either automatically converts the units to the new scale when we change our working resolution or simply represents all its operators in resolution-independent units in the first place. A pan, for instance, could be specified as a percentage of the image's width instead of as an absolute number of pixels. If you do not have a resolution-independent system, you can still work with proxies—you'll just need to do the conversion yourself. Knowing which operators need conversion can be tricky. Any operator

that physically moves an element by a specific pixel value will usually need converting, but element scaling will not. Rotations will probably need the location of the center of rotation converted, but the angle of rotation would remain the same. Convolution-based effects such as simple blurs will need to be converted, but color corrections will not.

Most compositing systems will, at the very least, produce temporary low-resolution proxies to speed the interactivity of testing. This feature is different from a fully functional resolution-independent system, but it still can be very useful. It may be limited to single-image test frames, for instance, but will still allow you to preview certain operations at this lower resolution and interactively tune any necessary parameters before applying them to the full-resolution original.

Chapter Nine

Image Viewing and Analysis Tools

In this chapter we will talk about a variety of ways to get information about the images with which one is working. This information is often visual in nature—certainly the most basic thing one can do with an image is to look at it—but there are also other tools that can be used to provide numerical or statistical data about an image. As was the case in the last chapter, different compositing systems may vary significantly in the way their tools deal with certain things, including the ability to view and analyze images. But the basic concepts are consistent throughout the industry, and consequently most of what we describe in this chapter will be available, in some form or another, on just about any basic compositing system. The processes and tools we describe will be useful *throughout* the compositing process. Before any operators are applied to an image sequence, you will need to know as much as possible about the images you will be using, and during the process you will constantly need information about the results of the various steps.

When it comes to viewing images, always keep in mind that the image you see represented on your computer screen is by no means guaranteed to accurately represent the way the image will appear in its final form. Ultimately, it is only by viewing your imagery in the format and medium for which it is eventually intended that you can be absolutely certain of what it will look like. Thus, if you are creating images for film, you must *see* them on film before you can be sure they are correct. While care should be taken to ensure that all viewing devices are calibrated to a consistent standard, don't assume that an image that looks acceptable on your computer monitor will look the same when it is displayed via a different method, even if that monitor is properly calibrated to some standard.

IMAGE VIEWERS

Certainly the most basic tool is one that can display a single image. It should allow the user to view individual channels of the image, including the alpha and *Z*-depth channels, and to zoom in and out of various areas of the image to see more detail. If the image you are viewing is of a greater resolution than your computer's monitor, your viewer should allow you to see a portion of the image at a true one-to-one pixel ratio and to resample the image so that you can see the entire image on your screen.

As you make iterative changes to your images, you will often need to do a detailed comparison between various trials. The best way is to be able to view the two images either by rapidly flipping between them or by interactively wiping or mixing back and forth between them.

If you are working with images that are stored in some sort of nonstandard color space, you may also find it useful to be able to apply a compensating LUT to the image when it is viewed. This LUT is strictly for display purposes: The data in the image is not changed; rather, the color conversion happens only in the viewing tool, for the benefit of the viewer. Chapter 15 discusses a couple of different situations in which this sort of functionality will be useful.

FLIPBOOKS

Not only will you want to view a single image, but most of the time you will also want to be able to look at a sequence of images, ideally at a fast enough rate so that they appear as a moving sequence. This type of viewer is often known as a "flipbook" tool, a term that goes back at least as far as the days of traditional animation, when one would quickly flip through a stack of sequential images to check the motion. The abilities of most flipbook tools are very tied to the hardware that you are using. Many flipbooks are reliant on the amount of main memory in the system for how many frames they can display as a moving sequence. Large images will mean that the system can display a shorter sequence, and smaller images will allow a longer sequence.

If you want to compute the amount of memory that will be needed for a computer to display a given image, don't assume that you can simply look at the file size of the image as it is stored on disk. Flipbooks generally need to uncompress an image before displaying it, and if your image file uses any compression then this size will not give you an accurate estimate. What you can do, usually, is to compute the amount of memory required by simply multiplying the width of the image (in pixels) by the height of the image (in pixels) by the number of bits per channel that your flipbook uses (which will generally never be more than eight, since this is the maximum most monitors will support) by the number of channels that the flipbook will display (usually three or four, depending on whether your

flipbook loads the matte channel for animated playback) by the number of frames in the animation. This gives the total number of *bits* that it will take to load the given sequence, so divide by 8 to get the number of bytes.

Some systems may make use of specialized storage hardware to display image sequences directly from a hard disk, instead of consuming main memory. The speed of playback with either scenario is very system dependent. You will almost always want to view your sequence at the same speed as that at which it will eventually be used, but you may find that your system is unable to keep up with this rate if using full-resolution images. Usually this problem can be solved by displaying lower-resolution images instead. Consequently, you will need to determine on a case-by-case basis whether the particular test you are working on is something that you want to see at the proper speed or the proper resolution. A good flipbook tool will give feedback about whether it is able to maintain the speed you have chosen.

Most of the features discussed for a single-image display should also be accessible for the flipbook display. You should still be able to zoom in and out of the moving sequence, view specific channels of the moving sequence, interactively compare sequences, and view a sequence with a specialized LUT in place.

IMAGE STATISTICS

In addition to a variety of different viewing modes, you should be able to get some numerical or statistical information concerning the images with which you are working. You will certainly need to be able to determine the spatial resolution of an image, as well as its bit depth. Be aware that just because an image is contained in a file with a particular bit depth does not mean that that image actually has information commensurate with that bit depth. An 8-bit image can be saved in a 16-bit file format, but doing so does not actually create any new information.

Other information may or may not be available, such as the average color in an image, or the high and low extremes. This information will be useful in a number of situations, particularly those in which we are attempting to synchronize the colors and contrast between two images. Even more useful than an overall analysis of statistics is the ability to analyze specific *areas* of the image, a topic we will discuss next.

Pixel or Regional Information Tools

There are times when you would like to determine information and detail about a subregion or even a particular pixel of an image. Most software will allow you to turn on a mode in which you can point to an area of an image and get feedback about the exact X and Y coordinates that your cursor is indicating. You should also be able to determine the RGB value of an individual pixel or get some type

of average value for a region. This information can be used to help with color-correction decisions or to check the solidity of a matte. Ideally, you will even be able to select the average color from an area and paste that value into a variety of different tools.

If you are viewing an image whose spatial or color resolution is greater than that of your display, be sure that any feedback tools you are using are providing information about the *original* image and not the proxy you are viewing. It is not uncommon, for instance, for nonprofessional viewing tools to simply return values from the image that is displayed on the screen, instead of looking back to the original image. If the original image was scaled to fit on your monitor, or if your tool is reading color values from an eight-bit video card, then the values returned from this image will be inaccurate relative to the original file.

Further statistics about an image can be determined using a specialized graph known as a **histogram**, which we will discuss next.

Histograms

A histogram is a way of displaying a plot of the number of pixels for each given brightness value in an image. Consider the four-bit, single-channel image shown in Figure 9.1b. With only four bits to represent the color in the image, each pixel can have one of 16 possible values. The graph shown in Figure 9.2 is a histogram of this image.

As you can see, there are a large number of pixels that have medium-gray values, but relatively few pixels that are pure white or pure black. In fact, we

(a) (b)

Figure 9.1 *Sample image. (a) 8-bit image. (b) Image represented with only 16 colors.*

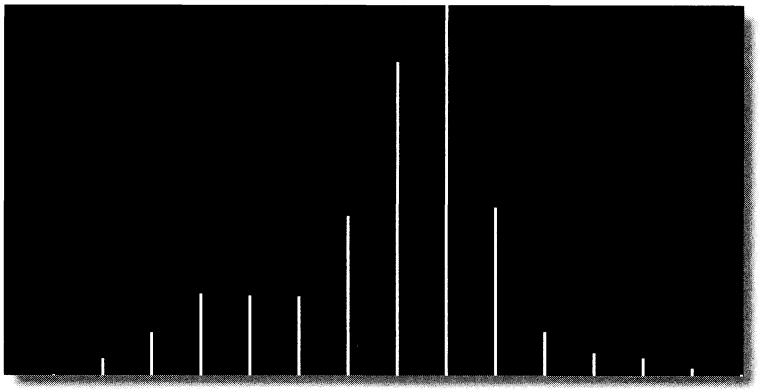


Figure 9.2 *Histogram of Figure 9.1b.*

could go so far as to tell that there are about 95,000 pixels with a value of 0.6 (60% gray), but only about 375 pixels with a value of 1.0 (pure white). The point of a histogram, however, is not to derive such *specific* statistics, but rather to examine overall data *trends* for the image.

Figure 9.3 shows the same type of histogram for the full eight-bit image, in which all 256 possible intensity values are graphed. This histogram presents a normal distribution of values, indicative of an image that was exposed and reproduced fairly normally. A histogram tool can also be useful for determining if an image has been artificially modified and if data has been lost. Unlike the uniform pixel distribution in Figure 9.3, Figure 9.4 gives a strong indication that the image has had some kind of image processing operation performed on it. The sharply

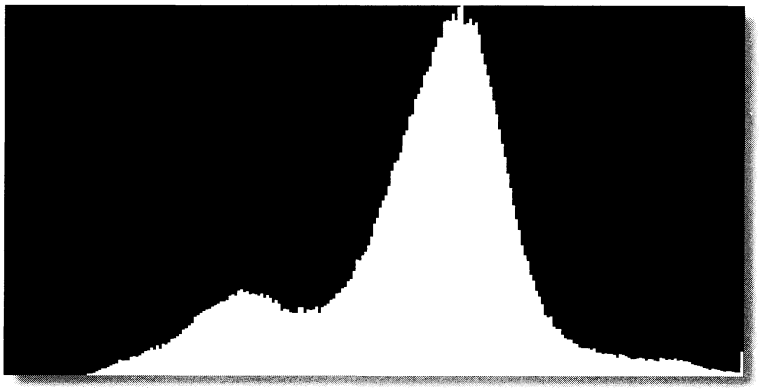


Figure 9.3 *Histogram of Figure 9.1a.*

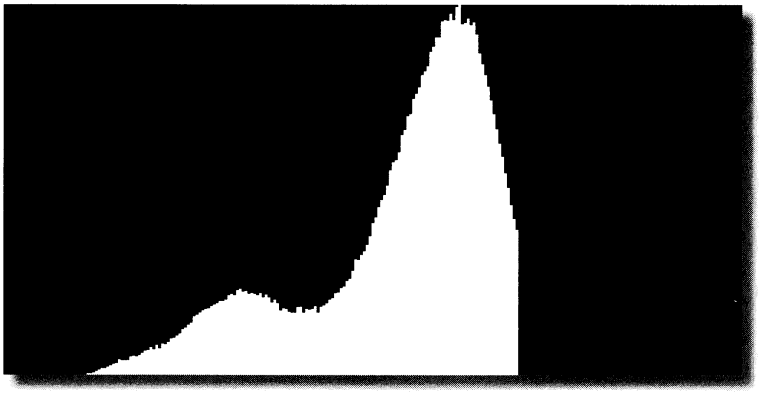


Figure 9.4 *Histogram of an image with data loss.*

clipped region at the high end of the palette is almost certainly an artifact of some unnatural data truncation.

Determining whether an image has been digitally modified is one of the most useful pieces of information that a histogram can provide. Most color corrections, for instance, tend to create an image that has a less even distribution of colors, which will show up as noticeable gaps in an otherwise normal histogram. To understand why, think about the situation in which we apply a brightness of 2.0 to an image. This operation will multiply every pixel by 2, producing an image in which all the values of every pixel must be an even number. There can be no pixels with an odd value, and consequently the histogram will look something like that shown in Figure 9.5.

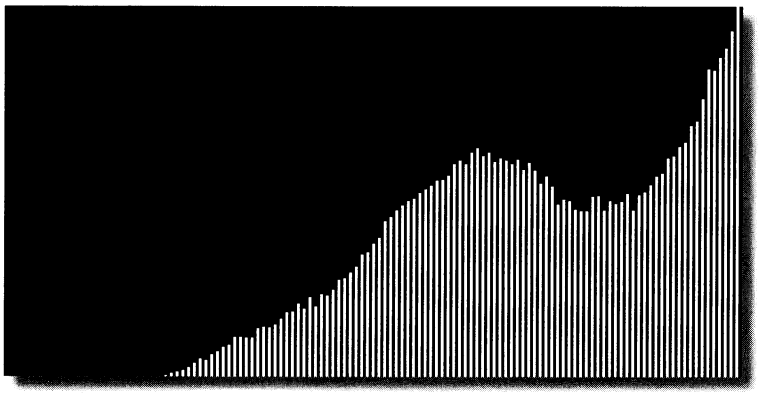


Figure 9.5 *Histogram of an image after a brightness of 2.0 has been applied. The histogram shows gaps due to an unequal distribution of colors in the image.*

Images that have been resampled from their original bit depth can also be detected via the use of a histogram. As mentioned earlier, if we create an 8-bit image from a 4-bit source image, we still really only have 4 bits of information present. Standard image-statistic tools will tell us that the image is an 8-bit image, but by examining a histogram of the image we would again see something like that shown in Figure 9.2. In this case, the regularly spaced gaps in the graph exist because the subtle color gradations that would be available in an 8-bit image were absent in the original 4-bit source. Although the difference between a 4-bit image and an 8-bit image is usually visible by viewing the color image directly, the difference between an 8-bit and a 16-bit image is much more subtle. Determining the true bit depth will require the use of a histogram or some other analysis tool.

CHAPTER TEN

Formats: Media, Resolution, and Aspect Ratios

Generally (we hope), when you set out to create a composite, you do so with the intention that it will be viewed via some specific medium. Whether you are providing images for broadcast video, CD-ROM, video games, feature film, or Web pages, you will need to have information about what the specific requirements are for that medium. In other words, you will need to know about the **format** that is being used.

Notice the usage of the term "format" here. It is another of those wonderfully ill-defined words that is extremely context dependent. In Chapter 2 (and Appendix C) we discussed digital image **file formats**—specific methods for storing digital images on a disk. This certainly may be part of the information required, assuming it is acceptable to deliver digital images directly. But more often the term "format" is used in the broader sense, referring to the specifics of the physical medium needed to capture or display the images being dealt with. Thus, film, video, 35mm, IMAX, VHS, Betacam, and so on would all be considered to be formats that we might use during the process of digital compositing. Some of the formats we just mentioned are subsets of others, which is part of the reason why it is so difficult to precisely determine what one is referring to when discussing the format. Care should be taken to ensure that all parties involved in the conversation are actually referring to the same topic.

In an attempt to organize the information presented in this chapter, we'll group the formats we discuss into three distinct categories: film, video, and other. Although this is an arbitrary classification system, there are certain basic traits for each category that make such a system worthwhile. This chapter will look at a variety of different formats within these categories, while keeping in mind that

the point of this discussion is to present an overview that covers, in as simple a fashion as is possible, the information that is most pertinent to digital compositing.

In an ideal world, we would be able to provide a simple table of different formats and the spatial resolution, color resolution, and aspect ratio for each of them. This is, unfortunately, an impossibility. First of all, there are so many different formats, and variations on these formats, that the task would be prohibitive. For any given format we often find that there is not really a definitive standard, but rather (at best) a group of standards, any of which you can choose and still claim to be following "the standard." This trend seems to be particularly prevalent in the video world.

Second, most of these formats represent images as **analog** data, and digitization methods will vary based on the hardware used for the conversion. Much of this hardware is manufacturer specific and is not subject to any particular standardization whatsoever. In other words, a piece of hardware can be built to sample an analog image at just about *any* resolution, and thus it is impossible to assign a specific resolution to a specific analog format. Only by knowing the characteristics of the hardware being used in your particular situation can you know with certainty the resolution of the format in question.

Due to the issues mentioned, we will tend not to give specific numerical values for the formats discussed, except as an aid to illustrating a concept or when a reasonably clear, simple standard exists. Appendix D has also been provided as an additional format reference. It attempts to diagram some common formats that are used when working with film and video, and gives a bit more detail about typical digital representations of these formats. Much of this information is highly susceptible to changing technology, which is part of the reason that it was placed in an appendix rather than in the main body of this book.

Ultimately, this chapter will talk about formats on several different levels, some analog and some digital. We will discuss the pipeline of imagery as it moves from the original recording medium through the digital realm (where we perform our compositing work) and back out again. Of course our primary concern (since this is a book about digital compositing, not about the myriad of different moving-image systems that exist in the world today) is how these sequences are used in the digital realm.

There are a multitude of different film formats and (depending on how you choose to define "format") probably an even greater number of video formats. Most of these formats are analog, but even when dealing with a source that is considered to be digital (such as D1 video), you'll find that the methods used to represent the digital data will be quite different from what we use to represent a frame within a computer. Fortunately, once an image has been taken from its original medium and format and converted into a digitized image that we are ready to composite with, the terms for specifying its format become much easier.

In fact, we usually only need to identify two specific parameters when detailing a digital image's format: the resolution (spatial, color, and temporal) and the **aspect ratio**.

ASPECT RATIO

We have already (in Chapter 2) discussed the concept of an image's resolution, which is simply the number of columns and rows of pixels in that image, the amount of data dedicated to color information for each of those pixels, and the number of these frames that are captured for a given period of time. The aspect ratio of an image provides additional information about the *proportions* of an image. This information can, in some situations, be even more important than the specific pixel resolution of the image.

The aspect ratio of an image is generally considered to be the width-to-height ratio of that image.[1] Note that when one is speaking of a *ratio*, the term is unitless. It will have the same value no matter what specific units of measurement are used. Consider the image shown in Figure 10.1. The aspect ratio of this image is 2:1. (When speaking, we say "two-to-one.") As mentioned, it doesn't matter what units we use to measure this image when determining its aspect ratio. If you get a ruler, you will find that the width of the image is approximately four inches and the height is half of that, or two inches. We would be correct in saying that

Figure 10.1 *Image with 2:1 aspect ratio.*

[1] But, of course, you can also find instances in which the aspect ratio is given as a height-to-width ratio instead. We'll consistently use width-to-height in this book, but don't assume that everyone else follows this convention.

the aspect ratio is 4:2, although in general we usually simplify everything by dividing both numbers by the smallest number so that we have a ratio that is relative to 1.[2] Divide both numbers in 4:2 by 2 to get 2:1. If we had measured the rectangle in centimeters instead of inches, we'd have an aspect ratio of 10:5, which again simplifies to 2:1. Usually, if we've already simplified the aspect ratio so that it is relative to 1, we don't bother including the ":1." Thus, a 1.85:1 aspect ratio will usually be specified as just 1.85.

If an image is already in digital form, then instead of measuring the sides with a ruler, we can simply divide the width of the image in pixels by the height, in pixels, to determine the aspect ratio. An image that is 2048 × 1024 will thus also have an aspect ratio of 2:1.

The aspect ratio of an image may seem like a redundant piece of information. After all, if we already know the width and height of our digital image, why bother to divide these to get the aspect ratio? The reason is that our digital images are usually just intermediate steps in the process of producing final imagery. That final imagery may not necessarily be formatted the same way that our intermediate images are. It may not need to be the same resolution as the intermediate resolution with which we've chosen to work. It may not even be a digital format, and thus the concept of a specific pixel resolution would be meaningless. More important, that final imagery may be intended for display via a method that distorts or modifies the width and height ratio of the source image. Consequently we need a unitless, media-independent method of discussing formats, and the aspect ratio will help to provide that.

Nonsquare Pixels

Although we've just finished telling you how to determine the aspect ratio of an image by dividing height into width, we will now look at some situations in which this may not give you the true aspect ratio of the format that you are using. This situation arises when your output medium does not display images in the same fashion as your intermediate compositing system displays them. More specifically, we are concerned with display systems that modify the width-to-height relationship of the images that we send to them.

When we give the aspect ratio of an image, we usually try to refer to the aspect ratio of the image as it will eventually be viewed. Both film and video have formats and situations in which the aspect ratio of the displayed image is different

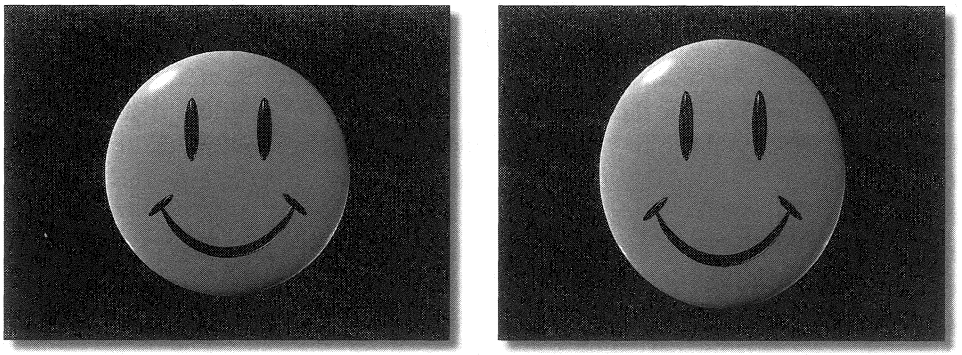
---

[2] Once again, there are always exceptions. The aspect ratios for various video devices are more commonly given as whole-number ratios. Thus, the standard NTSC television aspect ratio of 1.33:1 is often referred to as 4:3, and the new HDTV specification calls for an aspect ratio of 16:9.

from the original digital image. We often refer to such systems as having **nonsquare pixels**. In this situation, "nonsquare" indicates that the width and height of the pixel are not equal. Its shape is rectangular rather than square. (Actually, it's probably *oval* rather than *round*, but nobody ever uses the term "nonround.")

This width-to-height relationship of a nonsquare pixel can also be represented as an aspect ratio. To distinguish it from the aspect ratio for the entire image, we call it the **pixel aspect ratio**.[3] On a normal computer's monitor (assuming it is adjusted properly), the width of every pixel is the same as its height, and we say that it has square pixels. Its pixel aspect ratio is 1:1.

Do not confuse the pixel aspect ratio with an image's aspect ratio. The aspect ratio of an image can be (and usually is) a completely different number than the pixel aspect ratio of the pixels that make it up. For instance, the generally accepted standard for digital video images (in the United States) is a pixel aspect ratio of 10:11. The pixels are slightly taller than they are wide. If the individual pixels are stretched, then obviously the entire image will be stretched as well. Thus, if we were to digitally create an image that has a perfectly round circle in the center of it and then send this image directly to D1 video without any additional modification, the circle would appear as an ellipse. For example, Figure 10.2a shows an original image created within the computer. It is perfectly round. Figure 10.2b shows what this image would look like if it were viewed on a video monitor. It is, in fact, stretched by 11/10ths vertically. Depending on the software and hard-



(a) (b)

Figure 10.2 *Pixel aspect ratio. (a) Original image. (b) Image as viewed on a D1 digital video monitor with a pixel aspect ratio of 10:11.*

[3] Just as was the case with an image's aspect ratio, you will occasionally run across documents that give the pixel aspect ratio as a height-to-width ratio rather than width-to-height.

ware that is being used, you may need to manually scale your image before sending it to video in order to compensate for this discrepancy. Many systems will automate this step, however, so be sure you know how your particular setup behaves.

Some of the same situations can occur if we are working with film formats that involve optically distorting an image. Since images sent to film will no longer have identifiable pixels, instead of referring to these formats as having nonsquare pixels, we usually just call them **squeezed** formats. Formats that represent images with distorted aspect ratios are also commonly referred to as **anamorphic** formats. "Anamorphic" is a term borrowed from the art world that literally describes a distorted image that can be restored to its original shape. Historically, it involved an artist painting a wildly distorted image that would only look correct when viewed as a reflection in a specially curved mirror. In a later section of this chapter, we'll go into greater detail about some of the problems associated with anamorphic images when compositing them digitally.

DECIDING ON A RESOLUTION WHEN YOU HAVE AN ASPECT RATIO

Depending on your specific needs, the format of an image may be specified via resolution, aspect ratio, or both. If someone specifies that the images you should produce for a particular video game should have a resolution of 320 × 180 pixels, then you have enough information to do the work. But if you are going to be producing images for film, the only information that the client may give you is the aspect ratio of the images he or she desires. The reason for the lack of a specific resolution again has to do with the fact that film is a nondigital format, and as such it can be sampled, or digitized, at whatever resolution our hardware allows. The client may have little or no interest in the specific resolution, as long as the resulting image is acceptably sharp and has not suffered from any distortion of its proportions.

For instance, you may be told that you need to produce images with a 1.85 aspect ratio. It is then time to make a decision about the specific resolution that is necessary. This decision may be based on a number of factors, including practical considerations such as available disk space and processing power. Working at a resolution of 2048 × 1107 will produce an image with a 1.85 aspect ratio, but so will an image with a resolution of 185 × 100. However, if we were to transfer our 185 × 100 image back to film, we would probably be unhappy with the quality of the results, since we started with such a low resolution. It is almost as much of a problem if you decide to work at too *high* a resolution, since you will be confronted with unacceptable storage and computational requirements. Of course the decision about what resolution to use for a given aspect ratio is most often determined (or at least influenced) by the hardware that will be used to

scan or output the images, since such devices typically have certain fixed resolutions that they use for the various film sizes available.

As we've said, once an image sequence is converted to a digital form, the format is easily understood. But the process of doing the conversion, and the decisions that need to be made during that conversion, bear further discussion.

FORMAT CONVERSION PIPELINE

Let's look at the various stages of the compositing process where an image's format is important. The flowchart in Figure 10.3 gives a basic pipeline that will occur for every image sequence that is to be used in a composite.

Step 1 is where the source imagery that we will be using in our composite is first created. These may be images generated entirely within the computer, such as rendered 3D imagery coming from a specialized animation package, or they may be images shot on film or video.

What happens next is determined by whether the source images are already in a digital format or if they were created via an analog method. If the source material is analog—a piece of film for instance—then we proceed to step 2. An analog-to-digital conversion occurs, and a specific digital format (a resolution and a bit depth) is chosen at that time. This step is the first situation in which we need to start thinking about image formats and aspect ratios. Video images may or may not already be in a digital format, depending on the hardware available, but either way we will probably need to do some sort of conversion due to aspect-ratio as well as resolution issues.

If the source images are already digital (having been created by a 3D animation system, perhaps), there may be no need for any further conversion. However, for whatever reason, we may still want to alter the format of this imagery—possibly to produce a new, lower resolution that will make better use of our available disk resources. In some situations, we may even need to go through an additional format conversion for images that were originally digitized from an analog source. This is potentially an undesirable thing, simply because it would have been more efficient to choose the proper format for the original analog-to-digital conversion instead of wasting the time it takes for a second conversion.

Other chapters of this book will provide slightly more information about step 4, so we won't discuss it here other than to note that it is certainly conceivable that not all the image sequences that are being used in our composite will be in the same format. This possibility will be discussed further in a later section.

Steps 5 and 6 reverse the processes from steps 2 and 3. The digital images are, if necessary, converted to yet another digital format—one specific to the eventual output medium. This step is the least likely to be necessary, since one will usually choose to work in a digital format that is the same as the desired output format. Finally, in step 6 (assuming our final images need to be in some analog format)

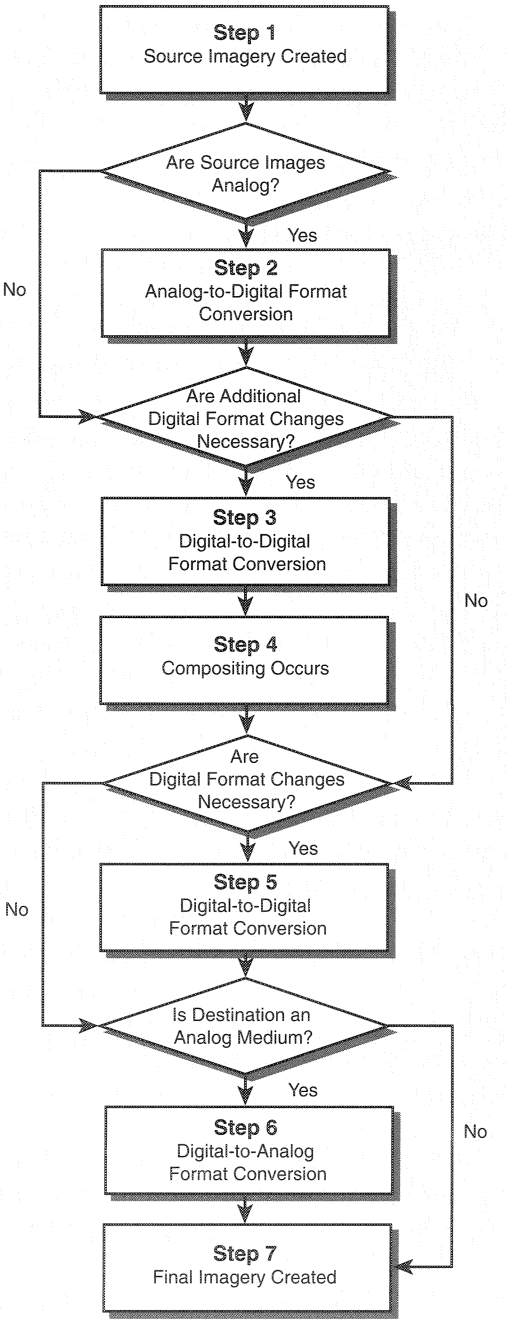


Figure 10.3 *The format conversion pipeline.*

the digital-to-analog conversion takes place. This new analog format may not necessarily be the same form as the original images.

We've gone through a scenario in which a number of different format changes occur. Remember that every digital-to-digital format change generally entails resampling the data and consequently will result in a slight loss of image quality. Avoid all unnecessary format changes wherever possible so that you will not suffer from this quality degradation.

The process of compositing is certainly much simpler if everything is created digitally and the intended final medium is also digital. None of the analog conversions are necessary, and a standard working resolution can be decided upon and maintained. Conceivably there will be *no* format changes necessary at all. The more complicated scenario occurs when our source and/or destination material is an analog medium, such as film. Not only do we need to deal with the analog-digital-analog conversions, but (particularly in the case of film) we will also probably need to be aware of a myriad of different physical formats that are in use throughout the industry.

Digital compositing is not the only discipline that needs to deal with format conversions. Conforming a feature film so that it can be viewed on video is a common example, one that can be particularly cumbersome if the original film footage was shot in some widescreen format.

The next section gives an example that follows the pipeline we've just detailed. Although it primarily uses a scenario based on film, it is suggested that you read through it even if you do not plan to do any film work yourself, since it touches on many format issues that are important no matter what the medium involved.

A Format Conversion Example

To get a better understanding of the format-conversion process and typical decisions that need to be made, let's run through the pipeline from Figure 10.3 while considering a piece of film as our source material.

The first question that one might ask about this film, particularly if one is dealing with the process of scanning the film into the computer, concerns the *format* of the film. Once again we must deal with the imprecision of the term "format." We already know that it was shot in a film format, as opposed to video. Let's also say that it was shot in a 35mm film format, the standard for feature-film work. Finally, to be as precise as possible, we find out that the film was shot in a 1.85 aspect ratio 35mm film format. We now have the information that we need for deciding how this film will be digitized.

Notice that this particular format, which would usually just be referred to as "1.85 format" (pronounced "one-eight-five format"), gives us a specific aspect ratio for our images. Not all formats have the aspect ratio so conveniently mentioned by

name, and you would instead need to consult a chart such as that shown in Appendix D. The aspect ratio is mentioned for this particular format because the camera actually captures more image than is necessary—the extra image will simply be masked or blocked when the print is eventually projected for the screen. Specifying the format (and thus the aspect ratio) lets everyone know that the shot was filmed with a specific framing and that the extra image will not be used. Consequently, when we go to scan this piece of film, we can choose to scan only the area that will be shown in the theater. For the sake of this example, we will do just that, although in the real world we would probably decide to scan the entire frame of the negative just in case we need to use some of the extra image.

Figure 10.4 shows a frame of the film for our shot. You can see the full image that has been captured, as well as a box that is drawn to indicate the standard framing for a 1.85 format. If you measure the width of this box and divide it by the height, you will come up with a ratio of 1.85:1, as expected. The usefulness of the aspect ratio should be starting to become apparent. Film, like any analog source, has no inherent resolution in the purely digital sense.[4] You can subdivide

Figure 10.4 *Film with 1.85 format framing.*

[4] There are, of course, ways of measuring the "resolution" of a piece of film, but this quality is distinct from a digital resolution. A print or negative's resolution has to do with the fineness of the grain particles that are in the emulsion.

it as finely as you wish and you will always end up with new unique values. It is only when we sample it into discrete packets (i.e., pixels) that it is useful to define it in terms of its resolution. In other words, the digital resolution is determined only when we actually scan the film into the computer. Depending on the hardware we use, we can sample it at a fairly low resolution (such as that used when a telecine operation occurs), or we can sample at very high resolutions—4000 lines or more.

Let's say that we choose to sample the frame so that we end up with a resolution of 3656 × 1976. As we've indicated, this resolution is somewhat arbitrary, subject primarily to the abilities and limits of the available technology. You will notice, however, that the aspect ratio of our new digital image is the same as that of our original film image, 1.85:1. We could just as easily have chosen to scan at 1828 × 988, which would have given us a lower resolution but still kept the same aspect ratio.

Let's assume a worst-case scenario now: For whatever reason, someone decides that our resolution of 3656 × 1976 is not what we should use for the bulk of our work. Perhaps someone sat down with a calculator and realized that the amount of disk space that is available won't be enough to store our entire shot at this resolution. (A 3656 × 1976 image stored in the Cineon file format, for example, will take up approximately 29 megabytes of disk space *per frame*. As you can see, high-resolution film images can have significant storage requirements.) Consequently, the decision is made to convert all our images to a lower resolution. Let's try something more reasonable—something about 2000 pixels wide. We'll be conventional and use 2048 pixels. There's nothing particularly magical about using 2048 instead of simply 2000, unless you happen to have a compositing system that prefers such a number (yes, there are some that do), but it's somewhat of a convention to use computer-friendly numbers like 2048. Given our 2K width, and since we know that we need to maintain our 1.85 aspect ratio, we do a little math, divide 2048 by 1.85, and come up with a new resolution of 2048 × 1107. This gives us a friendlier image size as well, about 9 megabytes per frame. We've just gone through step 3.

We could just as easily (at least on paper!) have gone back to step 2 and rescanned the film instead, going directly to our 2048 × 1107 resolution. Whether one would choose to do so would probably be decided by real-world considerations such as the accessibility of the scanner or the time available. In this particular situation, one would almost certainly just reduce the already-digitized images, since it would be faster and the image quality would be comparable.

Now that we have our images converted to the digital format that we desire, we can spend time working on the actual composite, producing a whole new sequence of images. These new images will be the same format as the images that are being used to create them, 2048 × 1107.

Eventually it will be time to get our completed digital composite *back* to film. In order to make sure that our example covers as many scenarios as possible,

let's say that the company that we were planning to use for our **film recording** (the process of getting digital images back onto film) has announced that it is getting out of the business and won't be able to help us. So we call up a different company, only to be told that they don't like to have images at 2048 × 1107 for creating 1.85 format 35mm film. They ask if we can convert the images to a resolution of 1828 × 988 before we send them over. We wish we had known about this sooner, since it means that not only are we going to spend time resizing a whole bunch of images, but also that we could have probably just done the entire composite at this lower resolution since we're dropping down to that resolution now anyway.

We go ahead and convert the images to the new resolution, which corresponds to step 5 in our pipeline, write them to a digital tape, and send them over to the film recording company. This company now feeds the images to their film recording hardware, which converts the digital images back to visible light, exposing them onto a piece of film negative. The image is properly placed within the film frame, and, if we care to take the time, we can go back and measure the shape of the resulting image to confirm that the width-to-height ratio is still 1.85 to 1.

Now that we have examined a specific example of how imagery may go through a variety of format changes during the compositing process, we'll spend a few sections getting an overview of some of the more common formats that a compositor may encounter.

FILM FORMATS

Although over the course of the past 100 years there have been numerous different film formats available, we will try to narrow this list down to a few of the most common. This section will not, as a rule, give detailed measurements for the formats—this is done for a few specific formats in Appendix D—but rather will attempt to give the reader a sense of how the different film formats are dealt with, the relative quality of the different formats, and any idiosyncrasies inherent to each format.

Most film systems today are designed to project images at a rate of 24 frames per second. There are a few exceptions to this, which we will describe on a case-by-case basis. The color resolution of a piece of film is, as with the spatial resolution, somewhat dependent on the hardware used to digitize the imagery. However, due to the large dynamic range of film, most systems are designed to scan at more than 8 bits per channel. The most common standard for scanning film imagery uses Kodak's Cineon specification, which utilizes a 10-bit nonlinear color space, as described in Appendix C.

Any given piece of film, as we've mentioned, can be scanned at a variety of different spatial resolutions, but these resolutions are at least partially determined by the physical size of the film that is being scanned.

The standard way to specify the size of a piece of film is by measuring its width in millimeters. This width is also known as the **gauge** of the film. By far the most common gauge for feature-film work is 35 millimeter, so we'll start our discussion with a look at this standard.

35mm Formats

Not only is 35mm the most common film format used for theatrical release, it is also, not surprisingly, the most common film format used for digital compositing work. As you will see, 35mm is not really a single, well-defined format. In fact, there are a number of different formats that fall under the general category of "35mm," and it is important to realize that the specific gauge of the film itself does not completely determine the size and shape of the image that is actually captured on that film.

The basic 35mm frame is shown in Figure 10.5. Every frame of standard 35mm film is bordered on each side by four of the perforations used to move the film through the camera. These perforations are often called "perfs," and consequently the 35mm format is sometimes referred to as **four-perf**. Along the left side of the film, between the perfs and the image itself, is a narrow area reserved for the soundtrack of the film. This is known as the sound stripe.[5]

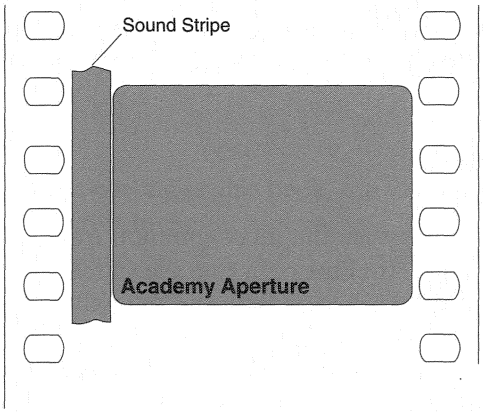


Figure 10.5 *Basic 35mm frame showing Academy framing.*

[5] The standard sound stripe is designed to store an *analog* soundtrack. To maintain backward-compatibility with the huge number of older projection systems in the world, the newer digital soundtrack methods were forced to find some other place to locate the sound information. There are a number of different, competing standards—some use the area between the perforations, while others use the margin of the film outside of the perfs. This particular bit of information has virtually nothing to do with digital compositing, and is presented solely for trivia's sake.

The area of the negative that is usually exposed when shooting standard 35mm feature films is indicated by the light gray rectangle in the diagram. Note that this framing, which excludes the sound stripe, has an aspect ratio of 1.37—it is generally referred to as the **Academy aperture**.

Imagery that is exposed with this framing will not necessarily be projected with the same framing. Instead, it will generally be masked for theatrical presentation to conform to one of the more popular projection standards. Since the Academy aperture framing is the most widely agreed-upon standard for 35mm work, we will try to point out how some of the different film and video sizes compare with the Academy frame in terms of absolute resolution. Appendix D also contains a chart showing this resolution comparison for a variety of formats.

Although the sound stripe area will be filled when the film is eventually projected, it is not something that necessarily needs to be protected when shooting the original negative. It is not uncommon, therefore, to remove the mask that blocks this area from exposure in order to capture a larger image. There is also some space above and below the Academy framing that can be used as well. Taking advantage of this extra area, which is called the **full aperture** (sometimes also referred to as the **camera aperture**), will result in a frame with an aspect ratio of 1.33. Simply by using this larger area, one is able to capture an image that is about 30% larger than normal Academy aperture. Certain formats make use of this extra area as an intermediate step before the final print is made, and it is not uncommon for visual effects elements to be shot "full-ap" to reap the benefits of a higher-resolution image. An example of the full aperture framing is shown in Figure D.1 of Appendix D, as well as for the rest of the 35mm formats we will be discussing.

1.85

An aspect ratio of 1.85 is by far the most common framing for movies that are shown in the United States. (In Europe, a 1.66 aspect ratio is much more common, and in the rest of the world it varies between the two standards.) Projecting an image that was shot with normal Academy aperture in a 1.85 format is simply a matter of placing a mask in the projector that blocks a small amount of the image—usually along the top. The cinematographer has planned for this framing from the outset, and thus will not have allowed anything important to occur in the top of the frame. A diagram of 1.85 framing was shown in Figure 10.4.

Super 35

The Super 35 format ignores the space normally reserved for the sound stripe and uses most of the full-aperture area of the negative to capture the desired

picture. This picture will eventually be projected with an aspect ratio of 2.35, but only after some intermediate steps.[6] These steps are used to reformat the captured image into a different standard, usually Cinemascope (which we'll discuss in a moment), and consequently Super 35 should be considered primarily a format for acquisition, not projection.

Cinemascope

The **Cinemascope**[7] format (also known as simply **C-scope**) is an anamorphic format that has been designed to capture images that are squeezed by 50% along the *X*-axis. This squeezing is done in order to capture a wider field of view while still using as much negative as possible. We will be using the Cinemascope format later in this chapter to illustrate some of the concepts related to working with nonsquare pixels, and will discuss the process it uses to capture this squeezed imagery in more detail in that section. Cinemascope images are intended to be projected so as to produce an image with an aspect ratio of 2.35. However, due to the two-to-one squeeze of the captured images, the aspect ratio of a Cinemascope image on a piece of film is approximately half of that, about 1.2. The reason it is not *exactly* half has to do with the masking applied to the projected image to force it into a 2.35 aspect ratio.

There is another 35mm format that we will discuss in a moment, known as the **Vistavision** format. Due to the particular characteristics of this format, we have chosen to cover it in the "Specialized Film Formats" section instead, for reasons that will become obvious.

16mm Formats

Generally, at least as of the writing of this book, the various standard 16mm film formats are not used very often in digital compositing. This is probably due mostly to cost issues: If a filmmaker has the money to digitize and manipulate film footage, he or she is probably able to afford shooting in a 35mm format. Only time will tell if the migration of digital tools will get to the 16mm market before it is supplanted by standard or high-definition television as a platform for lower-end work.

---

[6] Technically, "Super 35" can refer to a variety of formats that use the full aperture for their exposure, but in practice, Super 35 is used almost exclusively to produce images that are intended for projection in a 2.35 aspect ratio.

[7] The term "Cinemascope" is actually the name of a specific widescreen anamorphic format that 20th Century Fox developed in the 1950s. However, it seems to have become a generic term for the 2.35 anamorphic format that is now primarily created using **Panavision** lenses.

16mm film *is* occasionally enlarged to 35mm for theatrical release, usually when a studio decides that a low-budget film is worth giving a wide distribution. In situations such as this, it is certainly possible that digital techniques might be employed to fix specific problems that may be unsolvable by other means, and it is conceivable that original 16mm negative will be scanned for compositing work. A more common use for 16mm film is as a source for television commercials. In this situation, however, the images are generally transferred directly to video via a telecine device. Thus, the resulting resolution and aspect ratio of the images produced will depend primarily on the video equipment you are using. Figure D.2 in Appendix D shows the size of the basic 16mm frame, relative to the other formats. It also includes a box indicating the size of a *Super 16* frame. The Super 16 format extends the frame into the standard 16mm sound stripe area, resulting in a larger image. Both of these formats are still quite a bit smaller than even the standard 35mm Academy framing, and will only be able to capture about 25% of the resolution.

Specialized Film Formats

There are a number of less common film formats that are usually reserved for special situations. These formats are generally designed to produce much higher-quality imagery via the use of a larger exposed negative. Large-format films require the use of cameras that are much bulkier, more expensive, and generally less reliable than standard 35mm cameras, and as such their use is limited. These larger formats will sometimes be used as intermediate steps in the production of a standard 35mm print, or they may be part of a "special venue" presentation that is capable of directly displaying large-format films.

Vistavision

Vistavision (or VistaVision) was originally developed by Paramount studios in the 1950s as a proposed new widescreen format. As mentioned earlier, the process actually makes use of standard 35mm film stock to capture imagery. However, in order to increase the resolution of the frames that are captured, specialized cameras that run the film *sideways* through the camera (instead of vertically) are used.[8] The 35mm frame now limits the *height* of the image instead of the *width*, and consequently the camera can expose an area of the negative that is more than twice as much as that exposed with standard 35mm film photography.

---

[8] The Vistavision negative is actually formatted in the same way that a 35mm *still* camera exposes its film.

Although it never really caught on as a mass-distribution format, Vistavision was used for a time as a method for capturing high-quality negatives that were then optically reduced to standard 35mm prints. Even this usage eventually became less and less common, and after the early 1960s Vistavision was effectively obsolete as a mass-market system. However, the lure of a larger negative was enough to inspire visual effects people to start using the format again, and consequently it is now used quite often as a method for capturing elements for visual effects compositing work, be it optical or digital. Figure D.1 in Appendix D shows a Vistavision frame. Compared with the standard 35mm Academy aperture, a Vistavision negative exposes about 2.7 times more area. The Vistavision format is often referred to as **eight-perf** because the area captured by a single frame spans eight film perforations.

65mm

A wide-gauge film that is run vertically through a special camera, 65mm is used not only for visual effects work but also occasionally as a source for films that will be presented in 70mm.[9] 65mm is also widely used as a format for a number of different special-venue situations. One of the most common of these is the **Showscan** method, which captures images at a rate of 60 fps instead of the normal 24, resulting in a dramatically improved image in which grain is nearly undetectable and flicker, or strobing, is virtually eliminated. Since a single frame spans five perforations of the 65mm negative, it is also known as "five-perf." A five-perf negative will capture an image that is about 3.6 times larger than the standard Academy aperture.

IMAX

Another system that uses a 65mm negative to capture its image is one created by the Imax Corporation. It runs the film horizontally through the camera and projector in the same fashion as Vistavision, producing an extremely large negative. Each frame spans 15 perforations of the 65mm film, creating a negative that is more than ten times larger than a standard 35mm Academy exposure. A size comparison can be seen in Figure D.2 in Appendix D. Although this format is

[9] You may see your local movie theater advertising a film as being "shown in 70mm." This is a specification for the *print* being shown, not the size of the original negative that was used when shooting the footage. Ideally a 70mm print was originally shot with some large-format negative, but there are no guarantees that this is the case. These days it is much more common that the original footage was shot with a standard 35mm format and then optically enlarged to fit onto a 70mm print. This results in a brighter, less grainy print, although it will still have less definition than something that was shot with a large-gauge negative.

most commonly used when capturing imagery for IMAX productions, the basic 65mm, 15-perf format is not really "the IMAX Format." Rather, IMAX is a complete system that includes image capture on 65mm, 15-perf negative, projection from 70mm, 15-perf prints, and carefully calibrated projectors, screens, and sound systems. Strictly speaking, one should only refer to "65mm, 15-perf" when describing this format, but colloquially it is simply called "IMAX."

Digitizing some of these special formats via a high-resolution scanner will result in images that require an extreme amount of data for every frame of the image sequence. This problem may be even further compounded by the fact that some of these formats are also used to capture stereoscopic imagery by using two cameras to photograph a scene. This stereo imagery essentially doubles the amount of information that is created for every frame.

VIDEO FORMATS

Video is, if anything, an even more ill-defined category than film, since there are an incredible number of different methods for representing, displaying, and storing video information. We will attempt to define some broad categories, as well as give some idea of the variability within those categories, without delving into an inordinate amount of detail about how video works. Again, there is a bit more information given in Appendix D for those who are interested.

Unlike film, where one can easily grasp exactly how the image data is stored by simply holding the medium up to a light source, video is a fairly complicated subject. Before we can even begin to discuss specific video formats, we need to define a number of additional concepts.

The important thing to remember about video is that even though it is an electronic format, it is still primarily (or at least historically) an *analog* electronic format. Thus, just as we saw with film, you will find that it is really the digitization hardware that will define the resolution you are working with, and this hardware can be very arbitrary in the resulting resolution it chooses for a particular video signal. There is slightly less variability in the possible resolutions than there was with film, in that the electronic video signal is only capable of representing a limited, fairly well-defined bandwidth. As such, there is an upper limit on how dense a sampling it makes sense to use. We will briefly describe, in as simplified a fashion as possible, an analog video signal so that these resolution limits can be better understood.

Lines of Resolution

Analog video can be thought of as a continuously variable signal that is used to build an image line by line. Since the line created by this signal varies continuously

(providing color and brightness information), it is a true analog signal, with no inherent number of pixels associated with it. However, the number of these lines (known as **scan lines**) that makes up the full image remains fixed for any given format. In the case of the NTSC format, the standard in the United States, there are 525 lines in a single video frame. At this point, one would be tempted to assume that a digitized video frame should also have a vertical resolution of 525 pixels. One would be wrong. In fact, these 525 lines do not all contain image information. A good portion of them are used for other purposes, including things like closed captioning information and the time code. The portion of the video signal that *is* used for image information is known as the **active region**, and this is what we are usually interested in. This topic is discussed a bit more in Appendix D.

Ultimately, from a digital compositing artist's perspective, the video specification for the number of lines in an image is primarily important as a means to compare resolutions between other formats that use a similar signal.

Fields

Although we've just finished describing how a frame of video is composed of a number of scan lines, in actuality even this is a simplification. This is because those scan lines are not simply displayed sequentially, top to bottom, to produce an image. Instead, every *other* scan line is displayed first, creating an image that is the proper height, but with only half as many scan lines. Immediately after this image is displayed, the remaining scan lines are displayed, top to bottom, creating a second image composed of the alternate lines. In essence, we have two images displayed immediately after each other to produce a single frame. Each of these half-frames is known as a "field image," or simply a **field**. Each field is displayed for half the frame rate of the format with which we are dealing, so in the case of NTSC video (which runs at 30 frames per second), each field is displayed for 1/60th of a second. One of the primary advantages of showing 60 fields per second instead of 30 frames per second is that the resulting image has essentially no perceptible flicker.

Formats that use two fields to define a single frame are known as **interlaced** formats, and an image composed of two fields is known as an interlaced frame. Formats that do not use interlacing are known as **progressive scan** formats, since the scan lines are displayed progressively, without needing to double back to present a second field. Most computer monitors are progressive scan, and some of the HDTV formats we will discuss are progressive scan as well.

Most compositing systems are set up so that the user can ignore the interlaced nature of video. Typically, the hardware that converts the video stream into a digital RGB image will be able to automatically interlace the field images together

to produce a single normal frame. But there may occasionally be situations in which one wishes to deal with the distinct fields explicitly, and it will depend on the capabilities of your individual system whether or not this is possible.

Packages with this ability will allow the user to **deinterlace** a frame into two half-height fields and work on the images individually. This process will of course produce twice as many images. Usually one can choose whether or not to simply treat them as vertically squeezed anamorphic images or just temporarily resize them to full height. The advantage of working with field images instead of full frames has to do with the increased temporal resolution. Motion artifacts, particularly for elements that move quickly across the frame (horizontally), are significantly reduced.

Figure 10.6a shows two frames of an animated sequence in which a ball is moving across the screen, left to right. The same animation rendered to fields is shown in Figure 10.6b. Remember that even though the field images appear to be squashed in our digital image format, they will be full height when displayed

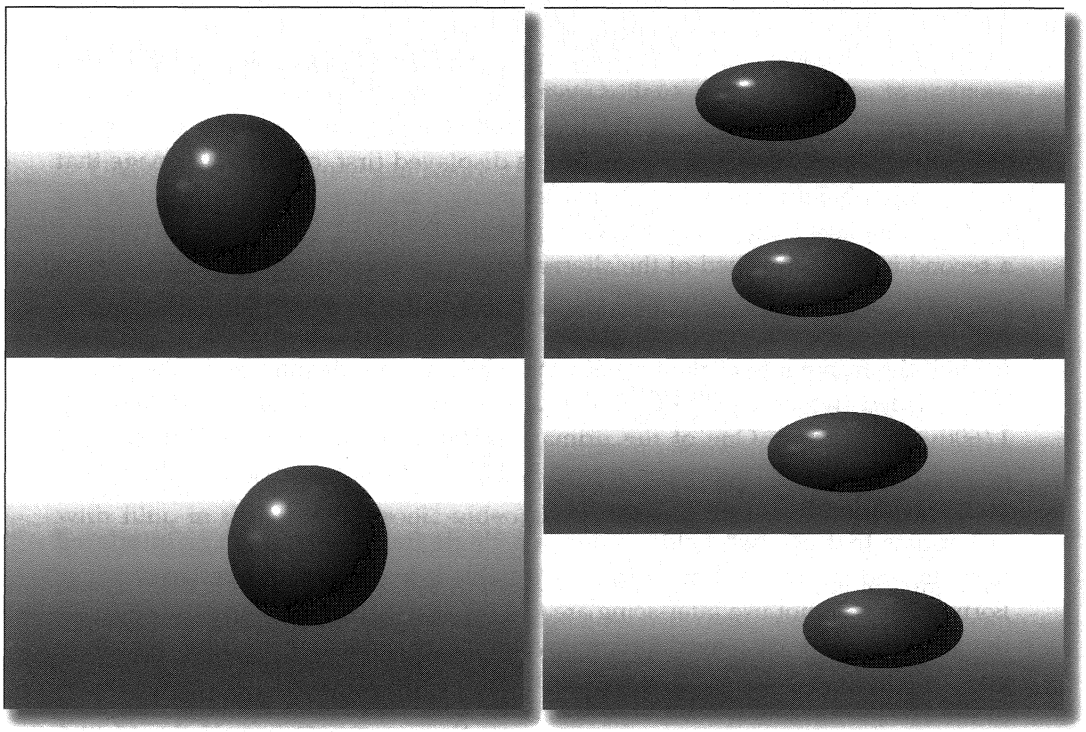


Figure 10.6 *Frames versus fields. (a) Two frames of an animated sequence. (b) The same animation represented as four field images.*

as a field image on a video monitor. As you can see, since we have twice as many images for a given time period, we can represent the movement more accurately. The result will be an animated element that has less flicker, or strobing, when viewed.

Once you have finished working with the field images, your system should allow you to reinterlace them to produce a new full-height frame. Figure 10.7 shows what these new frames would look like. This result might appear strange if you are looking at only a single frame, but remember that this is really an artificial way of viewing these images. They will appear perfectly correct when viewed at the proper speed on a normal video display system. In fact, if you were to examine a frame from video of a live-action ball thrown across the screen at the same speed, you would see exactly the same sort of interlacing characteristics.

Color Resolution

Video signals, particularly analog video signals, have historically been sampled at 8 bits per channel. This was primarily driven by the need to sample in real

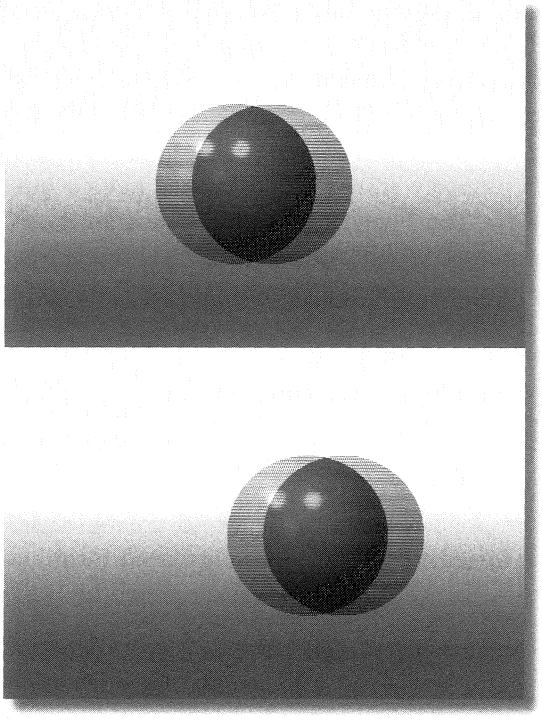


Figure 10.7 *Field images (from Figure 10.6b) interlaced to produce new frames.*

time, but the limited bandwidth and inherent noise in a video signal made this a reasonable bit depth to use. The standards for digital video have continued to evolve, and most specifications now provide the ability to sample at either 8 or 10 bits per channel. Working with video images that were digitized at 10 bits per channel is still fairly rare, but as the cost of 10-bit hardware drops, it will probably become more common.

Gamma

Anyone who has worked with video imagery from a technical perspective has probably been exposed to the concept of video's "built-in" gamma. The gamma correction that is inherent in a video image is actually a form of the nonlinear color space that we briefly mentioned in Chapter 2. It is primarily used as a way to maximize the available bit depth in an image, although it is also related to the way that a video display device (such as a television) converts a signal into an image. A complete discussion of this subject is far beyond the scope of this chapter, and you should consult the more detailed coverage that is given in Chapter 15 if you desire more information.

For the purposes of this section, it is sufficient to understand that standard video images include a gamma correction that is applied when the images are acquired. Typically this means that the video camera used to shoot a particular image applies this gamma correction before the image is written to tape. For all the different video formats we discuss (including HDTV), the standard gamma that is applied is 2.2.[10]

The fact that all properly stored video images have a built-in gamma of 2.2 means that if they are transferred without modification to a digital RGB image in a compositing system, they will appear to be too bright. You may need to apply a compensating gamma of 0.45 (1/2.2) to bring them back into the normal range. Don't do this blindly, however. Make sure that whatever hardware you are using to convert video signals into digital images isn't *automatically* applying the inverse gamma, which is becoming the common practice.

NTSC

Now that we've provided a bit of background on the basics of video, we can discuss the specific formats that are commonly used throughout the world. The most common of these is **NTSC**, the standard defined by the National Television

[10] Some of the standards documents for the PAL format actually specify a gamma of 2.8, although this generally seems to be ignored.

Systems Committee. It is the video standard in the United States, as well as in Canada, most of Central and South America, and a few Asian countries.

NTSC is designed to display images with 525 lines of resolution at 30 frames per second.[11] It is an interlaced format, so in reality it is displaying field images at the rate of 60 per second. With NTSC, the first field contains all the even-numbered scan lines, and the second field contains all the odd. This is referred to as the **field dominance** of the format. NTSC, as mentioned earlier, is a squeezed format, and a pixel that is converted to an NTSC image will have an aspect ratio of 10:11. The aspect ratio of the full NTSC frame is 4:3. See Appendix D for more information.

PAL

The **PAL** (Phase Alternation by Line) video format was developed by Germany and is used throughout Great Britain, parts of Europe, several African and Asian countries, and Australia. It is designed to carry images with 625 lines of resolution, displayed at a rate of 25 frames per second, or 50 fields per second. PAL is the opposite of NTSC in terms of field dominance, since field 1 contains all the odd-numbered scan lines and field 2 has the remaining even lines. The aspect ratio of a PAL image is also defined to be 4:3 (the same as NTSC), which, given the fact that the vertical resolution of the format is different, implies a different pixel aspect ratio. In this case, it turns out that any square pixels that are sent to a PAL monitor will have an aspect ratio of 59:54—they will be slightly squeezed vertically.

SECAM

SECAM is a less common format, developed by France and used in parts of Europe and Africa. Like PAL, SECAM is an interlaced system designed to run at 25 frames per second, carrying 625 lines of resolution. Unlike NTSC or PAL, SECAM is rarely used as a video storage format, or even as an intermediate format for editing. There is significantly less support for this format in terms of hardware, and generally most work in countries in which SECAM is the broadcast standard will use PAL as their primary format. The video signal will be converted to SECAM only when it is being broadcast.

HDTV

There is a fourth video "standard" emerging, known as high-definition television, or **HDTV**. Its primary distinguishing characteristic is the much higher resolution

[11] The official specification for NTSC playback defines the speed as 29.97 frames per second. Thirty fps is a fairly safe approximation for most purposes.

that it is capable of storing and displaying. The HDTV standard is actually yet another collection of standards, but fortunately HDTV is a much more computer-friendly format, since it is, by definition, a digital format. In theory, HDTV encompasses any video format with a resolution greater than standard video. Of course, as we've seen, "standard video" is a bit of a misnomer, so we've included a few specifics in Appendix D about what resolutions HDTV can encompass.

HDTV, being a collection of format descriptions, has provisions for either interlaced images or progressive scan. Most of the medium-resolution HDTV formats are progressive scan, but the high end of the specification also describes an interlaced HDTV format. HDTV is defined to be a square-pixel format, and has provisions for both 8 or 10 bits of color precision. It is considered to carry a built-in gamma of 2.2.

Of course, the fact that there are written specifications for a variety of HDTV formats is no guarantee that any of these particular resolutions will actually emerge as an *industry* standard. As of the writing of this book, worldwide HDTV broadcasts that are receivable by the general public are few, far between, and often incompatible. Only time will tell what true standard (or standards) will emerge.

OTHER FORMATS

Our final format category of "other" is, obviously, the least well defined. Certain media such as video games may have loosely defined standards concerning their working formats, but these are all as variable as the hardware on which they may run. The bottom line is that, for most of these "other" formats, there is no need for standard resolutions. Unlike film or video, these formats are all purely digital, and they are either flexible enough to be able to show images at just about *any* aspect ratio or resolution (as is the case with a standard PC) or are hardware-specific formats that are determined by the hardware's manufacturer. This is the case for video games, in which the game console itself will require the images to be in a very specific size, color depth, and file format. Most dedicated home-videogame consoles (such as those from Nintendo, Sega, or Sony) have a number of predefined resolutions that they support: 320 × 240, 640 × 240, and 640 × 480 are common, but there are quite a few variations.

Keep in mind that video games now tend to fall into the much broader category of **multimedia**, and even though the final images may be of limited resolution, the source material for this imagery can easily be from a higher-resolution format, perhaps even film. The color resolution of most video games, as well as most other multimedia applications, is, at best, eight bits per channel. More common is the use of indexed color in conjunction with a lower bit depth.

Understand that the reason for all these different formats, be they film, video, or something else (including the myriad of less common or obsolete formats that

haven't been mentioned) is primarily historical. A number of different image capture formats have been proposed over the years, some of which became popular, others which did not.

It may seem strange that some of the 35mm film formats mentioned in this chapter involve a scenario in which a significant portion of the frame is exposed, yet never used. There are a few reasons for this. First, having a standard frame that covers several cases means that we can use the same equipment no matter what the intended format. The only difference will be that the cinematographer will be framing his scenes with a specific aspect ratio in mind.

Capturing extra imagery is also useful when transferring a film to video. The taller aspect ratio of standard television (1.33) means that we need a different framing. We can either zoom in on the 1.85 framing and then use **pan and scan** techniques to try to keep the important action in the scene, or we can shoot the original footage while also planning for the eventual video release of the film. The difference between the 1.33 aspect ratio of television and the 1.37 of the Academy aperture is fairly small, and requires less extreme measures to get an acceptable video framing. Finally, as we've mentioned, there are a number of situations in which you may choose to shoot with a format that is larger than your intended delivery format in order to have extra image information with which to work.

No matter what format you are given to work with as a compositor, it is important to always keep in mind the issues of resolution and aspect ratio. When working with multiple formats, it is dangerously simple to convert between two different resolutions. If these two resolutions do not have the same aspect ratio, however, you will end up with an image that is either squeezed or stretched by a certain amount.

WORKING WITH NONSQUARE PIXELS

If you are working with formats that utilize nonsquare pixels, there are some special considerations that you will need to take into account. To illustrate some of these concepts, we'll use the most common anamorphic film format, the Cinemascope format. Everything we'll discuss applies equally to squeezed film or video, but we've chosen the Cinemascope format for our example because its two-to-one squeeze is the most dramatic and thus illustrates the concepts quite well.

C-scope footage is shot using a lens that compresses the image by one-half along the *X*-axis. If you examine a piece of film that was shot with a C-scope lens, you will see that everything in the frame appears to have been squeezed so that it is half as wide as normal. When it comes time to project this strange piece of film in a theater, another special lens (that makes everything appear twice as wide as normal) is used, and the image appears correct once again.

When planning to digitally composite images that were shot in this format, there are a few different routes one can take. Initially, your elements will probably have been scanned exactly as they appear on the negative. We say that they have been "scanned squeezed," and if you look at a frame of this on your computer it will indeed appear squeezed. Now, it is possible that you may choose simply to do your work directly on these squeezed images. When dealing with less extreme squeezes, such as video, this is a common scenario. (The caveat comes when it becomes necessary to rotate a squeezed element—we'll discuss this in a moment.) Working with squeezed C-scope elements can, however, be visually distracting, and properly judging movement is a skill that may take some time to develop. (Horizontally moving objects will appear to be moving half as fast as they normally would, yet vertically moving objects are going at the proper speed. Diagonally moving objects are particularly hard to judge intuitively.)

In an ideal world, the best way to deal with these squeezed C-scope images would be to immediately unsqueeze them—that is, scale them nonproportionally to restore their proper aspect ratio. In practice this method has drawbacks, in that you either must halve your *Y* resolution (thereby losing 50% of the information you just scanned) or must scale your image by two times in the *X* direction (thereby doubling the amount of data to deal with, even though you haven't really created any extra sharpness or detail in your image).

The best solution is to find a compositing system that is capable of *displaying* the images as if they were in an unsqueezed format, while still *processing* the data in a squeezed format. Such a system uses either software or hardware to let you view the images as if they have been unsqueezed for all the tests that you are doing, yet still keeps and processes the images at their original resolution when producing the final composite. A well-designed system will be able to do this with little or no additional processing overhead.

Incidentally, we often refer to systems with the ability to do this temporary unsqueeze as being able to display nonsquare pixels. Don't assume that this means that the individual pixels you see on your computer's monitor will become squeezed or stretched. If you were to take a magnifying glass and look at these pixels, each would still appear to be properly proportioned. It's just that the system has applied a simple geometric transformation to nonproportionally resize the image so that it *appears* the same way as it will in its final form. The underlying image data has not been modified; it is only the display of these images that goes through this temporary process.

Most image processing and compositing operations will work just fine when applied to squeezed images. Simply layering one squeezed element over another is no problem, and things like color corrections will behave identically whether or not they are applied to squeezed imagery. But certain processes can be very problematic and will require special attention. Consider the issue of rotation.

Figure 10.8a shows the squeezed-image representation of a circular object placed in a standard Cinemascope frame. In this case, the extreme squeeze of this format has produced an image that is *much* narrower than it is tall. If we immediately send this image back to film and view it with the proper lens on our projector, it will appear round again, as shown in Figure 10.8b.

(a)

(b)

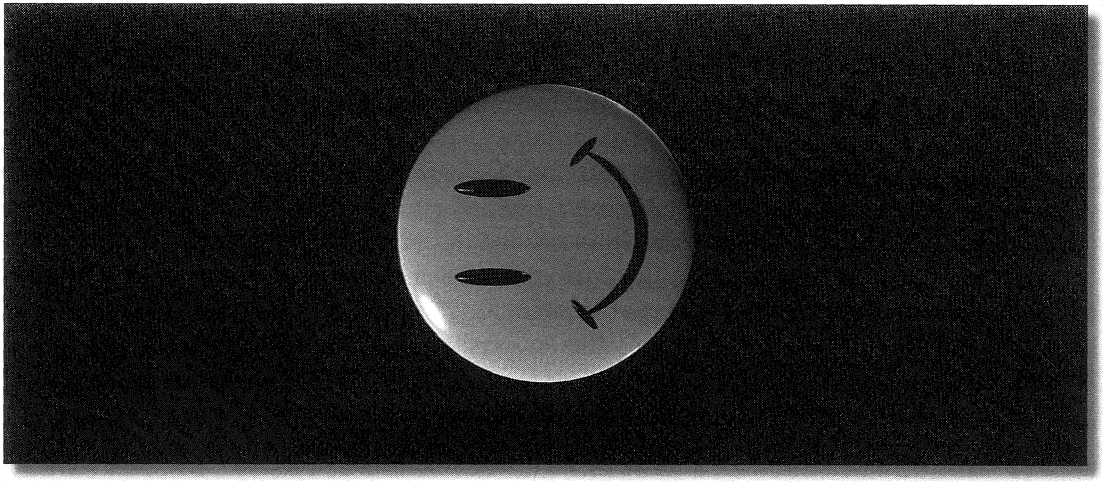
Figure 10.8 *Squeezed formats. (a) A circular object as viewed in a standard Cinemascope frame, producing an extreme squeeze. (b) The image as it will be projected.*

But let's say we wish to rotate this squeezed element by 90° before pasting it over a squeezed background. Figure 10.9a shows this rotated element, and 10.9b shows the result when we project it. We have suddenly generated an element that is drastically distorted, probably not the result we were looking for. To avoid

(a)

(b)

Figure 10.9 *(a) Circular, anamorphically squeezed object from Figure 10.8a rotated 90° without first unsqueezing it.(b) The object as it will be projected, showing unwanted distortion.*



(c)

Figure 10.9 *(c) The object as it should look when projected after a correctly performed rotation.*

this problem we would need to first unsqueeze the element, rotate it, and then resqueeze it along the X-axis. The result of this process would produce the image shown in Figure 10.9c.

Most spatial filters such as blurring will also need to be treated differently if you are working with squeezed elements. Thus, blurring a squeezed element by an equivalent amount in X and Y will result in an unsqueezed element that appears to have too much blur along one axis. If you have access to a blur algorithm that allows you to control both the vertical and the horizontal blur, you can adjust the relative amounts to compensate for any image squeeze.

The best compositing systems will usually be able to automate these tasks, allowing you to specify a working aspect ratio either for a specific operator (like rotation) or for the entire compositing session. The system will then automatically deal with the related issues, performing the unsqueeze/resqueeze process when rotating squeezed elements or transparently adjusting any other parameters for operations that require it.

Keep in mind that these issues can arise whether you are dealing with film *or* video. Because the amount of squeeze that is typical for video is much more subtle than for film, it is easier to overlook the fact that you are working with squeezed images. Many video systems automatically unsqueeze elements when transferring them from video to the computer and back, meaning that you will run into problems if you apply an additional unsqueeze. As always, be aware of the idiosyncrasies of your particular system.

COMBINING FORMATS

We've tried to give a very broad overview of the wide variety of formats that you may encounter. Depending on your particular situation, you may never need to deal with most of these formats. On the other hand, you may find yourself working on a composite in which you need to combine several different formats at the same time. Usually this doesn't present much of a problem, since the different images can just be scaled as needed. A mixture of squeezed and unsqueezed images will obviously require you to determine a working aspect ratio and then to conform everything to that standard. There may even be times when you wish to combine film and video in the same image. Chapter 13 discusses the process of creating elements and includes a section entitled "Choosing a Format," in which some of the factors that dictate format choices are presented in greater detail.

CONVERTING BETWEEN FILM AND VIDEO

Although this chapter gave a number of examples of how formats might be converted throughout the compositing process, there is a particular conversion that bears further scrutiny: the film-to-video conversion and its reverse, video to film. This conversion is problematic primarily because of the different frame rates that the formats in question use, particularly when converting between NTSC video at 30 fps and standard film at 24 fps.

In Chapter 6 we discussed some general-purpose methods for converting between different frame rates, but the specific conversion between film and video occurs often enough that a well-defined standard process has emerged. It involves taking a film frame and converting it to two fields of video, and then taking the next film frame and converting it to three fields of video. This alternating creation of two and then three fields in order to synchronize film timing to video (NTSC) timing is known as a **2:3 pulldown.**[12] A diagram of this process is shown in Figure 10.10.

Film frame A is used to create both fields of our first video frame. Film frame B is used to create both fields of video frame 2, and also the first field of video frame 3. Film frame C is then used to create the second field of video frame 3, and also the first field of video frame 4. Finally, film frame D is used to create the second field of video frame 4 as well as both fields of video frame 5. As you

[12] This is pronounced "two-three pulldown." You may also hear it referred to as 3:2 pulldown, which theoretically indicates that the first frame is converted to three fields and the second frame is converted to two. In practice the terms are usually used interchangeably.

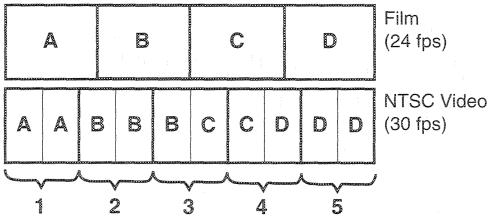


Figure 10.10 *The 2:3 pulldown conversion.*

can see, 4 film frames will make up exactly 5 video frames, and thus 24 film frames will make up 30 video frames.

Converting between film at 24 fps and PAL video (25 fps) can be done in one of two ways. The simplest method is to do a straight frame-for-frame transfer between the two, so that any given film frame corresponds directly to a certain video frame. This will obviously change the speed of any motion in the footage that is being transferred, but the speedup is only about 4% and is fairly subtle—subtle enough that the simplicity of the process makes it worthwhile. (Not to mention the fact that any movie that is transferred to video for television broadcast will be 4% shorter, giving more room for commercials!) But if a more accurate transfer is needed, a 25th video frame will need to be generated from the 24 film frames. Merely duplicating one of the frames would be visually noticeable, so instead two additional video *fields* are created, one from the 12th film frame and the other from the 24th.

These frame-rate conversions are usually accomplished by special **telecine** hardware that deals with both the digitization of the film *and* the pulldown. The process of going from video to film is much less common, but would follow the same logic for the speed change, in this case deinterlacing video fields in order to produce film frames. For the case of NTSC to film, the process is known as a **2:3 pullup**.

If film is being shot with the sole intention of transferring it to video, then the film camera can often just be adjusted to shoot at the appropriate frame rate, 30 fps or 25 fps, and any pulldown conversions can therefore be avoided. Many film cameras can actually be set to shoot at exactly 29.97 fps to perfectly match NTSC's rate.

Plate 24 *Image resulting from mixing 75% of Plate 18a with 25% of Plate 18b.*

Plate 25 *Masking to control brightness. (a) Mask image.*

Plate 25 *(b) Test image (Plate 18a) after being color corrected using Plate 25a as the mask.*

Plate 26 *Premultiplication. (a) Foreground image of a dog.*

Plate 26 *(b) A soft-edged matte for the dog.*

Plate 26 *(c) Foreground premultiplied by the matte.*

Plate 27 *Background image.*

Plate 28 *Premultiplied image (Plate 26c) placed over the background image (Plate 27) in a system that assumes all images are premultiplied.*

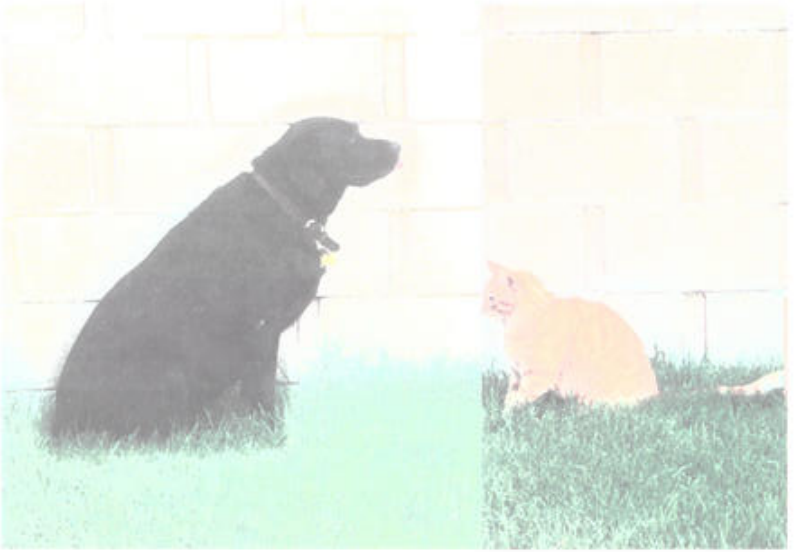


Plate 29 *Unpremultiplied image (Plate 26a) placed over the background image (Plate 27) in a system that assumes all images are premultiplied.*

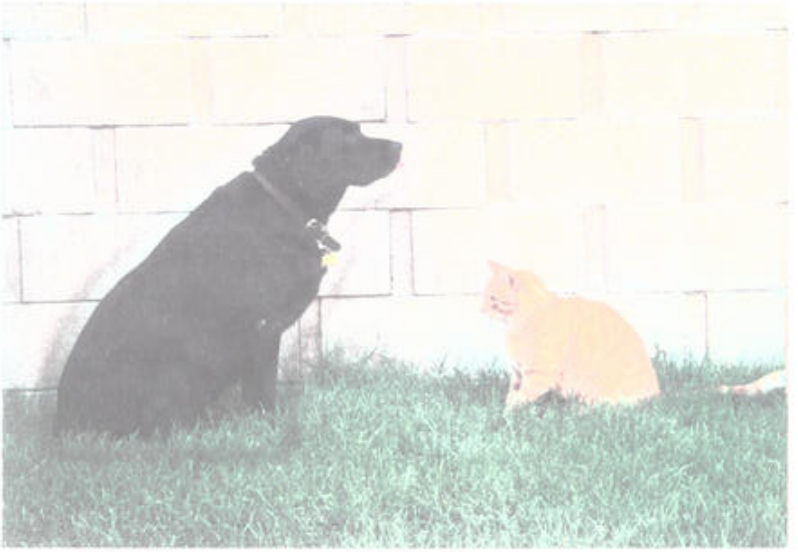


Plate 30 *A doubly premultiplied image composited over the background. A dark halo is produced along the matte's edge.*

Plate 31 *The result of redividing a premultiplied image (Plate 26c) by its own matte channel.*

Plate 32 *The luminosity operator applied to Plate 26c. The matte channel for the foreground image was multiplied by 0.6 before being placed over the background.*

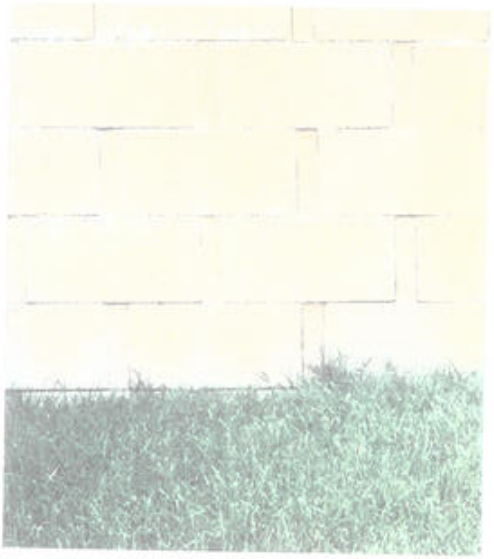


Plate 33 *Difference matting. (a) A clean plate of the background.*

Plate 33 *(b) The difference matte that is created by subtracting Plate 33a from Plate 26a.*

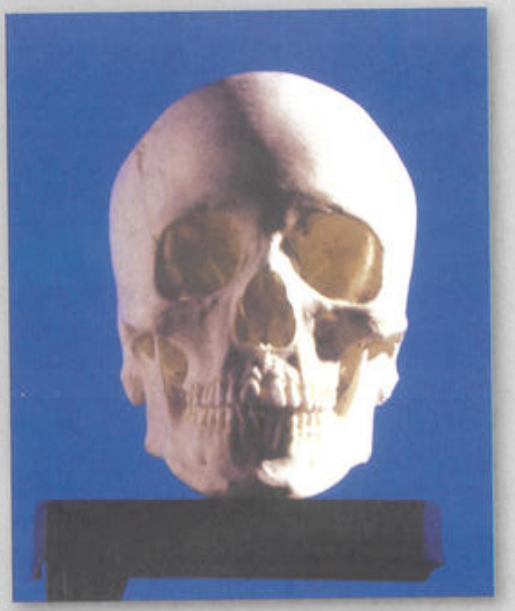


Plate 34 *Original bluescreen image.*

Plate 35 *The image from Plate 34 after applying spill suppression.*

Plate 36 *Plate 34 after matte extraction (reverse matte).*

Plate 37 *Data loss after applying two Brightness operators.*

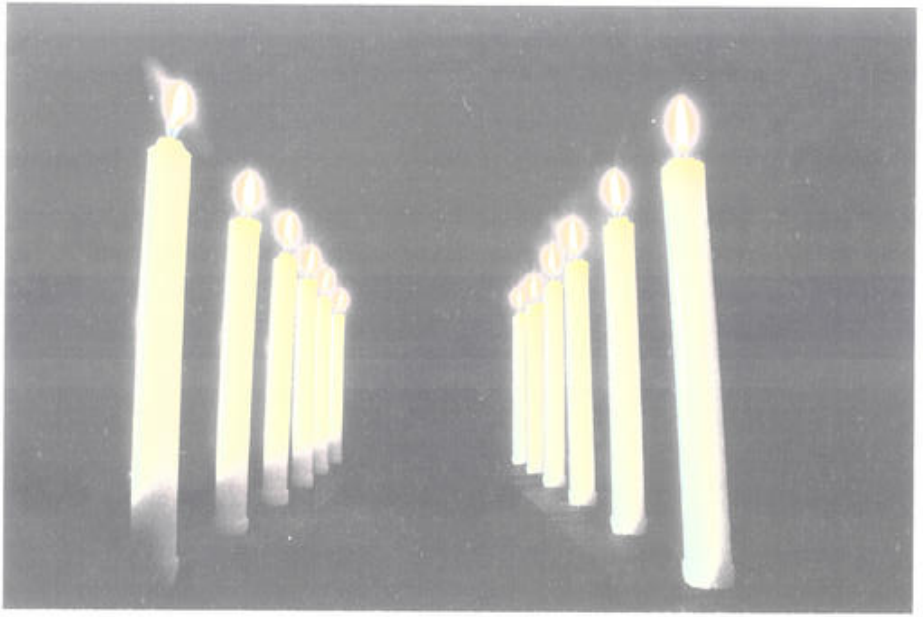


Plate 38 *Candles shot from a close-up distance using a wide-angle lens.*

Plate 39 *Candles shot from farther away using a telephoto lens.*

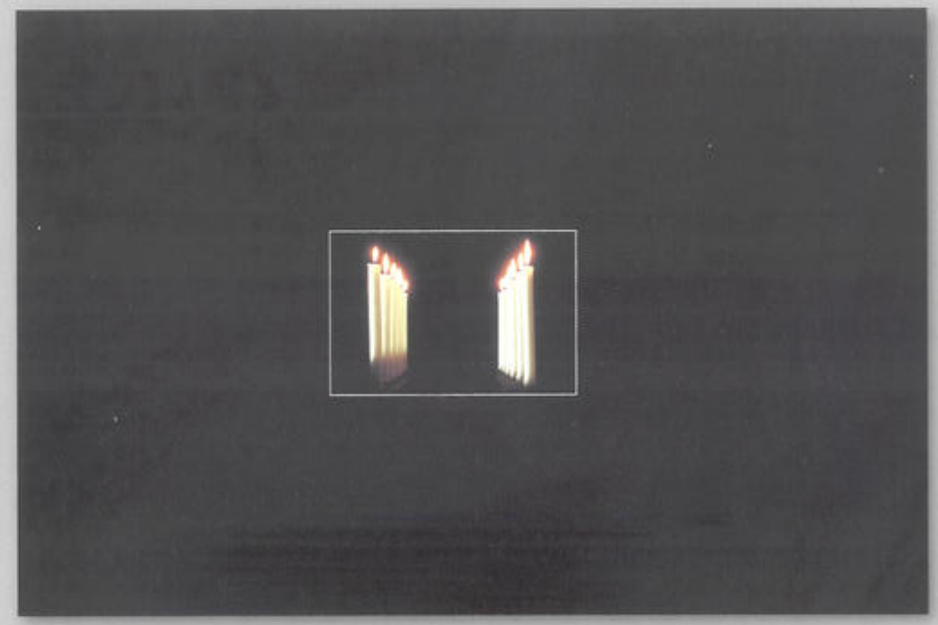


Plate 40 *Candles shot from the same position as those in Plate 39, but with the same lens as in Plate 38.*

Plate 41 *Depth of field. Only those objects within the depth of field are in focus.*

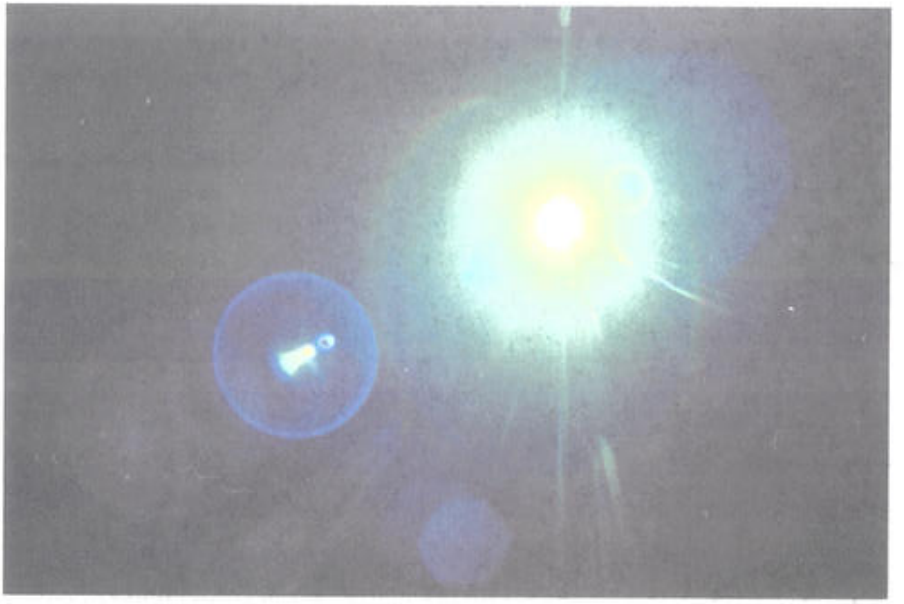


Plate 42 *Lens flares from a spherical lens.*

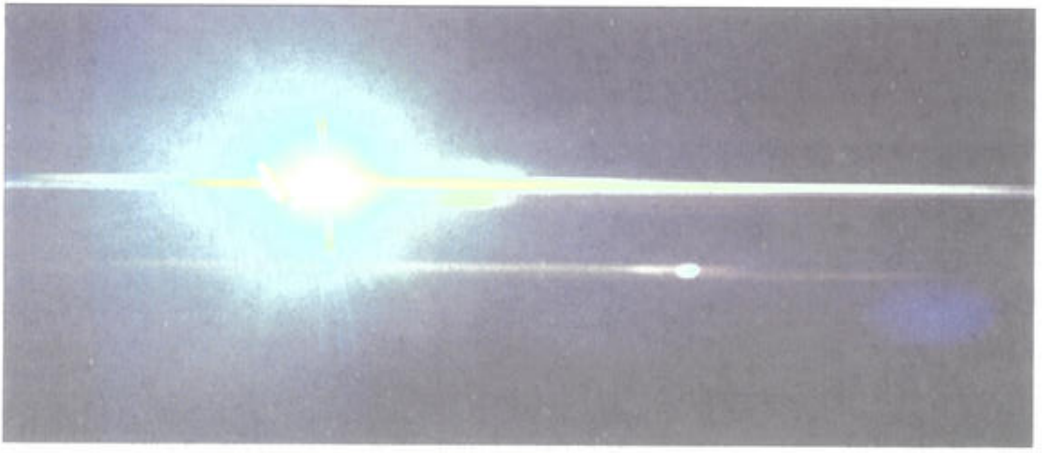


Plate 43 *Lens flare from an anamorphic lens (as seen when projected).*

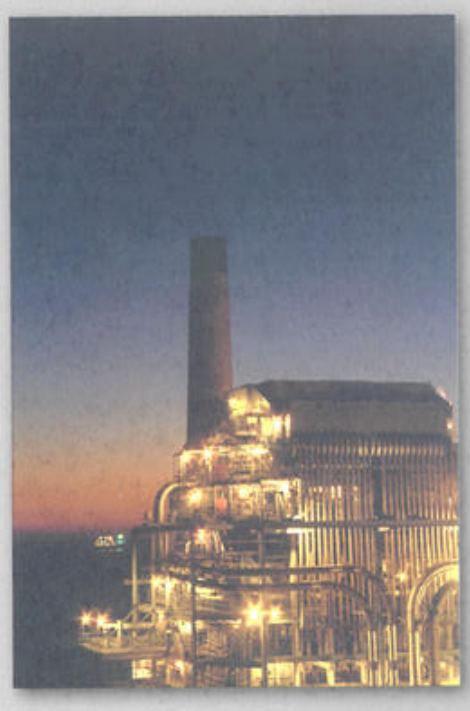


Plate 44 *Focus. (a) A well-focused scene.*

Plate 44 *(b) The out-of-focus scene.*

Plate 44 *(c) A Gaussian blur of the original scene.*

Plate 45 *Motion-blurred object.*

Plate 46 *Grain comparison. (a) Highly sensitive, low-light film stock (Kodak 5289).*

Plate 46 *(b) A slower-speed film stock (Kodak 5245).*

Plate 47 *The RGB channels of the film stock shown in Plate 46a. (a) Red channel.*

Plate 47 *(b) Green channel.*

Plate 47 *(c) Blue channel.*